Query Processing and Query Runtimes

A systems view of analytical query engines

October 2026

Contents

1. Query Processing Fundamentals 3

2. Execution Models 15

3. Velox Internals 28

4. Core Operators 42

5. Columnar Storage and Scans 53

6. Distributed Query Execution 64

7. BigQuery and Dremel 77

8. The Engine Landscape 90

9. Query Optimization 99

10. Query Processing for ML Data 112

11. GPU and Hardware-Accelerated Query Processing 122

Module 1 — Query Processing Fundamentals

Oct 5, 2026

Why this module

Every query engine, from DuckDB on a laptop to BigQuery on thousands of machines, runs the same five-stage pipeline: parse, bind, optimize, plan physically, execute. This module gives you that pipeline and its vocabulary so the rest of the series (execution models, Velox internals, operators, storage, distribution, Dremel) has a frame to hang on.

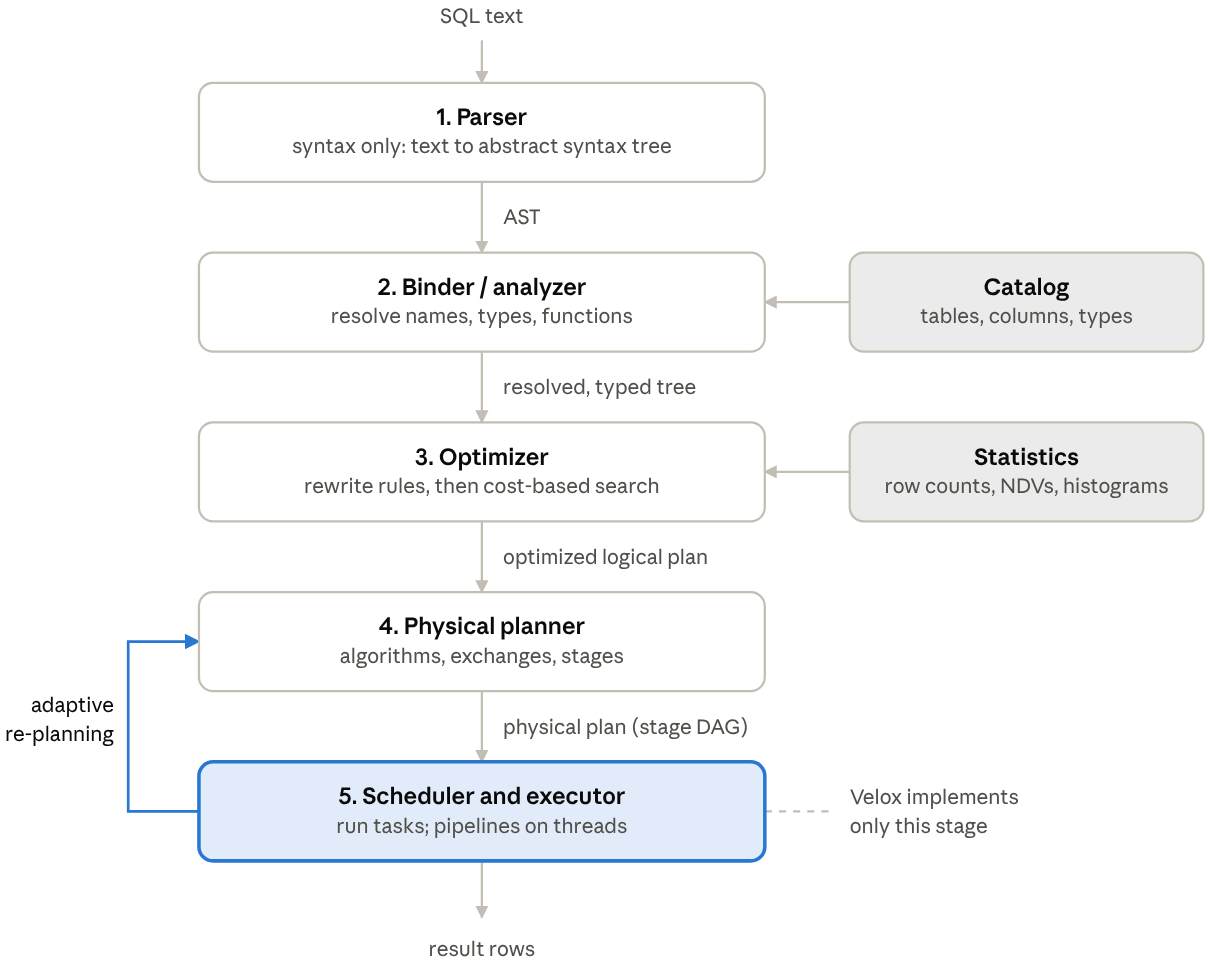
If you have used an execution library such as Velox, you know the bottom of the stack: a physical plan goes in, vectors come out. This module fills in everything above that plan and names the decisions that shape it.

How to read it: skim the pipeline diagram first, then read top-down. Sections 3 to 6 are one stage each. Section 8 traces one concrete query through all of them. Stop and ask whenever something is unclear; that is the point of the session.

The end-to-end pipeline

A query passes through five stages, each consuming the previous stage's output and producing a more concrete representation, from text to a tree to a plan to running code.

Every engine runs the same five stages; Velox is only the last



query pipeline · 5 stages, 2 inputs, 1 feedback loop

The first four stages run once, in milliseconds, on one coordinator. The fifth runs on every worker and is where nearly all of the time goes. The highlighted loop is what modern distributed engines add: the executor reports actual sizes back so the planner can change the rest of the plan mid-query.

Parsing and binding

Parsing checks syntax only; binding (also called analysis or semantic resolution) is where the query gains meaning, and it is where most real-world complexity lives.

Parsing turns SQL text into an abstract syntax tree (AST). Engines use a generated parser (ANTLR in Presto/Trino, a Bison grammar in PostgreSQL and DuckDB) or a hand-written recursive-descent one (ZetaSQL, which BigQuery uses). The AST knows that `t.a` is a qualified column reference, but not whether `t` exists.

Binding resolves every name against the catalog and produces a typed, resolved tree:

- **Name resolution**: tables, columns, aliases, CTEs, correlated outer references. `SELECT a FROM t1, t2` is an error if both have `a`.
- **Type checking and coercion**: `int32 + int64` becomes `CAST(a AS int64) + b`; implicit casts are inserted explicitly into the tree.
- **Function resolution**: picking the overload (`SUM(int64)` returns `int64`, `SUM(numeric)` returns `numeric`), distinguishing scalar vs. aggregate vs. window functions.
- **Semantic rules**: non-aggregated columns must appear in `GROUP BY`; `HAVING` may only reference groups; window frames are valid.
- **Desugaring**: `SELECT *` expands, `USING` joins become equality predicates, `IN (subquery)` becomes a semi-join marker, views are inlined.

The output is a resolved tree where every expression has a type and every column reference points to a unique column id (not a name). Unique ids matter: after optimization moves expressions around, names collide but ids do not.

BigQuery connection. BigQuery's front end is ZetaSQL (open-sourced as GoogleSQL), which outputs a *Resolved AST*. Spanner, F1 Query and BigQuery share it, which is why their SQL dialects match. If asked, name it: the analyzer and the dialect are a shared library across Google SQL systems.

Logical plans and relational algebra

A logical plan says *what* to compute as a tree of relational operators, with no commitment to *how*. Optimizers rewrite this tree because algebraic equivalences guarantee the rewrite returns the same rows.

The core operators and what they do to a stream of rows:

Operator	Symbol	Input to output	Notes
Scan	—	table to rows	leaf; carries the columns actually needed
Filter (selection)	σ	rows to subset of rows	predicate per row
Project	π	rows to rows with new columns	computes expressions, drops columns

Operator	Symbol	Input to output	Notes
Join	⋈	two inputs to combined rows	inner, left/right/full outer, semi, anti, cross
Aggregate	γ	rows to one row per group	GROUP BY keys + aggregate functions
Sort	τ	rows to ordered rows	often fused with LIMIT as Top-N
Limit	—	rows to first k rows	
Union / Except / Intersect	∪ − ∩	set operations	ALL vs DISTINCT variants
Window	—	rows to rows + window column	partition, order, frame

The equivalences that power most optimizations:

- **Filters commute with joins** when they reference one side only: σ_p(R ⋈ S) = σ_p(R) ⋈ S. This is predicate pushdown.
- **Inner joins are commutative and associative**: (R ⋈ S) ⋈ T = R ⋈ (S ⋈ T). This makes join reordering legal. Outer joins mostly are not, which is why they constrain the optimizer.
- **Projections commute with most operators**, so unused columns can be dropped early (column pruning).
- **Aggregation can be split**: SUM(x) = SUM(partial SUMs). This enables partial/final aggregation and pushing partial aggregates below joins or shuffles.

Plans are trees, sometimes DAGs. A CTE referenced twice, or a self-join on the same scan, creates shared subplans. Some engines duplicate the subtree; others (BigQuery, Snowflake) can materialize once and fan out. Hold this thought for Module 6.

Subqueries are the hardest part of the logical layer. A correlated subquery like `WHERE x > (SELECT AVG(y) FROM s WHERE s.k = t.k)` is naively a nested loop. *Decorrelation* (unnesting) rewrites it into a join with an aggregate. Neumann and Kemper's 2015 paper shows how to unnest any subquery; DuckDB implements it. Module 9 covers it in depth.

Query optimization

Optimization is two layers: rewrite rules that are almost always beneficial, then a cost-based search over choices that depend on data (join order, algorithms, distribution). The second layer is only as good as its cardinality estimates, and those are usually wrong.

Rule-based rewrites (heuristics, applied unconditionally)

- **Predicate pushdown**: move filters as close to scans as possible, and into the scan itself (min/max zone maps, partition pruning, Parquet row-group skipping).
- **Column pruning**: only read and carry columns that are used. In a columnar store this is the single biggest I/O win.
- **Constant folding and expression simplification**: `x + 0`, `1 = 1`, `CAST` of a literal.
- **Predicate inference**: from `a.k = b.k AND a.k = 5`, derive `b.k = 5` and push it to the other side.
- **Outer-to-inner join conversion**: a left join followed by a filter that rejects NULLs on the right side is an inner join.
- **Subquery decorrelation** (previous section), **view and CTE inlining**, **limit pushdown** through projections and union.

Cost-based optimization (CBO)

The optimizer enumerates alternative plans and picks the cheapest under a cost model. Three ingredients:

1. **Search space**: join orders (n! orderings; left-deep vs. bushy trees), join algorithms, aggregation strategies, where to put shuffles.
2. **Cost model**: estimated CPU, I/O, network and memory per operator, driven by row counts and row widths.
3. **Cardinality estimation**: row counts per intermediate result, from statistics (row counts, distinct counts via HyperLogLog, histograms, null fractions) and selectivity formulas.

Two classic search frameworks:

- **System R / bottom-up dynamic programming** (Selinger, 1979): best plan for every subset of relations, built from smaller subsets. DPccp and DPhyp (Moerkotte and Neumann) are the modern efficient versions; DuckDB and HyPer use them.

- **Volcano / Cascades** (Graefe, 1993/1995): top-down, rule-driven, memoized in a *memo* of equivalence groups. SQL Server, Apache Calcite, CockroachDB, Greenplum Orca use it. Strength: logical and physical rules in one extensible framework.

Why estimates fail. Selectivity formulas assume independence between predicates and uniformity within columns; real data has correlation and skew. Errors multiply across joins, so a 10x error per join becomes 1000x after three (Leis et al., "How Good Are Query Optimizers, Really?", VLDB 2015). This is the core argument for **adaptive execution**: decide join strategy or partition count at runtime once real sizes are known. BigQuery leans hard on this (Module 7), partly because it often lacks good statistics on freshly loaded or external data.

In a distributed engine, the optimizer cares less about exact join order than about *data movement*, choosing broadcast vs. shuffle and avoiding repeated shuffles, and it should be robust to bad estimates rather than trusting them.

Physical plans

The physical plan binds each logical operator to a concrete algorithm and, in a distributed engine, decides where data must move. Every choice trades CPU, memory and network against each other.

Logical operator	Physical choices	Decided by
Scan	full scan, index/zone-map scan, partition-pruned scan	predicates, storage layout
Join	hash join, sort-merge join, nested-loop (index) join	equi vs. non-equi, input sizes, sortedness, memory
Aggregate	hash aggregation, sort/streaming aggregation; single-step vs. partial + final	key cardinality, input already sorted by keys
Sort	in-memory sort, external merge sort, Top-N heap	input size, presence of LIMIT
Distribution (distributed only)	broadcast, hash repartition (shuffle), gather, co-located (no move)	sizes of inputs, existing partitioning

Physical properties drive the search: an operator *requires* properties of its input (sorted on k, hash-partitioned on k, single partition) and *delivers* properties to its parent. When the child does not deliver what the parent requires, the optimizer inserts an **enforcer**: a Sort for ordering, an Exchange for partitioning. In a distributed plan, Exchanges are where stage boundaries fall.

Join distribution, the decision you will discuss most:

- **Broadcast join**: copy the small side to every worker holding the big side. Network cost ≈ small size × workers. Good when one side fits in memory per worker.
- **Shuffle (partitioned) hash join**: hash-repartition both sides on the join key so matching rows meet. Network cost ≈ size of both sides. Scales to any size, but suffers from key skew.
- **Co-located join**: both sides already partitioned on the key (e.g. clustered tables); no movement.

Partial and final aggregation is the other workhorse: aggregate locally before the shuffle so far fewer rows cross the network, then merge partial states after it. Works when grouping-key cardinality is low relative to input; with near-unique keys the partial step is wasted CPU, so good engines disable it adaptively (Velox and Presto do this by measuring the reduction ratio at runtime).

Execution: fragments, pipelines, tasks

Execution cuts the physical plan twice: at **exchanges** into fragments (stages) that run on different workers, and at **pipeline breakers** into pipelines that run on threads inside a worker.

Fragments / stages. Each exchange splits the plan. Everything between two exchanges is one fragment, run as many parallel instances (tasks), each on a slice of the data. Presto calls them fragments and tasks; Spark calls them stages and tasks; Dremel calls them stages and workers in a query DAG.

Pipelines. Inside a fragment, an operator that must consume all of its input before producing output is a *pipeline breaker*: hash-join build, hash aggregation, sort. A pipeline is a chain of operators that can stream rows (or batches) from source to sink without materializing. A hash join therefore splits into a build pipeline (scan S, build hash table) and a probe pipeline (scan R, probe, continue upward), and the probe pipeline waits on the build.

Drivers / threads. A pipeline runs as several parallel instances, each a single-threaded loop that pulls a batch from its source, pushes it through each operator, and hands it to the sink. Velox calls these Drivers; HyPer and DuckDB schedule small chunks of input (*morsels*) to worker threads. This is Module 2 and 3 material.

The batch unit matters more than it looks: tuple-at-a-time (classic Volcano) spends most of its time in virtual calls; vector-at-a-time (Velox, DuckDB, BigQuery) amortizes that over ~1,000 rows and enables SIMD; compiled engines (HyPer) fuse a whole pipeline into one tight loop. Module 2 covers this trade-off in depth.

The vocabulary differs per engine; the concepts do not:

Concept	Velox	Presto / Trino	Spark	BigQuery / Dremel
Plan unit across machines	PlanFragment	Fragment	Stage	Stage
Parallel instance of it	Task	Task	Task	Worker / slot work unit
Data movement	Exchange / PartitionedOutput	Exchange	Shuffle	Shuffle (in-memory tier)
In-worker streaming chain	Pipeline	Pipeline	Whole-stage codegen block	Pipeline
Single-threaded executor	Driver	Driver	Task thread	Thread
Batch of rows	RowVector	Page	ColumnarBatch / rows	Columnar batch

Worked example: one query, every stage

The best plan for this query shuffles about 50 GB instead of broadcasting 1 TB, and pre-aggregating before the join can cut that further. Sizes are invented but realistic: `orders` has 10B rows, `customers` 50M rows, there are about 200 countries.

```
SELECT c.country, SUM(o.amount) AS revenue
FROM orders o JOIN customers c ON o.cust_id = c.id
WHERE o.order_date >= '2026-01-01'
GROUP BY c.country
ORDER BY revenue DESC
LIMIT 10;
```

1. Parse and bind. `c.country` resolves to column id #7, `o.amount` to #3 (NUMERIC). The string literal is coerced to DATE. `SUM(NUMERIC)` resolves to an aggregate returning NUMERIC. The `GROUP BY` rule passes: `c.country` is grouped.

2. Logical plan, after rewrites.

```
Limit 10
  Sort revenue DESC               -- fused with Limit into TopN(10)
    Aggregate [country] SUM(amount)
      Join o.cust_id = c.id
        Scan orders [cust_id, amount]  filter: order_date >= 2026-01-01
(pushed into scan)
        Scan customers [id, country]
```

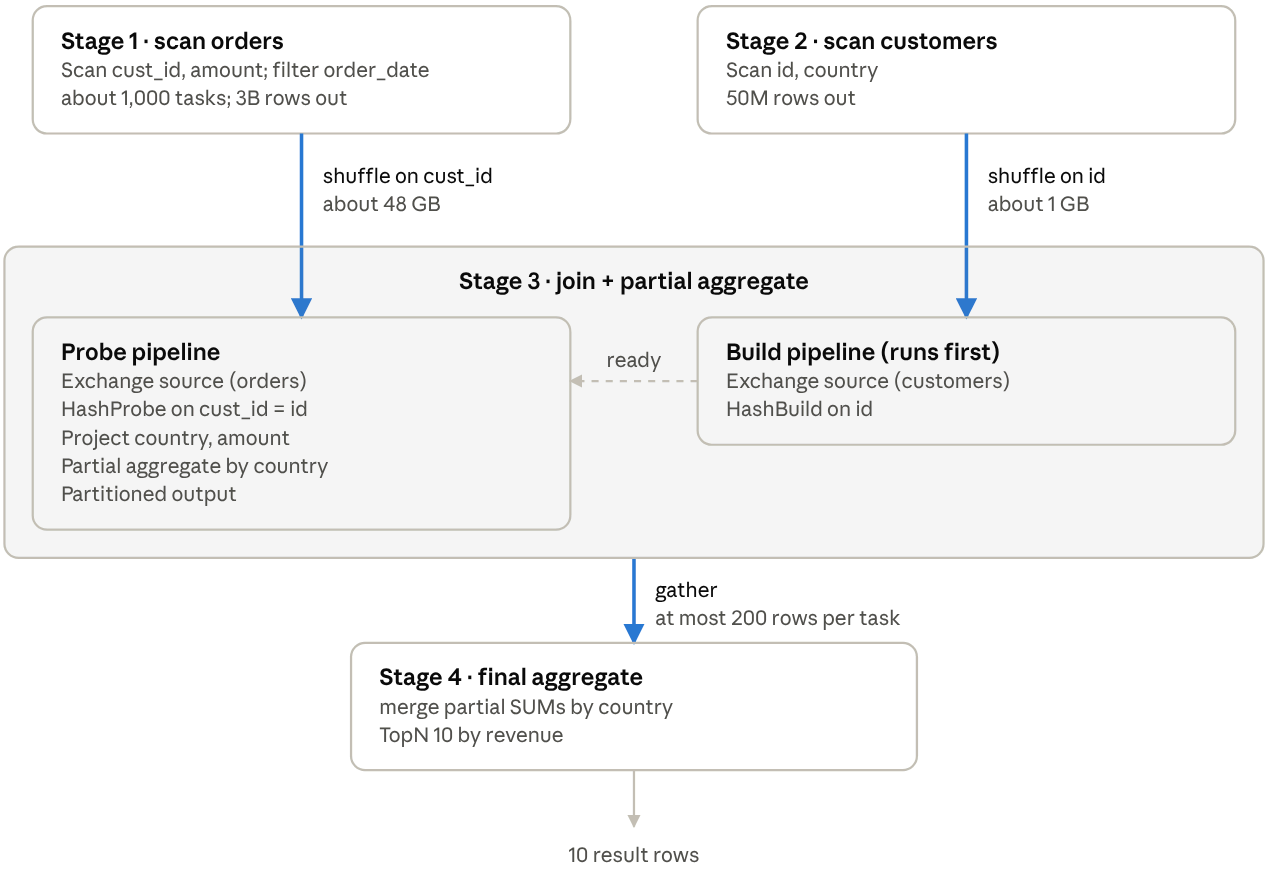
Column pruning dropped every unused column, so a columnar scan reads 3 of maybe 40 columns of `orders`. If `orders` is partitioned by `order_date`, partition pruning skips older partitions entirely. Say 3B rows survive.

3. Cost-based choices.

- *Join distribution.* Pruned `customers` is about 50M × 20 B = 1 GB. Broadcasting to 1,000 workers moves about 1 TB. Shuffling both sides moves about 3B × 16 B + 1 GB = 49 GB. Shuffle wins clearly.
- *Build side.* Build the hash table on `customers` (smaller); probe with `orders`.
- *Aggregation.* Grouping by country has about 200 groups, so partial aggregation after the join collapses each worker's output to at most 200 rows. The final aggregate then runs on one task, followed by a TopN.
- *Stronger alternative (eager aggregation).* Aggregate `orders` by `cust_id` *before* the join: 3B rows become at most 50M (cust_id, partial sum) rows, which also makes the shuffle much smaller. This is valid because the join is on the grouping key of the pushed-down aggregate. Good optimizers consider it.

4. Physical plan into stages and pipelines. Each exchange creates a stage boundary; inside Stage 3, the hash join splits the work into a build pipeline and a probe pipeline. The diagram below shows the shuffle-join version.

Each exchange starts a stage; the hash join splits Stage 3 in two



worked example · 4 stages, 3 exchanges, 2 pipelines in Stage 3

Highlighted arrows are exchanges, where data crosses the network. Only the two shuffles into Stage 3 are large; partial aggregation makes the final gather tiny.

5. Execution. Stages 1 and 2 run in parallel and write into the shuffle. Stage 3 tasks first drain their `customers` partition into a hash table, then stream `orders` batches through probe, project and partial aggregate. Stage 4 merges about 1,000 × 200 partial rows and keeps the top 10.

What can go wrong: one `cust_id` holding 20% of orders overloads one Stage 3 task (skew); the build side not fitting in memory (spill or grace hash join); the 50M estimate being wrong by 100x (adaptive switch to broadcast or more partitions). Modules 4 and 6 cover each.

How this maps to Velox and BigQuery

Velox covers only the bottom of the pipeline, physical execution on one node, while BigQuery owns all five stages plus distributed scheduling and storage. BigQuery's own runtime sits at the level Velox occupies, but inside a system that controls everything above and around it.

Velox: execution only, by design.

- No parser, binder or optimizer. The host engine hands Velox an already-physical plan: Presto C++ (Prestissimo) translates Presto fragments, Gluten translates Spark plans via Substrait, DPP built its own.
- The plan is a tree of `core::PlanNode` objects (TableScan, Filter, Project, HashJoin, Aggregation, OrderBy, PartitionedOutput, Exchange, ...). A `Task` executes one plan fragment.
- The `LocalPlanner` cuts the fragment into pipelines at pipeline breakers and instantiates `Driver`s, each holding a chain of `Operator`s. Exactly the model in the Execution section.
- Velox has no catalog and no network transport of its own: scans go through pluggable connectors (Hive, etc.), and Exchange relies on the host to move data between tasks.

BigQuery: everything, plus disaggregation. From public sources (the 2010 Dremel paper and the 2020 "Decade of Interactive SQL Analysis" paper):

- **Front end**: ZetaSQL/GoogleSQL parser and analyzer produce a resolved AST, then BigQuery's planner and optimizer produce a stage DAG.
- **Execution**: a coordinator schedules stages onto workers; compute capacity is measured in *slots*. The original Dremel used a fixed serving tree (root, intermediate servers, leaves); it was replaced by a general DAG of stages connected by shuffle.
- **Shuffle**: a distributed in-memory shuffle tier, separate from the workers. Disaggregating shuffle memory from compute lets producers and consumers run independently and lets the system re-plan between stages.
- **Dynamic query execution**: because the shuffle materializes stage outputs, the coordinator can change the plan mid-query: number of partitions, join strategy (broadcast vs. shuffle), handling skew. This is the practical answer to bad cardinality estimates.
- **Storage**: tables live in Colossus (Google's distributed file system) in the Capacitor columnar format; compute is stateless and scales independently.

A caveat on the vocabulary table in the Execution section: BigQuery's internal names for pipelines, threads and batches are not public. That column uses the general concepts; Stage, slot and shuffle are the terms Google itself uses.

Self-test and reading list

If you can answer these out loud in two minutes each, Module 1 is done.

1. Why does binding assign column ids instead of keeping column names?
2. Give two rewrites that are always safe and one that requires cost information.
3. Why can't the optimizer freely reorder a LEFT JOIN with an inner join? Give a counterexample.
4. When is partial aggregation before a shuffle a bad idea, and how can an engine detect that at runtime?
5. Broadcast vs. shuffle join: write the network cost of each in terms of |R|, |S| and the number of workers W.
6. In the worked example, why is eager aggregation by `cust_id` valid, and when would it be invalid?
7. Where are the pipeline breakers in a plan with a hash join, a GROUP BY and an ORDER BY … LIMIT?
8. Why does a disaggregated shuffle make adaptive re-planning easier?

Reading list, in priority order (skim; the abstracts and architecture sections are enough for now):

1. Melnik et al., "Dremel: A Decade of Interactive SQL Analysis at Web Scale", VLDB 2020. The most important paper for understanding BigQuery.
2. Pedreira et al., "Velox: Meta's Unified Execution Engine", VLDB 2022.
3. Leis et al., "How Good Are Query Optimizers, Really?", VLDB 2015.
4. Graefe, "The Cascades Framework for Query Optimization", IEEE Data Eng. Bull. 1995.
5. Neumann and Kemper, "Unnesting Arbitrary Queries", BTW 2015.
6. Melnik et al., "Dremel: Interactive Analysis of Web-Scale Datasets", VLDB 2010 (for history and the serving tree).

Next: Module 2, execution models (Volcano, vectorized, compiled, morsel-driven).

Module 2 — Execution Models

Oct 6, 2026

Why this module

Module 1 ended with a physical plan cut into stages and pipelines. This module answers the next question: once a pipeline runs on a thread, how does data actually flow through its operators, and what makes that fast or slow on a modern CPU?

There are three answers in the literature, and every engine you will be asked about picks one: **tuple-at-a-time iterators** (Volcano), **vectorized** batch processing (MonetDB/X100, Velox, DuckDB, Photon), and **compiled** pipelines (HyPer, Umbra, Spark's whole-stage codegen). A separate, orthogonal choice is how the work is spread across threads: **exchange-based** parallelism or **morsel-driven** scheduling.

The goals of this module are to explain *why* vectorization works (it is a CPU-efficiency argument, not a big-data one), to compare it honestly with compilation, and to describe Velox's Driver loop precisely enough that you could sketch it on a whiteboard.

The core problem: interpretation overhead

A query engine is an interpreter: the plan is a data structure, and the engine walks it to decide what to do with each value. The useful work per value is tiny (an add, a compare, a hash), so the cost of *deciding* what to do easily exceeds the cost of *doing* it.

What a modern CPU needs to run fast, and what interpretation breaks:

- **Pipelining and branch prediction.** A core executes several instructions per cycle only if it can predict branches. A mispredict costs about 15 to 20 cycles. Virtual calls through a function pointer chosen per operator, and type switches per value (`if type == INT64 ... else if DOUBLE ...`), are hard to predict when mixed.
- **Instruction and data cache.** Jumping between many operators' code per row thrashes the instruction cache; touching one value from many columns per row wastes data-cache lines.
- **SIMD.** One AVX2 instruction adds 4 int64 or 8 int32 values; AVX-512 doubles that. A compiler can only auto-vectorize a tight loop over an array, never code that handles one value per call.

- **Memory-level parallelism.** A cache miss to DRAM costs ~100 ns. Hardware hides it only when it can issue many independent loads at once, e.g. probing 1,000 hash-table slots in a loop rather than one probe per call chain.

The MonetDB/X100 paper (2005) measured this on TPC-H Query 1 in a classic row-at-a-time engine: the vast majority of time went to interpretation overhead (function calls, tuple navigation), with only a small fraction spent on the actual arithmetic. Every model below is a way to amortize or eliminate that overhead.

In one line: **amortize interpretation over many values (vectorization) or compile it away (code generation).**

Volcano: tuple-at-a-time iterators

Volcano (Graefe, 1990/1994) gives every operator the same three-method interface and composes them into a tree. The root pulls rows by calling `next()` on its child, which calls `next()` on its child, recursively down to the scan.

```
struct Operator {
  virtual void open() = 0;
  virtual std::optional<Row> next() = 0;   // one row, or end
  virtual void close() = 0;
};

struct Filter : Operator {
  Operator* child; Expr* pred;
  std::optional<Row> next() override {
    while (auto r = child->next())          // virtual call per row
      if (pred->eval(*r).asBool())          // interpreted expression tree
per row
        return r;
    return std::nullopt;
  }
};
```

Why it won for 30 years: every operator is independent and composable; the interface handles any plan shape; pipelining falls out naturally (a row flows to the root without being materialized); memory use is minimal. PostgreSQL, MySQL, SQL Server's classic row mode and early Presto all use it.

Why it is slow on modern CPUs: per row and per operator, there is at least one virtual call, and the expression is itself an interpreted tree (`Add(Col(3), Const(1))`) with a virtual call per node and a type dispatch per value. For a scan, filter, project, aggregate pipeline, that is easily dozens of indirect calls to do a handful of arithmetic instructions, with no chance of SIMD.

The Volcano exchange operator (Graefe, SIGMOD 1990) is the other lasting contribution: parallelism is hidden inside a special operator that looks like any other iterator but internally runs its subtree on other threads or machines and repartitions the data. The operators themselves stay single-threaded. Every Exchange in Module 1 descends from this idea.

Materialization, the opposite extreme. MonetDB (pre-X100) went the other way: each operator processes an *entire column* and writes a full intermediate column before the next operator starts. No per-row overhead and perfect tight loops, but intermediates of billions of values spill out of cache into memory, so the engine becomes memory-bandwidth bound. Vectorization is the middle ground between the two.

Vectorized execution

Vectorized execution (Boncz, Zukowski, Nes: MonetDB/X100, CIDR 2005; later VectorWise) keeps the Volcano interface but passes a **batch of ~1,000 rows in columnar form** per call. Interpretation overhead is paid once per batch instead of once per row, and the inner work becomes tight loops over arrays.

```
// One "primitive": a type-specialized loop, chosen ONCE per batch, not per
value.
int selectLessThanI64(const int64_t* col, int64_t c, int n,
                      const int32_t* selIn, int32_t* selOut) {
  int k = 0;
  for (int i = 0; i < n; ++i) {
    int32_t row = selIn[i];
    selOut[k] = row;
    k += (col[row] < c);          // branch-free: no mispredicts
  }
  return k;                       // number of surviving rows
}
```

The ideas that make it work:

- **Batch size is tuned to the cache.** ~1,000 to 4,000 values per column fit in L1/L2 together with the other columns in flight. Too small brings back overhead; too large (the MonetDB extreme) falls out of cache. Velox and DuckDB default to about 1,000 to 2,048 rows.
- **Primitives are pre-compiled per type combination.** `add<int64,int64>`, `lt<double,const>`, etc. are generated with templates or macros, so the compiler sees a tight typed loop and can auto-vectorize with SIMD.
- **Selection vectors instead of copying.** A filter does not compact the batch; it produces a list of surviving row indexes, and downstream primitives iterate only those. Velox's equivalent is `SelectivityVector` (a bitmap of active rows).
- **Branch-free code where selectivity is unpredictable.** The `k += (cond)` trick above trades a branch for always writing; it wins when selectivity is near 50% and loses when almost nothing or almost everything passes. Adaptive engines pick per batch.
- **Memory-level parallelism comes for free.** A hash probe primitive loops over 1,000 keys: hash all, then prefetch all buckets, then compare. Independent loads overlap, hiding DRAM latency. This is the biggest practical win for joins and aggregations.
- **Operating on encoded data.** Since a batch is columnar, it can stay dictionary- or run-length-encoded: compare a predicate once per dictionary entry instead of once per row. Velox's vector encodings (Module 3) exist for exactly this.

What it costs: a pipeline of operators still materializes each intermediate column per batch (into cache, not memory, so cheap but not free), and each expression node is still a separate loop over the batch, so a complex expression makes several passes over the data instead of one.

Compiled, data-centric execution

HyPer (Neumann, "Efficiently Compiling Efficient Query Plans for Modern Hardware", VLDB 2011) removes interpretation entirely: at query time, it generates machine code for each **pipeline** as a single loop, with all operators of that pipeline fused into it. A row is loaded into registers once and stays there until it hits a pipeline breaker.

For `SELECT country, SUM(amount) FROM orders WHERE date >= X GROUP BY country`, the generated code is essentially what you would write by hand:

```
// Pipeline: scan -> filter -> aggregate (build side of the aggregation)
for (size_t i = 0; i < n; ++i) {              // one loop, no virtual calls
  if (date[i] < X) continue;                  // filter, in a register
  auto& slot = agg.findOrInsert(country[i]);  // hash table insert (pipeline
breaker)
  slot.sum += amount[i];
}
```

Produce/consume model. The code generator walks the operator tree: each operator's `produce()` asks its child to generate the loop, and its `consume()` emits the code that handles one row inside that loop. Data flow is effectively **push-based**: the scan's loop pushes each row up through the fused operators.

Strengths

- Minimal instructions per row; values stay in registers across operators, with no intermediate columns at all.
- Excellent for computation-heavy pipelines (complex expressions, arithmetic, many predicates).

Costs

- **Compile latency.** LLVM optimization takes tens to hundreds of milliseconds per query, which dominates for short queries. HyPer mitigates with an LLVM-bytecode interpreter that starts immediately and switches to compiled code when it is ready (Kohn, Leis, Neumann, ICDE 2018). Umbra (CIDR 2020) uses a custom low-latency backend.
- **Engineering cost.** You are debugging a code generator; profiles point at generated code with no symbols; every operator must be written twice (as generator logic, and conceptually as runtime code).
- **Weak memory-level parallelism.** A tight row-at-a-time loop issues one hash probe, waits for the cache miss, then the next. Compiled engines have to add explicit batching or prefetching to match vectorized probes.
- **Hard to adapt mid-query.** Changing strategy (filter order, partial aggregation on/off) means recompiling.

Spark SQL's **whole-stage codegen** (Tungsten, 2016) applies the same idea by generating Java source per stage, compiled with Janino and then JIT-compiled by the JVM.

Vectorized vs. compiled

The head-to-head study is Kersten et al., "Everything You Always Wanted to Know About Compiled and Vectorized Queries But Were Afraid to Ask" (VLDB 2018). They built two engines, Typer (compiled, HyPer-style) and Tectorwise (vectorized, VectorWise-style), with identical algorithms, and ran TPC-H on both. Neither model dominated:

- **Compiled wins on computation**: arithmetic-heavy queries like TPC-H Q1 ran faster because values stay in registers and no intermediate vectors are written.
- **Vectorized wins on memory stalls**: join-heavy queries with large hash tables ran faster because batched probes issue many cache misses in parallel.
- **SIMD mattered less than expected** outside selective scans and a few primitives.
- Overall differences were within small factors either way; **both beat Volcano by an order of magnitude**.

So the industry choice is mostly about engineering, and recent systems have overwhelmingly picked vectorization:

Engine	Model	Note
Velox	Vectorized	Experimental codegen existed early and was dropped
DuckDB	Vectorized, push-based	Moved from pull to push pipelines in 2021
Databricks Photon	Vectorized (C++)	Chose it over Spark's JVM codegen, partly for debuggability and per-batch adaptivity (SIGMOD 2022)
Snowflake	Vectorized, push-based	Per the SIGMOD 2016 paper
ClickHouse	Vectorized	Optional JIT for expressions only
HyPer, Umbra	Compiled	Research and Tableau (HyPer)
Spark SQL	Compiled (whole-stage codegen)	Java source, JVM JIT
Amazon Redshift	Compiled	Generates and caches C++ per query segment

Arguments that win in practice, and that Photon's authors made explicitly:

1. **Observability.** Each operator and primitive is real code with symbols; per-operator timings and row counts come for free. That matters enormously when running customer workloads you cannot see.
2. **Adaptivity per batch.** A vectorized engine can check "no nulls in this batch" or "all ASCII strings" and switch to a faster primitive, batch by batch. Velox does exactly this.
3. **No compile latency** for short interactive queries.
4. **Development speed.** Primitives are ordinary C++ that ordinary engineers can write, test and profile.

In short: both are an order of magnitude better than Volcano, the measured differences are workload-dependent, and production engines choose vectorization mainly for observability, adaptivity and engineering velocity, sometimes adding JIT for hot expressions.

Push vs. pull, and parallelism

Two independent choices: who drives data through a pipeline (pull from the top, or push from the source), and how a pipeline's work is spread over threads (fixed partitioning via exchanges, or dynamic work units called morsels).

Pull vs. push

- **Pull (Volcano)**: the consumer calls `next()` on its producer. Control flow follows the tree; easy to write, and LIMIT naturally stops pulling.
- **Push (HyPer, DuckDB since 2021, Snowflake)**: the source loops and pushes each batch into its consumer, which pushes onward. The pipeline becomes a flat loop owned by a scheduler, not a call stack.

Push wins when a pipeline is a schedulable unit. Because control lives in one loop, the scheduler can pause a pipeline between batches (a blocked exchange, a full output buffer, a time slice) without unwinding a call stack, and one source can feed multiple consumers (shared scans). DuckDB's 2021 switch from pull to push was motivated by exactly this: better scheduling, pausable pipelines, simpler parallelism. Velox sits in between, as the next section shows.

Exchange-based parallelism (plan-driven)

In the classic Volcano style, the optimizer fixes a degree of parallelism (DOP) per stage, inserts exchanges, and each thread runs one copy of the pipeline on a static partition of the data.

Problem: static partitions mean static load balance. One slow or skewed partition leaves other threads idle; DOP chosen at plan time cannot react to the machine's current load.

Morsel-driven parallelism (scheduler-driven)

Leis et al., "Morsel-Driven Parallelism" (SIGMOD 2014, from HyPer): split the input into small **morsels** (~100,000 rows). A pool of worker threads, one per core, repeatedly grabs the next morsel of the current pipeline from a shared dispatcher and runs the whole pipeline on it.

- **Load balance is automatic**: fast threads simply take more morsels; skew in processing cost evens out.
- **Elastic**: the number of threads working on a query can change mid-query, which lets several queries share a machine fairly.
- **NUMA-aware**: the dispatcher prefers morsels in the thread's local memory socket.
- **Shared data structures** replace per-thread partitions: e.g. a single parallel hash table built by all threads, then probed by all threads.

DuckDB, Umbra and (partly) Photon follow this model. Distributed engines use both: exchanges *between* machines (data must move anyway), and morsel- or split-based dynamic scheduling *within* a machine. In Velox, splits added to a task play the morsel role for scans: drivers pull the next split when they finish one.

How Velox executes

Velox is vectorized and columnar, and each pipeline instance (a Driver) is a flat loop that moves batches between adjacent operators. When an operator cannot make progress, the Driver gives up its thread and returns later. No codegen; adaptivity happens per batch.

The operator interface

Unlike Volcano's single `next()`, a Velox operator exposes both directions, so the Driver can move data without recursion:

```
class Operator {
  virtual bool needsInput() const;                 // can I accept a batch
now?
  virtual void addInput(RowVectorPtr input);       // push a batch in
  virtual RowVectorPtr getOutput();                // pull a batch out (may
be null)
  virtual void noMoreInput();                      // upstream is finished
  virtual BlockingReason isBlocked(ContinueFuture* future);
  virtual bool isFinished();
};
```

The Driver loop (simplified)

```
// One Driver = one thread-at-a-time instance of a pipeline: ops[0] is the
source.
StopReason Driver::runInternal() {
  for (;;) {
    for (int i = 0; i < ops.size() - 1; ++i) {
      auto* op = ops[i]; auto* next = ops[i + 1];
      if (op->isBlocked(&future) != kNotBlocked ||
          next->isBlocked(&future) != kNotBlocked)
        return kBlocked;                           // park; resume when future
fires
      if (next->needsInput()) {
        if (auto batch = op->getOutput()) {        // pull from upstream...
          next->addInput(std::move(batch));        // ...push into downstream
          continue;
        }
        if (op->isFinished()) next->noMoreInput();
      }
    }
    if (allFinished()) return kAtEnd;
    if (timeSliceExpired()) return kYield;         // fairness between queries
  }
}
```

So Velox is **pull at each operator boundary, but driven by a flat loop**, which gives it the pausability of push-based engines. A Driver that returns `kBlocked` frees its thread; the future it handed back (split arrived, exchange data arrived, hash table built, memory granted, consumer drained) re-enqueues it on the executor (a folly thread pool) when it completes.

How pipelines coordinate: the hash join

A hash join creates two pipelines in the same Task. Each build-pipeline Driver runs a `HashBuild` operator that builds a partial table; the last one to finish merges them and publishes the table through a **join bridge**. Every probe-pipeline Driver's `HashProbe` reports `kWaitForJoinBuild` until the bridge has the table, then probes batches in parallel against the one shared table. This is the morsel-model idea (shared structures, many threads) inside a Task.

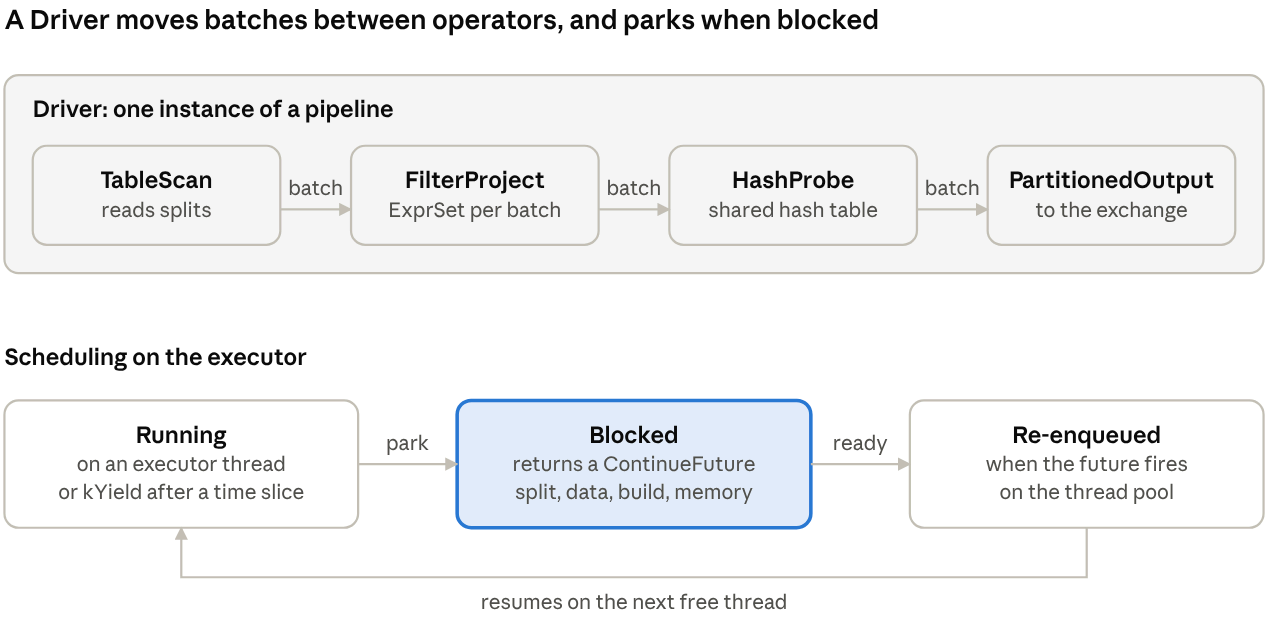
Expression evaluation

`Filter` and `Project` are fused into one `FilterProject` operator, which runs a compiled `ExprSet`:

- **At compile time**: common subexpressions are shared, constants are folded, and AND/OR conjuncts are flattened.
- **At eval time**, an expression is evaluated on a `SelectivityVector` (bitmap of active rows) and writes into a result vector. After a filter, only surviving rows are active; no compaction needed.
- **Encoding-aware**: `DecodedVector` gives uniform access over flat, dictionary or constant inputs, and *peeling* lets a function run once per dictionary value instead of once per row.
- **Fast paths per batch**: if a batch has no nulls, null-handling is skipped; ASCII-only strings take faster string paths; functions with default null behavior compute the null mask once instead of per function.
- **Adaptive conjunct ordering**: an AND tracks each conjunct's selectivity and cost and reorders them so cheap, selective predicates run first.

Late materialization in scans

`TableScan` pushes filters into the file reader and returns columns not needed by those filters as **lazy vectors**: they are decoded only when a downstream operator touches them, and only for rows that survived. On selective queries this skips most of the decoding work. Module 5 goes deeper.



Velox Driver · batch flow and thread lifecycle

A blocked Driver holds no thread, so a worker can run thousands of Drivers on a handful of threads; that is what lets one Velox process serve many concurrent queries.

What is known about BigQuery's execution model

Google has published far less about BigQuery's per-worker execution engine than about its distributed architecture, but what it has published since 2025 confirms that the engine is vectorized and encoding-aware: the same design family as Velox.

What BigQuery shows you directly. The query plan explanation (in the console and in `INFORMATION_SCHEMA.JOBS`) lists stages, and within each stage a sequence of **steps**: READ, COMPUTE, FILTER, JOIN, AGGREGATE, SORT, LIMIT, ANALYTIC_FUNCTION, WRITE (to shuffle or output). Per stage it reports wait, read, compute and write time, records in and out, and the number of parallel work units. That maps directly onto Module 1: a stage is a fragment, the steps are its operators, and WRITE is the producer side of an exchange.

The advanced runtime. Google's documentation describes BigQuery's execution as vectorized: columns processed in blocks sized to the CPU cache, using SIMD instructions. Its 2025 *advanced runtime* extends this with *enhanced vectorization*: filters evaluated directly on Capacitor's encoded data, encodings propagated through the query plan so data stays encoded longer, expression folding of deterministic and constant expressions, and multi-threaded hash-join build and probe inside a worker. Capacitor's own filter

pushdown uses the same vectorized engine. Google rolled the advanced runtime out as the default for all projects between September 2025 and early 2026. Module 7 covers it with the rest of BigQuery.

Related Google work. The Procella paper (VLDB 2019, YouTube's analytics engine) describes *Superluminal*, a vectorized, block-oriented evaluation engine with template-specialized primitives that operates directly on encoded data. It is the most detailed public description of a Google SQL engine's pipeline execution.

Open design questions. The public material leaves several runtime decisions unexplained, and they are good questions to think through for any engine:

- How does the runtime decide batch size and when to keep data encoded vs. decode it?
- Is there any JIT or code generation, e.g. for expressions or UDFs?
- How is work inside a stage scheduled across threads, and how does it interact with slots?
- How does the runtime handle memory pressure: spilling, or relying on the shuffle tier?

Module 7 puts these pieces together with what Google has published about the rest of BigQuery's architecture.

Self-test and reading list

1. In one sentence each: why is Volcano slow, and why is full-column materialization slow?
2. Why is a batch size of ~1,000 rows better than 10 or 10 million?
3. What is a selection vector, and why is it cheaper than compacting the batch after a filter?
4. Write a branch-free filter primitive. When does the branching version win?
5. Why do vectorized hash probes hide memory latency better than a compiled row-at-a-time loop? How could a compiled engine fix that?
6. Give three reasons a production engine might choose vectorization over compilation, besides raw speed.
7. What does push-based execution make easier for a scheduler? Where does Velox sit between push and pull?
8. Explain morsel-driven parallelism and how it handles skew in processing cost. What does it *not* fix?
9. Walk through what happens in Velox when a HashProbe driver starts before the build side is finished.

10. What is a lazy vector, and when does it save work?

Reading list, in priority order:

1. Kersten et al., "Everything You Always Wanted to Know About Compiled and Vectorized Queries But Were Afraid to Ask", VLDB 2018.
2. Boncz, Zukowski, Nes, "MonetDB/X100: Hyper-Pipelining Query Execution", CIDR 2005.
3. Leis et al., "Morsel-Driven Parallelism: A NUMA-Aware Query Evaluation Framework for the Many-Core Age", SIGMOD 2014.
4. Behm et al., "Photon: A Fast Query Engine for Lakehouse Systems", SIGMOD 2022 (the clearest industry argument for vectorization).
5. Neumann, "Efficiently Compiling Efficient Query Plans for Modern Hardware", VLDB 2011.
6. Chattopadhyay et al., "Procella: Unifying Serving and Analytical Data at YouTube", VLDB 2019 (Superluminal section).
7. Velox source: `velox/exec/Driver.cpp`, `Operator.h`, `HashBuild.cpp`, `HashProbe.cpp`, `velox/expression/Expr.cpp`.

Next: Module 3, Velox internals in depth (vectors and encodings, memory pools and arbitration, spilling).

Module 3 — Velox Internals

Oct 13, 2026

Why this module

Module 2 showed how a Velox Driver moves batches through operators. This module opens up what those batches and operators are made of: the vector layout, how expressions run over it, where memory comes from and what happens when it runs out, and how data enters and leaves a Task.

The Velox library is organized in layers that map onto these sections:

Layer	Main source directories	What it provides	Section
Types and vectors	`velox/type`, `velox/vector`	Type system, columnar in-memory format, encodings	Vectors; Encodings
Expression engine	`velox/expression`, `velox/functions`	Compiling and evaluating expressions; function libraries (Presto, Spark dialects)	Expression evaluation
Memory	`velox/common/memory`	Pools, allocators, arbitration	Memory management
Execution	`velox/exec`	Task, Driver, operators, hash tables, spilling	Spilling (operators were Module 2)
Connectors and formats	`velox/connectors`, `velox/dwio`	Data sources, file readers and writers	Connectors
Serialization	`velox/serializers`	Wire formats for exchange and spill	Exchange

A useful mental model: Velox is a *toolkit* of these layers plus a reference execution engine, and a host can adopt only the layers it needs.

Vectors

A Velox vector is a typed column of `size()` values plus a null bitmap, stored in reference-counted buffers so that slicing, wrapping and passing columns between operators never copies data unless it must.

The class hierarchy

- `BaseVector`: type, size, encoding, and an optional `nulls_` buffer (1 bit per row; no buffer means "no nulls", a fast path everywhere).
- `SimpleVector<T>`: anything that can return a scalar `T` at row i, whatever the encoding.
- `FlatVector<T>`: the plain layout. `values_` is a contiguous array of `T`. Booleans are bit-packed.
- Complex types: `RowVector` (struct), `ArrayVector`, `MapVector`.
- Encoded wrappers: `ConstantVector<T>`, `DictionaryVector<T>`, `LazyVector` (next section).

Buffers (`velox::Buffer`) are allocated from a memory pool and shared via `BufferPtr`. A vector is *mutable* only if it holds the sole reference to its buffers, which operators check before writing in place (`BaseVector::isWritable`), and otherwise allocate a new result.

Strings: StringView

`FlatVector<StringView>` stores a fixed 16-byte `StringView` per row:

- 4 bytes length, 4 bytes prefix (the first 4 characters).
- Strings of up to 12 bytes are stored **inline** in the remaining 8 bytes, no pointer at all.
- Longer strings keep an 8-byte pointer into a *string buffer* owned by the vector (`stringBuffers_`).

The prefix makes most comparisons and hash-table probes resolve without dereferencing the pointer, and short strings (codes, country names, IDs) avoid indirection entirely. Several vectors can share the same string buffers, so substring or projection results point into the original bytes.

Complex types

- `ArrayVector`: `offsets` and `sizes` buffers (one entry per row) plus a single `elements` child vector holding all elements of all rows.

- `MapVector`: `offsets`, `sizes`, and two children, `keys` and `values`.
- `RowVector`: one child per field (a struct column is itself a nested RowVector).

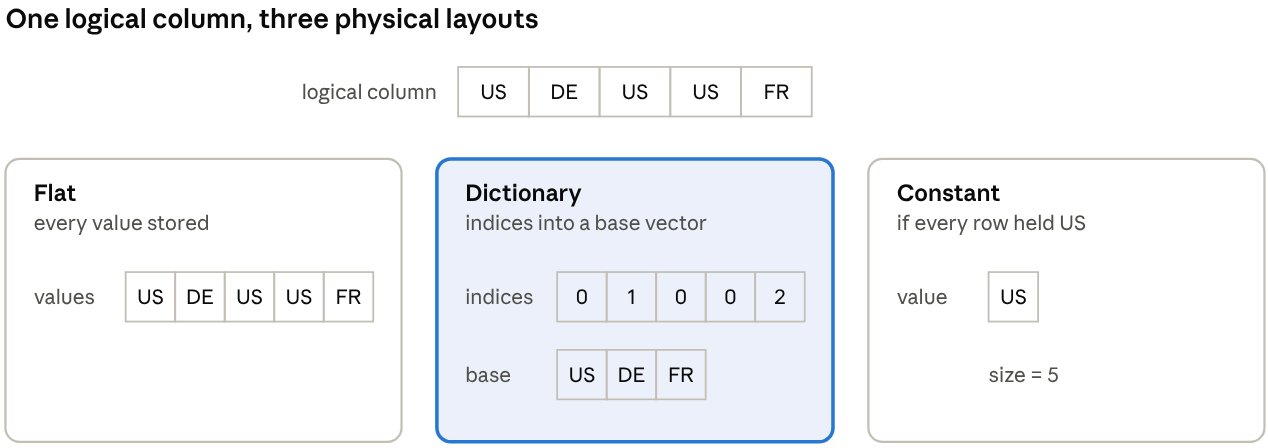
Unlike Arrow, which stores only offsets (size = next offset minus this one), Velox stores **both offsets and sizes**. Rows can therefore point to arbitrary, out-of-order or overlapping ranges of the elements vector, which lets operations like `slice`, `filter` or reordering produce a new ArrayVector by rewriting two small buffers instead of copying elements.

Relationship to Apache Arrow

Velox's layout is Arrow-compatible in spirit and converts to and from Arrow (`velox/vector/arrow`), but it deliberately diverges in three places, for execution speed: StringView instead of Arrow's offset-based strings, offsets plus sizes for arrays and maps, and extra encodings (constant, dictionary) as first-class vectors. Arrow later adopted StringView-style strings and list views, partly under Velox and DuckDB influence.

Encodings

Encodings let Velox represent a column without materializing it value by value. Operators produce them to avoid copying, readers produce them to pass file encodings through, and the expression engine exploits them to compute once per distinct value instead of once per row.



vector encodings · flat, dictionary, constant

Constant

`ConstantVector<T>` holds a single value and a size. It appears for literals, for partition-key columns in a scan (every row of a file has the same `ds`), and as the result of any expression over constants. A function receiving only constant inputs runs once and returns a constant. Constant vectors of complex types wrap one row of a base vector instead of copying it.

Dictionary

`DictionaryVector<T>` holds an `indices` buffer (one int32 per row), a `base` vector, and optionally its own nulls. It arises in two very different ways:

1. **From the data**: a file column that is dictionary-encoded in ORC/DWRF or Parquet can be passed through as a dictionary rather than decoded.
2. **From operators, to avoid copying.** After a filter keeps rows {0, 3, 7}, the output need not copy every column; each column is wrapped in a dictionary with indices [0, 3, 7]. This costs one small indices buffer shared by all columns, regardless of how many columns there are or how wide they are. A hash join's probe side uses the same trick, and indices can repeat when a probe row matches several build rows.

Dictionaries can be nested (a dictionary over a dictionary), and are flattened only when something genuinely needs contiguous values.

Lazy

A `LazyVector` is a placeholder that loads its values on first access, and only for the rows requested. TableScan returns non-filter columns as lazy vectors, so rows eliminated by later filters or joins are never decoded. A loader can even receive a *value hook* that feeds values directly into an aggregation as they are decoded (aggregation pushdown), skipping the intermediate vector.

DecodedVector: uniform access

An operator or function cannot write separate code for every combination of encodings. `DecodedVector` takes any vector plus the active rows and exposes:

- `valueAt<T>(row)` and `isNullAt(row)`, whatever the encoding;
- `index(row)` and `base()`, to work directly on the underlying values;
- `isIdentityMapping()` and `isConstantMapping()`, so code can take a flat or constant fast path.

Hash tables, aggregations and most vector functions read their inputs through it.

Peeling

When all inputs of an expression are dictionaries with the same indices (common after a filter), the expression engine **peels** the wrapper: it evaluates the function on the base vectors, only on base rows that are actually referenced, then wraps the result in the same dictionary. With a file dictionary of 200 countries over a million rows, an expensive function like a regex runs 200 times instead of a million. When consecutive batches share the same dictionary base, results can even be cached across batches.

Expression evaluation

The expression engine compiles a typed expression tree once per Task and then evaluates it batch by batch on the active rows only. Most of its cleverness is about running each function on as few rows, and as few distinct values, as possible.

From plan to ExprSet

A plan carries **typed expressions** (`core::TypedExpr`): field accesses, constants, calls, casts, lambdas. At operator construction, they are compiled into an `ExprSet` , a tree of `exec::Expr` nodes:

- **Function resolution** against the registry by name and argument types.
- **Common subexpression elimination** across all expressions in the set (a Project computing `f(x) + 1` and `f(x) * 2` evaluates `f(x)` once).
- **Constant folding** of subtrees with only constant inputs.
- **Flattening** of nested AND/OR into one n-ary node.

The evaluation loop

`exprSet.eval(rows, ctx, results)` runs each Expr over a `SelectivityVector` of active rows. For a function call node:

1. Evaluate the inputs recursively on the same rows.
2. **Nulls**: for functions with *default null behavior* (the vast majority), any row with a null input produces null without calling the function; those rows are simply removed from the active set.
3. **Peel** shared dictionary or constant wrappers (previous section).

4. Call the function on the remaining rows, writing into the result vector.
5. Re-wrap the result in any peeled encoding.

Writing functions: two APIs

Simple functions cover most of the library. You write a scalar `call()` and Velox generates the vectorized loop, decoding, null handling and fast paths for flat inputs:

```
template <typename T>
struct PlusOneFunction {
  VELOX_DEFINE_FUNCTION_TYPES(T);
  FOLLY_ALWAYS_INLINE void call(int64_t& out, const int64_t& in) {
    out = in + 1;
  }
};
// registerFunction<PlusOneFunction, int64_t, int64_t>({"plus_one"});
```

Optional extras: `callNullable` for custom null semantics, `callAscii` for an ASCII-only fast path chosen per batch, and `initialize()` to precompute from constant arguments once (e.g. compile a regex pattern).

Vector functions implement `apply(rows, args, outputType, ctx, result)` directly on vectors, for logic that benefits from seeing the whole batch: special handling of encodings, building complex outputs, or amortizing setup.

The same engine hosts different SQL dialects, with separate libraries in `velox/functions/prestosql` and `velox/functions/sparksql`. Semantics differ in details (integer overflow, cast rules, null and error behavior), which is why Gluten and Prestissimo can share one engine.

Special forms and conditional evaluation

AND, OR, IF, SWITCH, COALESCE, TRY and CAST are special forms, not functions, because they control *which rows* their children are evaluated on:

- AND evaluates its second conjunct only on rows the first did not already reject (and OR only on rows not yet accepted).
- IF evaluates THEN only on rows where the condition is true, ELSE only on the rest.

This is a correctness requirement, not just an optimization: `IF(x <> 0, y / x, 0)` must not raise division by zero, and it does not, because the division never sees the rows where x = 0.

Errors

Errors are recorded **per row** in the EvalCtx instead of being thrown immediately. Under `TRY(...)`, failing rows simply become null; otherwise the first recorded error is raised after evaluation. Combined with conditional evaluation, this gives row-precise SQL error semantics in a vectorized engine.

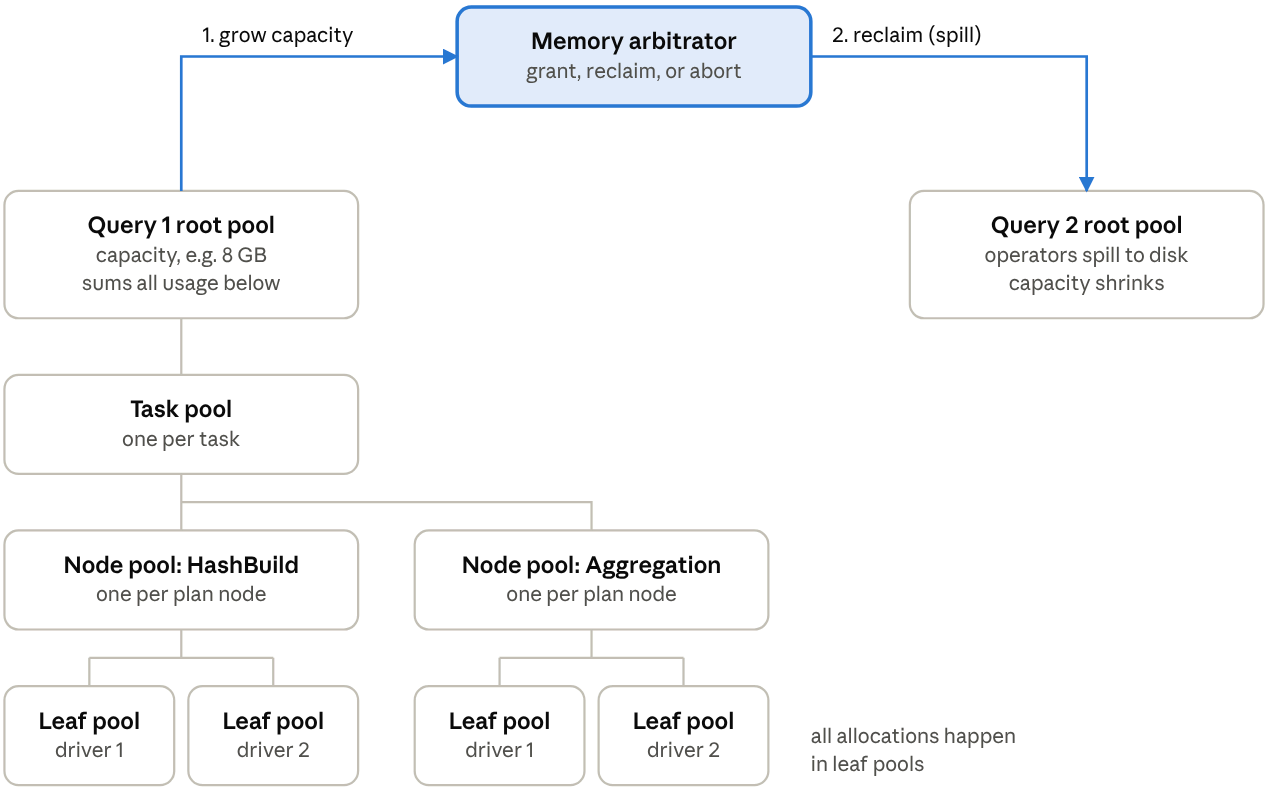
Adaptivity

An AND tracks each conjunct's observed selectivity and cost, and periodically reorders conjuncts so cheap, selective predicates run first. Together with per-batch fast paths (no nulls, ASCII-only, flat inputs), this is how Velox gets some of the benefits of runtime specialization without code generation.

Memory management

A Velox worker runs many queries in one process, so memory must be accounted per query, capped per query, and, when it runs short, *reclaimed* by spilling rather than ending in a process-wide out-of-memory crash. Velox does this with a tree of memory pools per query and an arbitrator that moves capacity between trees.

Memory is a pool tree per query; the arbitrator moves capacity



memory pools · per-query tree, arbitration between queries

Pools: accounting

- A process-wide `MemoryManager` owns the allocator and the arbitrator.
- Each query has a **root pool** with a *capacity*. Below it sit a pool per Task, a pool per plan node, and a **leaf pool** per operator instance (one per Driver).
- Only leaf pools allocate. Usage rolls up the tree, so every level knows its total.
- To avoid updating shared counters on every allocation, leaf pools **reserve** memory in quantized chunks and allocate within the reservation; only crossing a chunk boundary touches the parents.

Allocators: getting bytes

- `MallocAllocator` (default) wraps malloc/jemalloc; `MmapAllocator` manages its own address space in size classes of pages and returns memory to the OS with madvise.

- Big structures (hash tables, the row containers behind joins and aggregations) are allocated as **non-contiguous runs of pages** where possible, so they never need one huge contiguous block, and can grow without copying.

Arbitration: what happens at the limit

When a query's reservation would exceed its root pool's capacity, the pool asks the `MemoryArbitrator` to grow it. The `SharedArbitrator` tries, in order:

1. **Grant** from free capacity in the process.
2. **Reclaim** from queries (including possibly the requester itself): operators that can release memory are asked to, which for most memory-heavy operators means spilling. Reclaim happens only at a safe point: the victim's Drivers are paused off-thread first, then operators spill, then the Task resumes.
3. **Abort** a query with a memory-exceeded error if nothing can be reclaimed, preferring to sacrifice one query over failing the worker.

Operators declare whether they can reclaim (`canReclaim()`). Spillable ones (next section) make the system degrade to disk I/O under pressure; non-spillable ones limit what the arbitrator can do.

Spilling

Spilling turns "out of memory" into "slower": an operator writes part of its state to disk, frees the memory, and reads it back later in a form that still produces the correct result. Each spillable operator needs its own strategy, because what it must restore later differs.

Spillable operators: HashBuild (join build side), HashAggregation, OrderBy, Window, RowNumber, TopNRowNumber. Their state lives in a `RowContainer` (row-wise layout used behind hash tables and sorts), and a shared `Spiller` serializes rows into compressed spill files in a configured spill directory.

Spilling starts either when the arbitrator asks an operator to reclaim, or when an operator crosses its own configured threshold.

Aggregation: sorted runs, merged at the end

1. When asked to spill, sort the accumulated groups by grouping key, write them as a **sorted run**, clear the hash table, continue consuming input.

2. At end of input, k-way merge all runs plus what is still in memory. Because each run is sorted by key, groups with equal keys arrive adjacent, so they combine in one streaming pass with constant memory.

This works because aggregate states are mergeable: the same partial/final property from Module 1.

OrderBy: external merge sort

The textbook version: write sorted runs as memory fills, then merge them on output.

Hash join: partitioned spilling (grace hash join)

1. The build side is split into partitions by **hash bits** of the join key. Under pressure, some partitions are spilled and their memory freed.
2. During probing, probe rows that hash into an in-memory partition are joined immediately; probe rows that hash into a spilled partition are themselves spilled, into the matching probe-side partition.
3. After the in-memory partitions are done, each spilled partition is **restored**: build its hash table from disk, then probe it with its spilled probe rows.
4. If a restored partition still does not fit, it is spilled again, splitting on the *next* hash bits. A configurable maximum spill level stops infinite recursion.

The weak point is the same as in distributed hash joins (Module 1): a single huge key cannot be split by hashing. Recursive spilling cannot help it, and the query fails once the spill level limit is reached. Skew handling needs a different technique (Module 6).

Connectors, splits and readers

Data enters a Task through a **connector**: a pluggable source that turns *splits* into vectors. Most of the work of a fast scan happens before values reach a vector: pruning files, row groups and rows, and decoding only what survives.

Connectors and splits

- A `Connector` is registered by id and provides a `DataSource` for TableScan (and a `DataSink` for TableWriter).

- A `ConnectorSplit` is a unit of scan work, typically a file plus a byte range. The host adds splits to the Task (`Task::addSplit`); each TableScan Driver takes the next split when it finishes one, and blocks with `kWaitForSplit` if none are queued. Splits therefore do the load-balancing that morsels do in DuckDB.
- The **Hive connector** reads files from local disk, HDFS or object stores (S3, GCS, ABFS) in DWRF/ORC, Parquet, Nimble (Meta's newer columnar format) or text.

ScanSpec: what to read

A `ScanSpec` tree describes, per column and per subfield, whether it is needed, which pushed-down filter applies (typed filters such as integer ranges, value sets, null checks), and whether it is a constant (like a partition key). Subfield pruning means reading only the struct fields or map keys a query touches, a large saving for wide nested ML-style tables.

Predicates that cannot be expressed as simple per-column filters stay as a **remaining filter**, evaluated as a normal expression after reading.

Selective readers: filter while decoding

1. **Skip coarse units first**: use file, stripe or row-group statistics (min/max, null counts) to skip data that cannot match.
2. **Filter columns one at a time**: decode the first filter column, evaluate its filter *during decoding*, and keep only passing row numbers. Decode the next filter column only for those rows, and so on.
3. **Order filters adaptively**: the reader tracks each filter's cost and selectivity and reorders them so the cheapest, most selective one runs first.
4. **Everything else is lazy**: non-filter columns are returned as lazy vectors and decoded only for rows that survive downstream.

Caching and I/O

For remote storage, Velox coalesces nearby reads into larger I/Os, prefetches upcoming column chunks, and can keep hot data in an in-memory `AsyncDataCache` backed by a local SSD cache. On object stores, avoiding a round trip often matters more than any CPU optimization.

Exchange, hosts and testing

Velox provides the endpoints of an exchange and the wire formats, while each host supplies the transport. That split, plus an unusually heavy investment in randomized testing, is what lets one engine serve very different systems.

Exchange endpoints

- **PartitionedOutput** (the producer side) partitions each batch by hash, round-robin or broadcast, serializes it, and appends it to the Task's `OutputBuffer`, one queue per destination (a broadcast shares a single queue across destinations). A full buffer blocks the Driver: backpressure, as discussed in Module 2.
- **Exchange** (the consumer side) pulls serialized pages through an `ExchangeClient` from host-provided `ExchangeSource`s, then deserializes them into vectors.
- **LocalPartition / LocalExchange** do the same between pipelines inside one Task, with no serialization at all.

Serialization formats (VectorSerde)

- `PrestoVectorSerde`: Presto's columnar page format, so Prestissimo workers interoperate with Presto's protocol.
- `CompactRow` and `UnsafeRow`: row-wise formats, the latter compatible with Spark's.

The same serializers write spill files, with optional compression.

Hosts

- **Prestissimo** (Presto C++): replaces Presto's Java workers with a C++ process embedding Velox; the Java coordinator still parses, optimizes and schedules, and talks to workers over Presto's HTTP protocol.
- **Gluten**: a Spark plugin that converts Spark physical plans (via Substrait) into Velox plans and runs them through JNI, while Gluten provides columnar shuffle.
- **Meta-internal systems**, including ML data preprocessing (DPP), use Velox as a library directly, often only some of its layers.

Testing with fuzzers

A vectorized engine with many encodings has a combinatorial bug surface: every function must give the same answer for flat, dictionary, constant and lazy inputs, with and without nulls. Velox attacks this with differential fuzzers:

- **ExpressionFuzzer** generates random expressions and inputs, evaluates them with different encodings and evaluation paths, and checks that results agree.
- **AggregationFuzzer, JoinFuzzer, WindowFuzzer** generate random plans and compare results against a reference engine (DuckDB or Presto) and across execution variants, such as with and without spilling.
- Memory arbitration and spilling fuzzers inject memory pressure to exercise the reclaim paths.

For engine work, fuzzers like these are worth as much as unit tests.

Self-test and reading list

1. Why may an operator write into a vector only when it is the sole owner of the vector's buffers?
2. Describe the StringView layout. Why the 4-byte prefix, and why the 12-byte inline limit?
3. Why does Velox store both offsets and sizes for arrays and maps, unlike Arrow?
4. Name the two ways dictionary vectors arise, and why a filter's output is wrapped rather than copied.
5. When can the engine peel a dictionary, and why is that so effective on file-dictionary columns?
6. Why does `IF(x <> 0, y / x, 0)` never raise division by zero in Velox?
7. When would you write a vector function instead of a simple function?
8. Why do leaf pools reserve memory in quantized chunks?
9. What three things can the arbitrator do, in which order, and why must a victim's Drivers be paused before reclaim?
10. Walk through a spilled hash join from build to the last restored partition. What input makes it fail?
11. How does a selective reader decide in which order to evaluate column filters?

Reading list:

1. Pedreira et al., "Velox: Meta's Unified Execution Engine", VLDB 2022.
2. Velox documentation (facebookincubator.github.io/velox): the pages on vectors, expression evaluation, simple functions, memory management and arbitration, and spilling.

3. Source to skim: `velox/vector/BaseVector.h`, `DecodedVector.h`, `velox/expression/Expr.cpp`, `velox/common/memory/SharedArbitrator.cpp`, `velox/exec/Spiller.cpp`, `velox/dwio/common/SelectiveColumnReader.h`.
4. Gluten and Prestissimo documentation, for how hosts embed Velox.

Next: Module 4, core operators in depth (hash join, aggregation, sort, window).

Module 4 — Core Operators

Oct 13, 2026

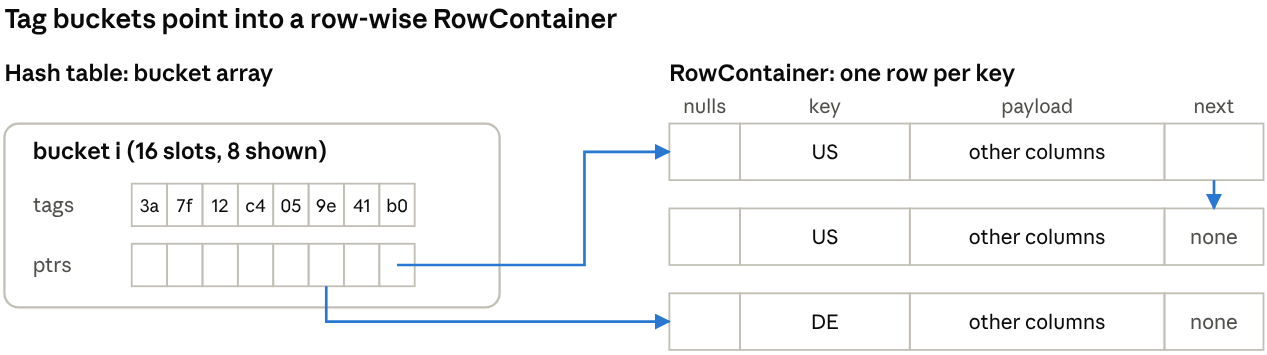
Why this module

Scans, filters and projections are streaming and mostly cheap. Nearly all of the CPU time and memory of an analytical query goes into a handful of stateful operators: **hash join, aggregation, sort and window**. They are the pipeline breakers from Module 1, the spill clients from Module 3, and the places where engines differ most.

Three of them share one data structure, the hash table, so this module starts there. Each operator section then covers the algorithm, how Velox implements it, and the alternatives other engines choose.

Hash tables for query processing

An engine's hash table is not `std::unordered_map`: it separates a compact, cache-friendly index from the row data, batches its operations over a whole vector of keys, and picks the cheapest key representation the data allows.



Velox hash table · tag buckets and RowContainer rows

Two parts: rows and index

- The **RowContainer** stores one row per distinct key (aggregation) or per build row (join), *row-wise*: null flags, key columns, then payload columns (join) or accumulators (aggregation), plus a `next` pointer for duplicate join keys. Rows live in pages from the

memory pool; long strings go to a separate arena. Row-wise is deliberate: once a key matches, every needed column of that row is in one or two cache lines.

- The **bucket array** is the index. Each bucket is roughly 128 bytes, holding 16 one-byte **tags** (7 bits of the hash) and 16 row pointers. This is the Swiss-table / F14 idea from Module 2.

Batched probing

For a batch of ~1,000 keys, the table works in passes, not key by key: hash all keys, prefetch all buckets, then compare tags with one SIMD instruction per bucket, and only follow pointers whose tag matches. A full key comparison happens only for likely matches. This is where the memory-level parallelism argument from Module 2 pays off in practice.

Velox key modes: avoid hashing when possible

While building, a `VectorHasher` per key column tracks the value range and distinct values seen. The table then picks the cheapest of three modes, and can switch as data arrives:

Mode	When	How a lookup works
Array	All key combinations map into a small dense range (e.g. small integers, few distinct strings)	Direct index into an array, no hashing, no comparison
Normalized key	Combined keys fit into one 64-bit value	Hash and compare a single `uint64`, even for multi-column keys
Hash	Otherwise	Full hashing and key comparison as above

Array mode makes small-cardinality GROUP BYs (the 200 countries from Module 1) almost free, and normalized keys turn multi-column string comparisons into one integer compare.

Sizing and growth

The table is sized from the build input when it is known (join build) and doubled as needed otherwise (aggregation); rehashing re-inserts pointers only, never rows, since rows stay in the RowContainer. Parallel join builds construct partial tables per Driver and merge them once (Module 2).

Hash join

Build a hash table on the smaller input, then stream the larger one past it. Every join type is a variation of what happens on a hit, on a miss, and after the last probe.

Building the output

For each probe batch, the probe produces pairs (probe row index, build row pointer). Output columns are then assembled without row-by-row copying where possible: **probe-side columns are wrapped in a dictionary** over the probe batch (an index can repeat when one probe row matches several build rows), and build-side columns are extracted from the RowContainer rows in bulk.

Join types, as hit/miss/finish rules

Join type	On match	On no match	After the last probe
Inner	emit each pair	drop	nothing
Left outer (probe side kept)	emit each pair	emit probe row + NULLs	nothing
Right / full outer (build side kept)	emit, and set the build row's *matched* flag	drop (full: emit with NULLs)	emit build rows never marked matched
Left semi (EXISTS, IN)	emit probe row once, stop at first match	drop	nothing
Anti (NOT EXISTS)	drop	emit probe row	nothing
Null-aware anti (NOT IN)	drop	emit, unless NULLs make the result unknown	nothing

Two details matter in practice:

- **Finishing build-side outer joins in parallel.** Each probe Driver marks matches in the shared table, so only the *last* Driver to finish may scan for unmatched build rows; Velox coordinates this through the join bridge. This is why keeping unmatched build rows is harder than keeping unmatched probe rows.

- **NOT IN is not NOT EXISTS.** `x NOT IN (subquery)` returns no rows at all if the subquery contains a NULL, and a probe row with NULL `x` is not returned unless the subquery is empty. Null-aware anti join implements this three-valued logic; rewriting NOT IN into a plain anti join is a classic optimizer bug.

An extra non-equality condition (`ON a.k = b.k AND a.ts > b.ts`) is evaluated on candidate pairs after the key match. For outer joins, a row counts as matched only if the condition passes.

Dynamic filters

Once the build side is complete, the engine knows every join key that can possibly match. Pushing that knowledge into the probe-side scan can eliminate most of the probe input before it is ever decoded:

- In Velox, HashProbe derives a filter from the build keys (a value range, or a set of values when small) and pushes it into the TableScan at the start of the probe pipeline. The selective reader (Module 3) then skips row groups and rows that cannot match.
- Distributed engines do the same across stages: Presto's dynamic filtering and Spark's runtime filters collect build-side keys or a Bloom filter and ship them to probe-side scans, possibly on other machines.

The classic win is a star schema: `fact JOIN dim ON ... WHERE dim.region = 'EU'` turns the dimension filter into a filter on the billion-row fact table scan.

Alternatives to the classic hash join

- **Radix-partitioned hash join**: first partition both inputs by hash bits into cache-sized partitions, then join each partition with a table that fits in cache. Fewer cache misses, at the cost of an extra pass. Research (Balkesen et al., ICDE 2013 and later) went back and forth on whether this beats a large shared table with batched prefetching; most production engines choose the latter.
- **Sort-merge join**: merge two inputs sorted on the key, with no hash table. Best when inputs already arrive sorted (Velox `MergeJoin`) and as a robust fallback for huge inputs.
- **Nested-loop join**: for cross joins and non-equality conditions only; quadratic.
- **Index nested-loop join**: probe an existing index or key-value store per row, as OLTP systems and some lakehouse engines do for point lookups.

Aggregation

Hash aggregation maps each row's grouping key to a group row in the RowContainer and updates that row's accumulators. The interesting parts are how aggregate functions are split so work can be done in stages, and what to do when there are almost as many groups as rows.

Hash aggregation, batch at a time

1. Look up or insert all grouping keys of the batch at once, producing one group-row pointer per input row.
2. For each aggregate function, update all affected accumulators in one vectorized call, e.g. `addRawInput(groups, rows, args)`.

Accumulators live at fixed offsets inside each group row. Variable-size state (a set for `array_agg` or exact `COUNT(DISTINCT)`) is allocated in a separate arena and referenced from the row. A global aggregation (no GROUP BY) skips the hash table and updates a single accumulator.

Splitting work: partial, intermediate, final

Every aggregate function is defined by four parts: an accumulator, "add raw input", "merge intermediate results", and "extract final value". That makes it splittable across stages:

Step	Input	Output	Where it runs
Partial	raw rows	intermediate state	before the shuffle
Intermediate	intermediate state	intermediate state	optional middle level
Final	intermediate state	final value	after the shuffle
Single	raw rows	final value	no shuffle needed

The intermediate type is part of each function's definition: `SUM` keeps a number, `AVG` keeps (sum, count), `approx_distinct` keeps a HyperLogLog sketch. In Velox, these map to the `Aggregate` methods `addRawInput`, `addIntermediateResults`, `extractAccumulators` (intermediate) and `extractValues` (final).

Partial aggregation is an optimization, never required for correctness, so it can cut corners: when its memory limit is hit, it simply flushes its groups downstream and starts empty, and when it observes poor reduction, it switches to passing rows through (Module

1, question 4).

Streaming aggregation

If the input arrives sorted or clustered by the grouping keys (after a merge join, or from a file sorted on them), no hash table is needed: accumulate the current group and emit it when the key changes. Memory is constant regardless of the number of groups. Velox has a separate `StreamingAggregation` operator for this.

Distinct aggregates

`COUNT(DISTINCT x) ... GROUP BY g` is expensive because each group must remember every distinct value. Three strategies:

1. **Set accumulator per group**: simple, but memory grows with distinct values and does not distribute well.
2. **Two-level aggregation**: rewrite as `GROUP BY g, x` (deduplicate), then `GROUP BY g` with `COUNT(*)`. Both levels are ordinary aggregations, so they parallelize, shuffle and spill normally. With several distinct aggregates on different columns, engines use a MarkDistinct operator or an Expand into one stream per distinct column.
3. **Approximate**: `APPROX_COUNT_DISTINCT` (BigQuery) or `approx_distinct` (Presto) uses HyperLogLog: about 1 to 2% error with a few kilobytes per group, and sketches merge across partial and final steps.

Grouping sets, ROLLUP and CUBE

An Expand (Velox `GroupId`) operator duplicates each input row once per grouping set, nulling out the columns not in that set and adding a group-id column; a single ordinary aggregation then computes all sets at once.

High cardinality

When the number of groups approaches the number of rows (GROUP BY a user id or transaction id), the hash table is as large as the input. Partial aggregation stops helping, the final aggregation may spill (sorted runs, Module 3), and skewed keys concentrate on one task (Module 6).

Sort and Top-N

Sorting in an engine is mostly a fight against comparison cost: comparing two rows means dereferencing both, dispatching on each key's type and handling nulls and sort direction. The best engines make most comparisons a single `memcmp` on a short, contiguous prefix, or avoid comparisons entirely.

In-memory sort

1. Accumulate input rows into a RowContainer.
2. Build an array of row pointers and sort the *pointers*, not the rows.
3. Emit by gathering rows in pointer order into output vectors.

Normalized keys and prefix sort

Encode the leading sort keys into a fixed-size, **binary-comparable** byte string stored next to each pointer: flip the sign bit of integers so negative sorts before positive, invert all bits for DESC columns, prepend a byte for NULLS FIRST/LAST, copy the first bytes of strings. Comparing two rows then becomes a `memcmp` on a contiguous array; full key comparison happens only when prefixes tie. Velox's prefix sort works this way. DuckDB goes further and **radix-sorts** the normalized keys, needing no comparisons at all.

Top-N

`ORDER BY x LIMIT k` never needs a full sort: keep a heap of the best k rows and compare each new row against the heap's worst. Cost is O(n log k) time and O(k) memory. Distributed, each task computes a partial Top-N, and a final Top-N over at most k rows per task picks the winners: the same partial/final pattern as aggregation.

Large and distributed sorts

- **External**: sorted runs on disk, then a k-way merge (Module 3).
- **Distributed ORDER BY**, two options. (a) Each task sorts its own data, then a *merge gather* streams all sorted outputs into one consumer, which still produces the output on one node. (b) Range-partition by sampled key boundaries so each task sorts a disjoint key range; outputs then simply concatenate in order and the sort itself scales out.

BigQuery's own best-practice docs recommend pairing ORDER BY on large results with LIMIT, for exactly this reason: the final ordering of an unlimited result can concentrate on a single worker.

Window functions

A window function computes, for every row, a value over a set of related rows (its *frame*) without collapsing rows the way GROUP BY does. Execution therefore needs rows grouped by partition and ordered within each partition, and then an efficient way to compute each frame.

Semantics in one example

```
SUM(amount) OVER (PARTITION BY cust_id ORDER BY ts
                  ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
```

- **PARTITION BY**: rows of different customers never see each other.
- **ORDER BY**: defines "preceding" and "following" within a partition.
- **Frame**: which rows feed this row's value. ROWS counts physical rows; RANGE uses value distance on the ORDER BY key; GROUPS counts peer groups of equal keys.
- Function families: ranking (`ROW_NUMBER` , `RANK` , `DENSE_RANK` , `NTILE`), navigation (`LAG` , `LEAD` , `FIRST_VALUE` , `NTH_VALUE`), and any aggregate evaluated over the frame.

Execution

1. **Distribute** by the partition keys (shuffle), so each partition lands on exactly one task.
2. **Sort** by (partition keys, order keys), unless the input already arrives in that order.
3. **Process partition by partition**: for each row, compute frame bounds, then each function's value.

Velox's `Window` operator buffers input in a RowContainer, sorts it, and evaluates functions over one partition at a time. When the input is already sorted, a streaming mode holds only the current partition, so memory is bounded by the largest partition instead of the whole input. The operator can also spill.

Computing frames efficiently

Recomputing an aggregate over every row's frame costs O(n × frame size). Better options:

- **Running frames** (`UNBOUNDED PRECEDING` to `CURRENT ROW`): carry the accumulator forward, O(n).
- **Sliding frames with invertible aggregates** (SUM, COUNT, AVG): add the row entering the frame, subtract the one leaving, O(n).
- **Sliding frames with non-invertible aggregates** (MIN, MAX, and many others): a **segment tree** over the partition answers any frame in O(log n) (Leis et al., "Efficient Processing of Window Functions in Analytical SQL Queries", VLDB 2015). Fixed-size MIN/MAX frames can also use a monotonic deque.

Pitfalls

- **No PARTITION BY means one partition**: `ROW_NUMBER() OVER (ORDER BY ts)` on a billion-row table puts every row on a single task. It is the window-function version of an unlimited ORDER BY.
- **Skewed partitions**: one huge customer becomes one huge partition on one task.
- **Top-k per group**: `WHERE ROW_NUMBER() OVER (PARTITION BY g ORDER BY x) <= k` should not sort everything. Velox's `TopNRowNumber` keeps a k-sized heap per partition instead.

Smaller operators worth knowing

These are cheap in CPU terms but each hides a semantic or control-flow detail that shows up in real plans.

Operator	What it does	The detail that matters
Limit	Pass the first k rows, then finish	Finishing early must stop upstream work: the Driver sees `isFinished()` and stops pulling, and distributed engines cancel upstream tasks once the final limit is met
Unnest	Turn each array or map element into a row	Other columns are dictionary-wrapped with repeated indices, so a 1-to-many expansion does not copy them

Operator	What it does	The detail that matters
MarkDistinct	Add a boolean "first occurrence of this key" column	Lets several `COUNT(DISTINCT ...)` on different columns share one pass
EnforceSingleRow	Fail if a scalar subquery returns more than one row	Required by SQL semantics of `x = (SELECT ...)`
AssignUniqueId	Add a unique row id	Used by decorrelation rewrites (Module 1) that must regroup rows later
LocalMerge	Merge several sorted streams within a Task	The in-process counterpart of the merge gather exchange
TableWriter	Write output files, possibly partitioned	Files must become visible atomically: write to temporary locations, then commit through the host's metadata or catalog

Self-test and reading list

1. Why does a RowContainer store rows row-wise when everything else in the engine is columnar?
2. What do the tag bytes in a bucket buy, compared with storing full hashes or comparing keys directly?
3. When can Velox use array mode or normalized-key mode instead of hashing?
4. Write the hit/miss/finish rules for a full outer join, and explain why only one Driver may emit unmatched build rows.
5. Why is rewriting `NOT IN` into a plain anti join wrong? Give an input that shows it.
6. How does a dynamic filter reach the probe-side scan, and why does it help star schemas so much?
7. Define the four parts of an aggregate function, and give the intermediate state for AVG and for approximate distinct count.
8. Why may a partial aggregation flush early or give up entirely, while a final aggregation may not?
9. Three ways to compute `COUNT(DISTINCT x)` per group: costs and trade-offs.

10. How does a normalized key make descending sort order and NULL ordering a plain `memcmp`?
11. Why is `ROW_NUMBER() OVER (ORDER BY ts)` dangerous on a large table, and what is the fix?
12. How does a segment tree make a sliding MIN over a window frame O(log n) per row?

Reading list:

1. Leis et al., "Efficient Processing of Window Functions in Analytical SQL Queries", VLDB 2015.
2. Balkesen et al., "Main-Memory Hash Joins on Multi-Core CPUs: Tuning to the Underlying Hardware", ICDE 2013.
3. Graefe, "Implementing Sorting in Database Systems", ACM Computing Surveys 2006.
4. Flajolet et al., "HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm", 2007; and Heule et al., "HyperLogLog in Practice", EDBT 2013 (Google's improvements).
5. DuckDB blog posts on its sorting and aggregation implementations.
6. Velox source: `velox/exec/HashTable.cpp`, `VectorHasher.cpp`, `HashProbe.cpp`, `GroupingSet.cpp`, `Window.cpp`, `PrefixSort.cpp`.

Next: Module 5, columnar storage and scans (Parquet, ORC, Capacitor; encodings, pushdown, late materialization).

Module 5 — Columnar Storage and Scans

Oct 13, 2026

Why this module

In a disaggregated engine like BigQuery, the cheapest byte is the one never read: storage sits across a network, so pruning and compression often matter more than any CPU optimization. This module covers how analytical file formats are laid out, how they encode values, and how a scan uses their metadata to skip data at every level.

Module 3 covered the reader side in Velox (ScanSpec, selective readers, lazy vectors). This module covers what those readers are reading.

Row vs. column storage

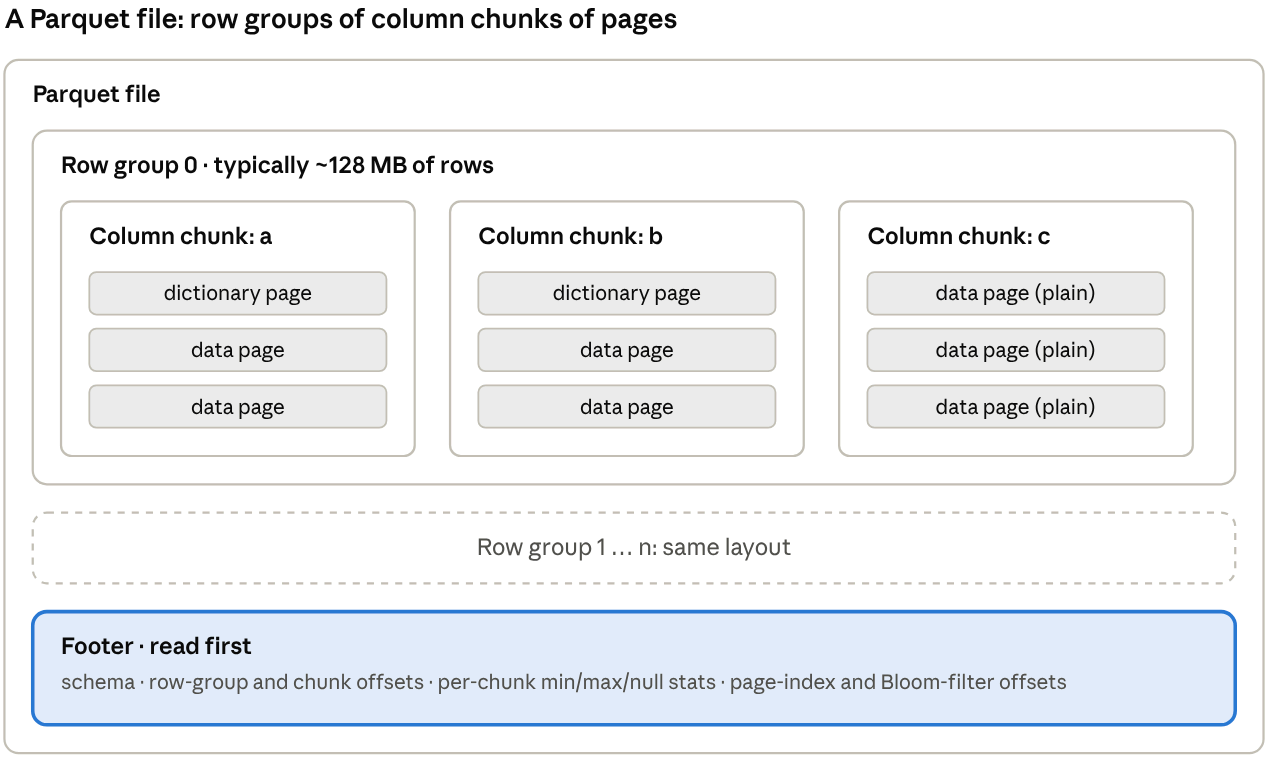
Analytical queries read few columns over many rows, so storing each column contiguously means reading only what is needed and compressing similar values together. Pure column files, though, make it expensive to reassemble rows, so every modern format uses a hybrid: **row groups of columns**.

- **Row-oriented** (N-ary storage, NSM): each row's fields are stored together. Great for OLTP (insert or read a whole record), wasteful for `SELECT AVG(price)` over a 100-column table, which reads 100 columns to use one.
- **Column-oriented** (DSM): each column is stored separately. A scan reads only referenced columns, and a column of one type compresses far better than mixed rows. The cost is reassembling rows when many columns are needed.
- **PAX** (Partition Attributes Across; Ailamaki et al., VLDB 2001): split the table horizontally into chunks of rows, and store each chunk column by column. Within a chunk you get columnar reads and compression; across chunks you get independent units that can be read, skipped and processed in parallel.

Parquet's row groups, ORC's stripes and Capacitor's blocks are all PAX. The row-group size is a trade-off: large groups give long, well-compressed column chunks and efficient sequential I/O; small groups give finer-grained skipping and parallelism.

Parquet in depth

A Parquet file is a sequence of row groups, each holding one column chunk per column, each chunk made of pages, with all metadata in a footer at the end. A reader fetches the footer first and uses it to read only the chunks and pages it needs.



Parquet layout · file, row groups, column chunks, pages, footer

The levels

- **Footer** (at the end, so a writer can stream data and write metadata last): the schema, the byte offsets of every row group and column chunk, and per-chunk statistics (min, max, null count, sometimes distinct count). Reading a Parquet file starts with a small read of the last bytes for the footer length, then the footer itself.
- **Row group**: the PAX unit, usually tens to hundreds of MB. The unit of parallelism: a split (Module 3) is typically one or more row groups.
- **Column chunk**: one column's values for that row group, contiguous on disk, so reading one column of one row group is a single range read.
- **Page**: the unit of encoding and compression, typically ~1 MB. A chunk has at most one *dictionary page* first, then *data pages*. Each data page carries its own encoding header, and its repetition and definition levels for nested data (next section).

Finer-grained skipping

- **Page index** (column index + offset index): per-page min/max and row ranges, stored near the footer. Lets a reader skip individual pages, not just whole row groups, which matters most for sorted or clustered columns.
- **Bloom filters** per column chunk: answer "is value v possibly here?" for equality predicates on high-cardinality columns where min/max are useless (IDs, UUIDs).

Encodings and compression in Parquet

- `PLAIN`: values back to back.
- `RLE_DICTIONARY`: dictionary page plus indices encoded with the RLE/bit-packing hybrid. Writers fall back to plain when the dictionary grows too large.
- `DELTA_BINARY_PACKED`, `DELTA_LENGTH_BYTE_ARRAY`, `DELTA_BYTE_ARRAY`: deltas between consecutive values (good for sorted integers, timestamps, strings with shared prefixes).
- `BYTE_STREAM_SPLIT`: splits floating-point bytes into separate streams so a general-purpose compressor works better.
- Then each page can be compressed with Snappy, ZSTD, LZ4 or GZIP: lightweight encoding first, general-purpose compression second.

Encoding is chosen per column chunk, by the writer, which is why two Parquet files with the same data can differ several-fold in size and scan speed.

Nested data: repetition and definition levels

The 2010 Dremel paper's lasting contribution to storage is a way to store nested, repeated records **column by column**, so a query can read one nested leaf field without touching the rest of the record. Parquet adopted it directly; it is also the foundation of how BigQuery stores nested data.

Each leaf value is stored with two small integers:

- **Definition level (d)**: how many of the optional or repeated fields on the path to this leaf are actually present. If d is below the maximum, the value is NULL or missing at that depth.
- **Repetition level (r)**: at which repeated field on the path this value repeats. **0 means "this value starts a new record".**

A small example

Schema: `record { repeated string tags; }`. Three records: r1 has tags [a, b], r2 has [] (empty), r3 has [c]. Maximum d and r are both 1. The `tags` column stores:

Value	r	d	Meaning
a	0	1	new record (r1), tag present
b	1	1	same list continues
NULL	0	0	new record (r2), list empty
c	0	1	new record (r3), tag present

With deeper nesting, d and r range up to the number of optional or repeated ancestors, and record boundaries at each depth are recovered purely from the levels, column by column.

Why it works well: levels are tiny integers that are mostly constant, so they compress to almost nothing with RLE, and each leaf column is self-sufficient. **The cost:** levels do not map directly to the offsets and nulls of in-memory formats, so readers must convert them into Arrow or Velox layouts (offsets, sizes, null bitmaps). That conversion is a known hot spot in Parquet readers.

ORC takes the other approach: it stores a list as a *lengths* stream plus a child column, and nulls as *present* bitmaps at each level. That maps more directly onto offsets-based vectors, at some cost in compression for deep nesting.

ORC, DWRF and Nimble

ORC came out of the Hive ecosystem at the same time as Parquet, with the same PAX idea but finer internal structure. Meta's DWRF and Nimble descend from it, tuned for very wide tables.

ORC

- **Stripes** (default ~64 MB) are the row groups. Each stripe has three parts: *index data* (a row-index entry every 10,000 rows with min/max statistics and stream positions), *row data*, and a *stripe footer* describing the streams and their encodings.

- Each column is stored as several **streams**: `PRESENT` (null bitmap), `DATA`, `LENGTH` (for strings and lists), `DICTIONARY_DATA`, and so on. Each stream is compressed independently in chunks with ZLIB, Snappy, LZ4 or ZSTD.
- **Integer RLE v2** picks per run among short repeat, direct bit-packing, delta and "patched base" (frame of reference with a few outliers patched separately); strings use dictionaries when they repeat.
- The row index lets a reader skip 10,000-row groups *inside* a stripe, which is finer than Parquet's row-group statistics without a page index.

DWRF

Meta's fork of ORC, used in its data warehouse and read by Velox. Its most distinctive addition is **flat maps**: a `MAP<key, value>` column whose keys are a known, large set (typical of ML feature maps) is stored as one stream per key. A query or training job that needs 50 of 5,000 features reads 50 streams instead of decoding every map.

Nimble

Nimble (originally called Alpha) is Meta's open-source columnar format, built as a replacement for Parquet and ORC in its own workloads. Its starting point is that ML training and feature-engineering tables are **wide**: thousands of columns, and with flat maps tens of thousands of streams. At that width, the costs that dominate are not the ones Parquet and ORC were tuned for.

Four design principles, as the project states them:

- **Wide**: metadata organized to stay cheap at thousands to tens of thousands of columns and streams. Parquet and ORC footers grow with the column count and must be parsed before any data is read, which becomes a real cost at that width.
- **Extensible**: stream encoding is decoupled from the physical file layout, because encoding techniques evolve faster than file layouts. Users can add new encodings without a new format version.
- **Parallel**: encodings chosen to be SIMD- and GPU-friendly, with the stated goal of exposing enough metadata to plan decoding work before touching the data.
- **Unified**: shipped as a single C++ library that is the reference implementation, rather than a specification with several divergent implementations (a recurring source of Parquet compatibility problems).

How it differs from Parquet and ORC

Aspect	Parquet / ORC	Nimble
Encoding choice	Fixed menu; writer picks per page or stream	Pluggable encodings, applied **recursively** (cascading), e.g. dictionary, whose codes are then RLE, whose run lengths are then bit-packed
How encodings are chosen	Simple writer heuristics	An encoding-selection policy estimates the encoded size of candidate cascades from statistics of the data
Physical units	Row groups or stripes → column chunks or streams → pages	Stripes → streams, written in chunks; a stream is the unit of encoding
Maps of features	ORC: one MAP column; DWRF: flat maps	Flat maps native: each map key is its own stream
Implementation	Many independent readers and writers	One library, integrated with Velox's selective readers

Why it matters for scans

A training job that reads 200 of 20,000 features wants to pay only for those 200 streams: no full-schema metadata parse, no decoding of unused map keys, and decoding that writes straight into Velox vectors. Nimble's current work shows the same priorities: deferring I/O for streams behind lazy vectors until they are actually accessed, and bulk bit-unpacking fast paths in its fixed-bit-width encoding.

Nimble is still young outside Meta; for example, adding it as a recognized file format in Apache Iceberg is under discussion, as it is for Vortex. It is a good example of where formats are heading: cascading, sampled encoding selection (the BtrBlocks idea, below), and layouts designed around the in-memory format of the engine that reads them.

Other recent formats, such as Lance and Vortex, pursue similar goals: cheaper random access for ML and AI workloads, and encodings designed for SIMD decoding.

Capacitor and BigQuery storage

Capacitor is BigQuery's internal columnar format, introduced in 2016 to replace ColumnIO. Its file layout is not public, but Google has described its main ideas, and they line up closely with the rest of this module, pushed further in a few directions.

What Google has described

- **Dremel levels for nested data.** Capacitor stores nested and repeated fields with repetition and definition levels, building on the Dremel research that Parquet also adopted, with Google's own variations.
- **Operating on compressed data.** The step change over ColumnIO was evaluating queries directly on encoded data (dictionary, RLE, delta) instead of decompressing first. The 2025 advanced runtime extends this: filters run on Capacitor's encodings, and encodings are propagated through the query plan (Module 2).
- **Row reordering.** The order of rows decides how long RLE runs are, and choosing the best order is NP-hard. Capacitor uses an approximation model to pick a good reordering per block. This is something a general-purpose writer like a Parquet library does not attempt.
- **Background re-optimization.** BigQuery can later rewrite stored data into a better layout, for example after its models improve or table properties change, then atomically swap it in without disturbing running queries. A managed service can do this because the user never owns the files.
- **Semi-structured data.** For the JSON type, Capacitor shreds frequently occurring paths into virtual columns, so a query on `json.user.id` reads one column rather than parsing whole documents.

The storage stack around it

- **Colossus** (Google's distributed file system) stores Capacitor files, replicated and encrypted. Compute is stateless and reads them over Google's Jupiter network: this is the disaggregation from Module 1.
- **Partitioning and clustering**: a table can be partitioned (usually by date) and clustered by up to four columns, which keeps similar values in the same storage blocks so block-level min/max pruning works well.
- **Big Metadata**: BigQuery's block-level statistics are themselves very large, so they are stored in columnar form and queried with the same distributed engine during planning, to decide which blocks a query can skip (Edara and Pasumansky, VLDB 2021).

- **Open formats**: BigQuery also scans Parquet or ORC files in Cloud Storage, including Iceberg tables via BigLake, and exports through the Storage Read API as Arrow or Avro streams. These paths cannot benefit from Capacitor-specific encodings, which is one reason native tables are usually faster.

Encodings and compression

Formats layer two kinds of size reduction: **lightweight, type-aware encodings** that are fast to decode and can often be queried directly, and **general-purpose compression** on top, which squeezes more but must be undone before anything else can happen.

Lightweight encodings

- **Dictionary**: distinct values once, small integer codes per row. Best for low-to-medium cardinality.
- **Run-length (RLE)**: (value, run length) pairs. Best for sorted or clustered columns.
- **Bit-packing**: store integers in exactly as many bits as their range needs, e.g. 5 bits for codes 0 to 31.
- **Frame of reference (FOR)**: store a block's minimum once, then small offsets from it, bit-packed. *Patched* FOR handles a few outliers separately so they do not inflate the bit width.
- **Delta**: store differences between consecutive values; ideal for timestamps and sorted IDs.
- **Newer schemes**: FSST compresses strings with a static symbol table while keeping random access to individual strings (VLDB 2020); ALP compresses floating-point numbers losslessly and fast (SIGMOD 2024).

Encodings **cascade**: dictionary codes are then bit-packed or RLE-encoded, delta values are then bit-packed. Choosing the cascade per block by sampling the data is the approach of BtrBlocks (SIGMOD 2023) and Nimble, and in spirit of Capacitor's model.

General-purpose compression

Snappy and LZ4 are fast with modest ratios; ZSTD gives much better ratios at moderate cost and has become the common default; GZIP is mostly legacy. All operate on opaque bytes, so a whole page or stream must be decompressed before decoding, and nothing can be evaluated on the compressed bytes.

Computing on encoded data

The payoff of lightweight encodings is that many operations never need decoded values (Abadi, Madden, Ferreira, SIGMOD 2006):

Encoding	Operation	Shortcut
Dictionary	`country = 'US'`	Evaluate once per dictionary entry, then select rows by code
Dictionary (sorted)	`x BETWEEN a AND b`	Map the bounds to a code range, then compare integers
Dictionary	`GROUP BY country`	Group by small codes, ideal for array-mode hash tables (Module 4)
RLE	`SUM(x)`, `COUNT(*)`	value × run length, one operation per run
RLE	any filter	Evaluate once per run
FOR	`x > c`	Compare stored offsets with `c - base`, no decoding

This is exactly what BigQuery's enhanced vectorization does on Capacitor, and what Velox's dictionary peeling does in memory (Module 3): the format's encoding becomes an execution-time asset.

Pruning, clustering and table formats

A scan skips data at up to eight levels, from whole partitions down to individual values. Each level uses a different piece of metadata, and all of them depend on how well the data is clustered.

The pruning hierarchy

1. **Partitions**: skip whole directories or partitions whose partition value fails the predicate (`WHERE ds = '2026-10-01'`).
2. **Files**: skip files using per-file statistics from table-format manifests (Iceberg, Delta) or BigQuery's Big Metadata, without opening the files.
3. **Row groups / stripes**: min/max and null counts from the file footer.
4. **Pages or ORC row groups**: Parquet's page index, ORC's row index.

5. **Bloom filters**: equality predicates on high-cardinality columns.
6. **Dictionaries**: if the predicate's value is not in the chunk's dictionary, skip the chunk.
7. **Rows while decoding**: selective readers evaluate filters column by column (Module 3).
8. **Late materialization**: never decode non-filter columns for rows that were dropped.

Dynamic filters from joins (Module 4) plug into levels 2 to 7 at runtime.

Clustering makes statistics useful

Min/max statistics prune only if each block covers a narrow range of values. On randomly ordered data, every row group spans almost the whole range of `user_id`, and nothing is skipped. Hence:

- **Sorting** data on a commonly filtered column at write time.
- **Multi-column clustering** with space-filling curves (Z-order, Hilbert) to keep blocks narrow on several columns at once, as in Delta Lake's Z-ordering and liquid clustering.
- **BigQuery clustering** on up to four columns, maintained automatically in the background.

Table formats: a footer for the whole table

Apache Iceberg, Delta Lake and Apache Hudi add a metadata layer over a directory of Parquet (or ORC) files:

- **Snapshots** give atomic commits, consistent reads while writers commit, and time travel.
- **Manifests** list the data files of a snapshot with their partition values and per-column statistics, so query planning prunes files before opening any of them: the same trick as a Parquet footer, one level up.
- **Row-level deletes** use delete files or deletion vectors, so updates do not rewrite whole files; readers merge them at scan time.

From an engine's point of view, the scan first plans over table metadata, then over file footers, then reads pages, which is the hierarchy above, top to bottom.

Self-test and reading list

1. What does PAX combine from row and column storage, and what decides a good row-group size?

2. Why is Parquet's metadata in a footer at the end of the file, and what does a reader fetch first?
3. When does a Bloom filter help where min/max statistics do not?
4. Encode records with tags [x], [], [y, z] using repetition and definition levels.
5. Why must readers convert Dremel levels into offsets and nulls, and why can that be slow?
6. What problem do DWRF flat maps solve for ML feature tables?
7. Why does row order affect Capacitor's compression, and why can a managed service rewrite storage while a data lake usually cannot?
8. Show how `SUM(x)` and `x = 5` can run on RLE-encoded data without decoding.
9. Why does range filtering on dictionary codes require an order-preserving dictionary?
10. List the pruning levels from partition to value. Which ones become useless on randomly ordered data?
11. What does an Iceberg manifest contain, and how does it change scan planning?

Reading list:

1. Melnik et al., "Dremel: Interactive Analysis of Web-Scale Datasets", VLDB 2010 (section on nested columnar storage).
2. Google Cloud blog, "Inside Capacitor, BigQuery's next-generation columnar storage format" (2016).
3. Edara and Pasumansky, "Big Metadata: When Metadata is Big Data", VLDB 2021.
4. Abadi, Madden, Ferreira, "Integrating Compression and Execution in Column-Oriented Database Systems", SIGMOD 2006.
5. Kuschewski et al., "BtrBlocks: Efficient Columnar Compression for Data Lakes", SIGMOD 2023.
6. Zeng et al., "An Empirical Evaluation of Columnar Storage Formats", VLDB 2024 (Parquet vs ORC, measured).
7. Apache Parquet format specification, and the Apache Iceberg table spec.

Also: the Nimble repository (github.com/facebookincubator/nimble), whose README links to a talk on its internals.

Next: Module 6, distributed query execution (shuffle, partitioning, broadcast vs. shuffle joins, skew, scheduling, fault tolerance, adaptive execution).

Module 6 — Distributed Query Execution

Oct 13, 2026

Why this module

Modules 2 to 5 were about one worker: how it executes a pipeline, what its operators do, and how it reads storage. This module is about what changes when a query runs on hundreds or thousands of workers: data has to move between them, work has to be spread evenly, some workers will be slow or fail, and the plan made before the query started will often be wrong.

Each section takes one of those problems and compares how the main engines solve it: Presto/Trino, Spark SQL, Snowflake and BigQuery. Module 7 then goes deep on BigQuery's specific design.

Architecture

Every distributed analytical engine has the same three roles: a **coordinator** that plans and schedules, **workers** that execute tasks, and **storage**. The big architectural choice is whether storage lives on the workers or apart from them.

The roles

- **Coordinator** (Presto/Trino coordinator, Spark driver, Snowflake cloud services, BigQuery's query master): parses, optimizes, splits the plan into stages (Module 1), assigns splits and tasks to workers, tracks progress, and assembles the result.
- **Workers**: run tasks, each a fragment instance over some splits or partitions (Modules 2 and 3), and exchange data with each other.
- **Metadata and catalog**: schemas, table locations, statistics, file lists (Hive Metastore, Iceberg catalogs, BigQuery's metadata service).
- **Storage**: where table data lives.

Shared-nothing vs. disaggregated

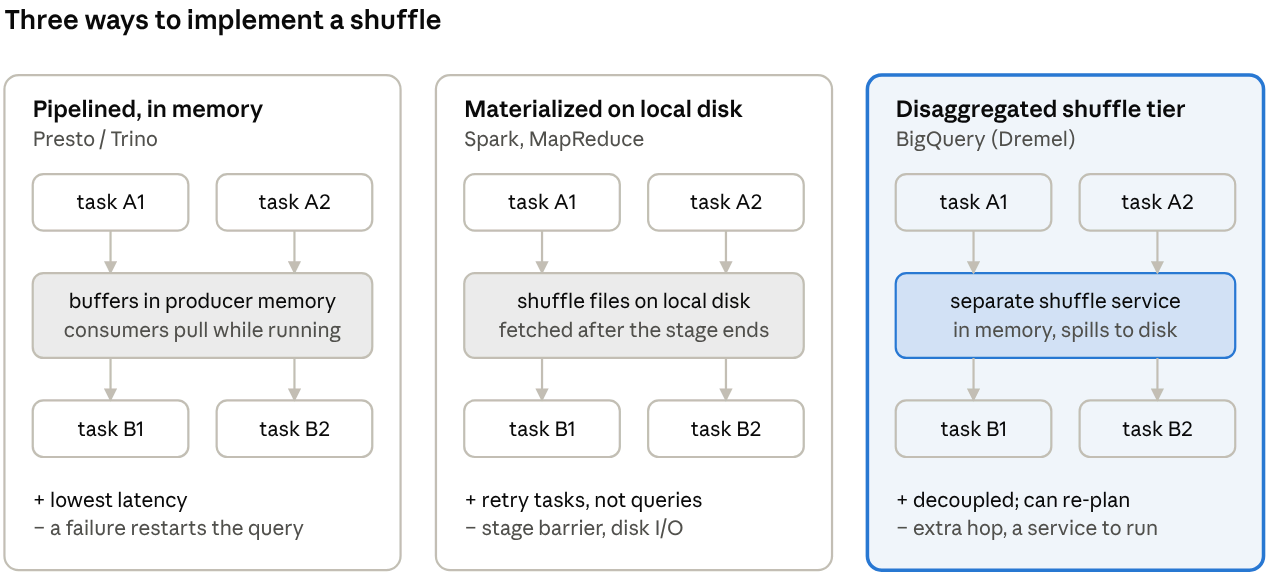
	Shared-nothing (classic MPP)	Disaggregated storage and compute
Data location	Partitioned across the workers' local disks	In a separate storage service (object store, Colossus)
Examples	Teradata, Greenplum, early Redshift, Vertica	BigQuery, Snowflake, Presto/Trino and Spark on data lakes, Redshift RA3
Scaling compute	Requires redistributing data	Add or remove workers freely; any worker can read any data
Data locality	Scans are local; co-located joins avoid shuffles	Every scan crosses the network; caching recovers some locality
Failure of a worker	Its data partition must be recovered or replicated	Nothing durable is lost; reassign its work

The industry has moved almost entirely to disaggregation, because network bandwidth in data centers grew faster than disk bandwidth, and because elastic, multi-tenant cloud services need stateless compute. The price is that every byte is remote, which is why Module 5's pruning and caching matter so much.

BigQuery goes one step further: besides separating storage from compute, it separates **shuffle memory** from compute, in a dedicated tier. That is the subject of the next section.

Shuffle

A shuffle moves every row of a stage to the task that owns its partition: an all-to-all transfer between M producers and N consumers. Where the data sits *between* producer and consumer is the single most consequential design decision in a distributed engine, because it decides latency, fault tolerance and how much the plan can change mid-query.



shuffle designs · pipelined, materialized, disaggregated

Pipelined, in memory (Presto/Trino)

Producers partition their output into in-memory buffers, one per consumer, and consumers pull over HTTP *while producers are still running*. All stages of a query run at the same time, so the first rows can reach the client before the scans finish. This is the fastest design for interactive queries. The cost: buffered data exists only in the producers' memory, so losing a worker loses data that cannot be recovered, and the classic answer is to restart the whole query. Backpressure (Module 2) is what keeps producer buffers bounded.

Materialized on local disk (Spark, MapReduce)

Each producer (map) task writes its complete output to local disk, sorted or grouped by destination partition, with an index file. Consumer (reduce) tasks start after the producing stage finishes and fetch their partition from every producer's files. Because outputs are durable, a failed consumer just re-fetches, and a lost producer's output can be recomputed from lineage. The costs: a hard barrier between stages, disk writes for every shuffled byte, and M × N small reads, which becomes a bottleneck at scale. Hence external shuffle services, and push-based shuffle (LinkedIn's Magnet, VLDB 2020), which merges blocks per partition on remote nodes into larger files.

Disaggregated shuffle tier (BigQuery)

Producers write into a separate, distributed shuffle service that holds partitions in memory and spills to disk when needed; consumers read from it. Compute and shuffle scale independently, a stage's output survives the loss of the producing worker, consumers can start before producers finish or after, and, crucially for Module 7, the coordinator can inspect a complete stage output in the tier and change the plan before the next stage reads it. The cost is an extra network hop and a large service to operate. Research on fast, disaggregated temporary storage for shuffle (for example Apache Crail and Exoshuffle) explores the same direction for open-source engines.

Practical details shared by all three

- **Partition count**: more partitions than consumers lets work be rebalanced later; too many create tiny partitions and per-partition overhead. Adaptive engines start high and coalesce (see Adaptive execution).
- **Compression and serialization**: shuffled data is compressed (LZ4 or ZSTD), usually in a columnar page format, because network bandwidth, not CPU, is typically the limit.
- **Local exchange first**: when consumer and producer happen to share a machine, engines can skip the network, but they cannot count on it.

Distributed joins and aggregations

Module 1 introduced broadcast, shuffle and co-located joins and partial/final aggregation. In a real plan, the larger savings come from *not shuffling at all*: reusing an existing partitioning, broadcasting several small tables at once, and shrinking data before it moves.

Reuse partitioning instead of reshuffling

The optimizer tracks which columns each stage's output is hash-partitioned on (a physical property, Module 1). If a join on `cust_id` is followed by `GROUP BY cust_id`, the join output is already partitioned correctly, so the aggregation needs no second shuffle. The same holds for joining the result to a third table on the same key. Tables stored pre-partitioned on a key (bucketed tables in Hive, Spark and Trino) allow **bucket joins**: matching buckets are joined directly, without shuffling either side.

Star joins: broadcast all the dimensions

A fact table joined to several small dimension tables can broadcast every dimension to the fact table's scan tasks, so the billion-row fact table never moves. Combined with dynamic filters from those dimensions (Module 4), the fact scan also reads far less.

Shrink before moving

- **Partial aggregation** before the shuffle (Module 1).
- **Semi-join reduction / dynamic filtering**: send the build side's key set or a Bloom filter to the probe side's scan stage *before* it shuffles, so rows that cannot match are dropped at the source. A Bloom filter of a few MB can save hundreds of GB of shuffle.
- **Projection and filter pushdown**: never shuffle a column or row the rest of the plan does not need.

Aggregation topologies

- **Shuffle-based** (the default): partial aggregation, hash shuffle on the group keys, final aggregation. Scales to any number of groups.
- **Tree aggregation**: for global aggregates or very few groups, partial results flow up a tree (leaves to intermediate servers to root), bounding fan-in at each level. This was the original Dremel serving tree; BigQuery has since generalized it into shuffle-based DAGs (Module 7).
- **Distinct and holistic aggregates** (COUNT DISTINCT, MEDIAN, exact percentiles) do not reduce in a partial step, so they need the two-level rewrite (Module 4) or approximate sketches that do merge (HyperLogLog, t-digest, KLL).

Non-equality joins

Cross joins and joins on `a.ts BETWEEN b.start AND b.end` cannot hash-partition on the condition, so engines broadcast the smaller side and run a nested-loop join locally. Range or band joins can instead be range-partitioned on the interval boundaries, at the cost of replicating rows whose interval spans several partitions.

Skew

A stage finishes only when its slowest task finishes. Skew is anything that makes one task much slower or larger than the median, and on large clusters it, not average throughput, usually decides how long a query takes.

Kinds of skew

Kind	Cause	Example
Key skew	One key holds a large share of rows	`country = 'US'`, `user_id = 0` placeholder, NULL keys
Partition skew	Several medium keys hash into the same partition, or too few partitions	200 partitions for 150 distinct keys
Processing skew	Same row count, very different cost per row	A few rows with huge JSON blobs or arrays
Input skew	Uneven input splits	One 50 GB file next to many 100 MB files
Machine skew	A slow or overloaded worker	Noisy neighbor, failing disk

Detection

- **Before execution**: sampling, or column statistics with heavy hitters (most frequent values).
- **During execution**: a materialized shuffle reveals the exact bytes and rows per partition. A partition far above the median (Spark's default rule: more than 5 times the median and above a size threshold) is declared skewed. This is one more benefit of materializing stage outputs.

Mitigations

1. **Let partial aggregation absorb it.** For decomposable aggregates, each producer collapses the hot key into one row before the shuffle, so key skew mostly disappears. Holistic aggregates do not get this benefit.
2. **Broadcast instead of shuffle**, when the other join side is small: no partitioning, so no partition can be hot.
3. **Split the skewed partition** (Spark AQE skew join): divide the large side's hot partition into k pieces processed by k tasks, and replicate the matching partition of the other side to each of them.

4. **Salting**: append a random salt 0..k-1 to the hot key on the large side and replicate the other side's matching rows k times, once per salt. This is the same idea as splitting, done in the plan instead of at runtime.
5. **Heavy-hitter handling**: route rows of known hot keys differently, broadcasting the small side's hot-key rows and leaving the large side's hot-key rows where they are, while all other keys shuffle normally (Teradata's partial redistribution / partial duplication).
6. **Over-partition and schedule dynamically**: many more partitions than workers, assigned as workers free up, so several medium partitions spread out (morsel thinking, Module 2, applied to a cluster). This handles partition and machine skew, but not a single giant key.

What no technique fixes: a key that is hot on *both* sides of a join produces |hot rows of R| × |hot rows of S| output rows. That output has to be produced somewhere; splitting only spreads it.

Scheduling and stragglers

Scheduling happens at two levels: across queries (who gets the cluster) and within a query (which stages run when, and which task runs where). Both are about using all resources without letting one slow piece hold everything back.

Across queries

- **Admission and queues**: resource groups in Trino, pools and queues in Spark and YARN, warehouses in Snowflake.
- **BigQuery slots**: a slot is BigQuery's unit of compute (roughly a share of CPU, memory and network). A query's work is broken into units that run on whatever slots are available, and **fair scheduling** redistributes slots among concurrent queries and projects as they start and finish, with idle capacity shareable between reservations. A query does not get a fixed set of machines; it gets a changing share of a pool.

Within a query: when stages start

Policy	How	Used by	Trade-off
All at once	Schedule every stage immediately, stream data between them	Presto/Trino (default)	Lowest latency; holds all resources for the whole query
Phased	Start stages in dependency order where they would only wait (e.g. build sides before probes)	Trino option	Less idle resource use, slightly later first results
Stage by stage	Start a stage only when its inputs are complete	Spark, MapReduce	Simple, retryable, adaptive; full barrier between stages

Within a query: where tasks run

In a disaggregated engine, data locality is replaced by **cache affinity**: send a split preferably to the worker that has read that file before, so its local cache (Module 3) is warm. Trino and Presto do this with soft affinity, Snowflake by consistently hashing files to warehouse nodes.

Stragglers

One slow task delays the whole stage. Three standard remedies:

- **Speculative execution** (from the MapReduce paper): when a task runs far slower than its peers, launch a backup copy elsewhere and keep whichever finishes first. It requires tasks that are deterministic and safe to run twice, and inputs that can be re-read, which is easy with materialized shuffles and hard with pipelined ones.
- **Work stealing**: idle workers take not-yet-started work from busy ones. Snowflake describes stealing unscanned files from a slow worker's scan.
- **Smaller work units**: many small splits or partitions instead of one per worker, so a slow worker simply takes fewer of them.

Fault tolerance

With thousands of workers, a long query will regularly see a failure. Engines choose between restarting the whole query, which is cheap when queries are short, and retrying only the failed piece, which requires keeping intermediate data somewhere that survives the failure.

The spectrum

1. **Restart the query** (classic Presto/Trino): no overhead while things go well. Works when queries are short; the chance of a failure during a query grows with duration × number of machines, so long ETL jobs suffer.
2. **Retry tasks from materialized inputs** (MapReduce, Spark, Trino's fault-tolerant execution): stage outputs are written somewhere durable enough (local disk with a shuffle service, or object storage), so a failed task is re-run alone. Trino added this mode (Project Tardigrade) by spooling exchange data to S3, GCS or HDFS, as an opt-in for long-running queries.
3. **Recompute from lineage** (Spark): if materialized outputs are lost with their machine, recompute just the lost partitions from their parents, walking back the stage graph as far as needed.
4. **Disaggregated shuffle** (BigQuery): stage outputs live in the separate shuffle tier, so losing a worker loses only its in-progress work, which is rescheduled on another slot.

The subtle part: determinism

Retrying a task is only safe if it produces the same output as the first attempt, or if consumers can tell attempts apart:

- **Duplicate output**: if a failed attempt already sent some rows downstream, the retry must not deliver them twice. Exchanges tag data with task attempt ids and keep only one attempt's output.
- **Non-determinism**: `RAND()`, `LIMIT` without `ORDER BY`, or arrival-order-dependent operators can produce *different* rows on retry. A consumer that had read the first attempt's rows and then reads the retry's would see an inconsistent mix. Engines either guarantee deterministic partitioning, or re-run all consumers of a non-deterministic stage.

Choosing

Interactive engines optimize for the common case (no failure) and restart on failure; batch engines pay a constant overhead to survive failures cheaply. The trend is to offer both: Trino's retry policies, and BigQuery's design, where the materialization the shuffle tier performs anyway also provides the recovery point.

Adaptive execution

Module 1 showed that cardinality estimates are routinely off by orders of magnitude. Adaptive execution accepts that and moves decisions to the point where real numbers are known: inside an operator, at a stage boundary, or in the next run of the same query.

Three places to adapt

1. **Inside an operator, continuously**: switch off partial aggregation when it doesn't reduce (Module 1), reorder filters by measured selectivity (Modules 2 and 3), choose a hash-table mode from observed keys (Module 4). No re-planning needed.
2. **Between stages, from real statistics**: once a stage's output is materialized, its exact size and per-partition distribution are known, and the rest of the plan can be re-optimized before it starts.
3. **Across runs, from history**: reuse statistics from previous executions of the same or similar query.

Between stages: Spark AQE as the reference design

Spark's Adaptive Query Execution (Spark 3.0) re-optimizes the remaining plan after every shuffle stage, using its map output statistics:

- **Coalesce partitions**: merge many small shuffle partitions into fewer reasonably sized ones, so starting with a high partition count costs little.
- **Switch join strategy**: if one side turns out small, replace the planned sort-merge join with a broadcast hash join.
- **Split skewed partitions** (the skew join from the Skew section).

BigQuery's **dynamic query execution** follows the same principle with its shuffle tier as the source of truth, and goes further, adjusting parallelism and plan shape as stages complete (Module 7).

Runtime information pushdown

Dynamic filtering (Presto/Trino) and dynamic partition pruning (Spark) are adaptivity of another kind: they don't change the plan's shape, but they push information learned at runtime, the set of build-side keys, into scans that have not run yet.

Across runs: history-based optimization

Recurring queries (dashboards, scheduled reports) run with similar data again and again, so their actual cardinalities are better than any estimate. Presto's history-based optimizer (from Meta) and BigQuery's history-based optimizations both record per-plan-node statistics and use them for later runs: join order, join strategy, partition counts, and which queries can run as a single stage.

The tension

Re-planning needs a point where a stage's output is complete and inspectable, which is exactly what pipelined execution avoids. Engines resolve this by materializing at shuffle boundaries (Spark, BigQuery) and accepting the barrier, or by staying pipelined and limiting adaptivity to inside operators and runtime filters (classic Presto).

Engine comparison

The four engines differ mostly in where they put intermediate data, and the other columns follow from that choice.

	Presto / Trino	Spark SQL	Snowflake	BigQuery
Worker execution	JVM vectorized pages; Presto C++ uses Velox	JVM whole-stage codegen; Photon or Gluten+Velox as vectorized options	C++, vectorized, push-based	C++, vectorized (advanced runtime)
Shuffle	Pipelined, in memory, HTTP pull	Materialized to local disk; external or push-based shuffle service	Direct worker-to-worker exchange, spilling locally	Separate in-memory shuffle tier

	Presto / Trino	Spark SQL	Snowflake	BigQuery
Stage scheduling	All at once (phased optional)	Stage by stage	Pipelined within a warehouse	Dynamic, over shared slots
Fault tolerance	Restart query; task retry with spooling (Trino, opt-in)	Task retry, lineage recompute	Transparent query retry	Reschedule work from shuffle tier
Adaptivity	Dynamic filters, in-operator	AQE between stages, dynamic partition pruning	Runtime decisions, file stealing	Dynamic query execution, history-based optimizations
Storage	Data lake (Hive, Iceberg, Delta) and connectors	Data lake	Own micro-partition format on object storage	Capacitor on Colossus; open formats via BigLake

Read it as one design space: pipelining buys latency, materialization buys recovery and adaptivity, and BigQuery's shuffle tier is an attempt to get most of both by making materialization fast enough to sit on the critical path.

Self-test and reading list

1. Why did the industry move from shared-nothing to disaggregated storage, and what did it cost?
2. Compare the three shuffle designs on latency, fault tolerance and adaptivity.
3. Why does materializing shuffle output make speculative execution and adaptive re-planning possible?
4. A join on `cust_id` is followed by `GROUP BY cust_id`. How many shuffles are needed, and why?
5. How does a Bloom filter shipped before a shuffle reduce network traffic?
6. Why does partial aggregation neutralize key skew for SUM but not for MEDIAN?
7. Explain how Spark AQE splits a skewed join partition. What does it replicate?
8. What kind of skew can no technique fix, and why?

9. What is a BigQuery slot, and how does fair scheduling change a query's resources over time?
10. Why is retrying a task unsafe with non-deterministic operators, and what do engines do about it?
11. Name the three places adaptivity can happen, with an example of each.

Reading list:

1. Melnik et al., "Dremel: A Decade of Interactive SQL Analysis at Web Scale", VLDB 2020 (shuffle tier and dynamic execution; the main reading for Module 7).
2. Sethi et al., "Presto: SQL on Everything", ICDE 2019.
3. Armbrust et al., "Spark SQL: Relational Data Processing in Spark", SIGMOD 2015; and the Databricks blog on Adaptive Query Execution in Spark 3.0.
4. Dageville et al., "The Snowflake Elastic Data Warehouse", SIGMOD 2016.
5. Shen et al., "Magnet: Push-based Shuffle Service for Large-scale Data Processing", VLDB 2020.
6. Dean and Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters", OSDI 2004 (backup tasks).
7. Trino documentation on fault-tolerant execution.

Next: Module 7, BigQuery and Dremel in depth.

Module 7 — BigQuery and Dremel

Oct 14, 2026

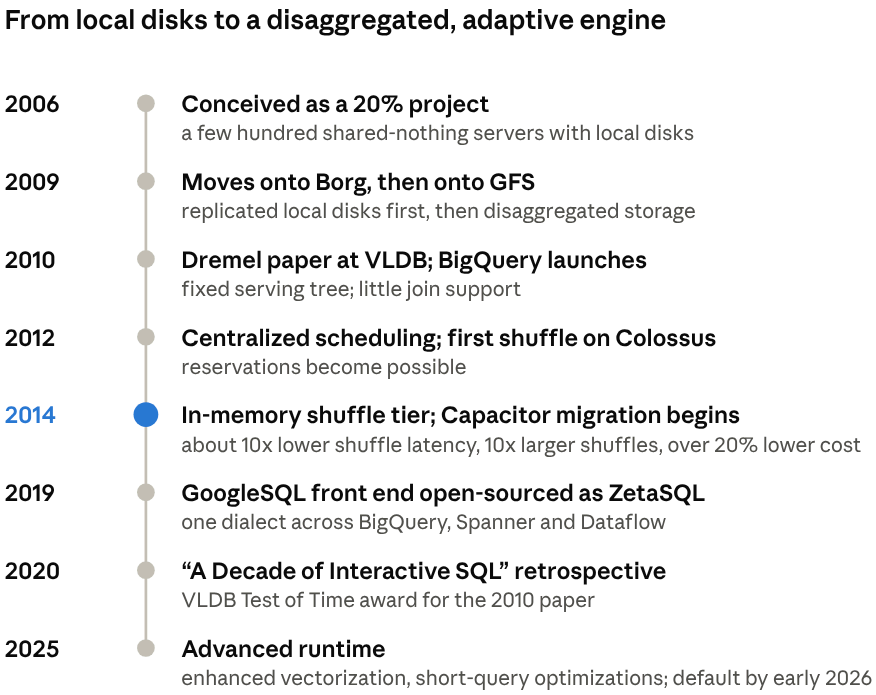
Why this module

Every earlier module ended with "how BigQuery does it". This module puts those pieces together into one system, using what Google has published: mainly the 2020 retrospective "Dremel: A Decade of Interactive SQL Analysis at Web Scale", plus the Capacitor and advanced-runtime material from Modules 2 and 5.

The through-line is a series of decisions that each looked expensive at the time: give up local disks, give up the fixed serving tree, put shuffle memory in its own service, and accept that plans will be wrong and fix them at runtime. Most of BigQuery's distinctive behavior follows from those four choices.

A decade in one timeline

Dremel started as a classic shared-nothing system and spent the following decade disaggregating one resource after another: first storage, then shuffle memory, then scheduling decisions themselves.



Dremel and BigQuery milestones, 2006 to 2025

The 2014 in-memory shuffle tier is the pivot: almost everything later in this module (flexible stage DAGs, preemption, dynamic re-planning, joins at scale) depends on it.

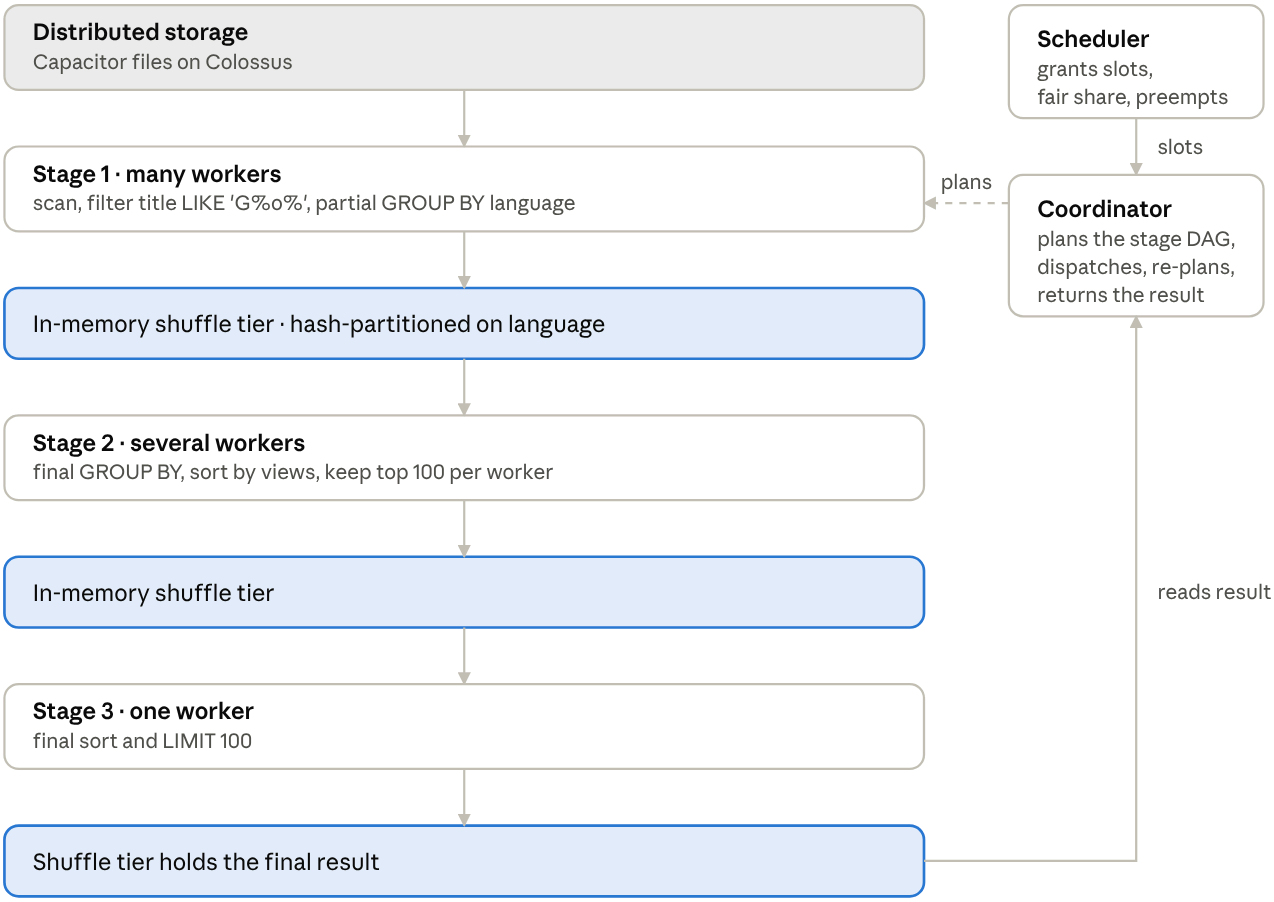
Architecture today

BigQuery separates four things that classic databases keep on the same machines: storage, compute, shuffle memory and scheduling. Workers hold no durable state; every stage reads from storage or the shuffle tier and writes back to the shuffle tier.

The paper illustrates this with a top-k query over a Wikipedia table:

```
SELECT language, MAX(views) AS views
FROM `wikipedia_benchmark.Wiki1B`
WHERE title LIKE "G%o%"
GROUP BY language
ORDER BY views DESC LIMIT 100
```

Stateless workers between storage and an in-memory shuffle tier



BigQuery execution · the paper's top-100 languages query

The components:

- **Coordinator**: the first node to receive the query. It builds the plan as a DAG of stages, obtains workers from the scheduler, sends each worker a ready-to-execute local plan, and after the last stage reads the result from the shuffle tier and returns it to the client.
- **Scheduler**: one centralized scheduler with a view of the whole cluster, allocating slots to queries with fair sharing and reservations, and able to preempt.
- **Workers**: an unstructured pool, not a fixed tree. Leaf-stage workers read from storage; all other stages read from and write to the shuffle tier.
- **Shuffle tier**: distributed transient memory (spilling to disk) that holds every stage's output, including the final result.

Mapped onto Module 1: each stage is a fragment, the three stages are separated by two exchanges, and the plan is exactly the partial-aggregate, shuffle, final-aggregate, gather pattern from the worked example. The paper stresses that a query like this can run on anywhere from one to tens of thousands of workers, and the shuffle tier is what makes that possible.

The shuffle persistence layer

Shuffle came to Dremel only after the 2010 paper, and its implementation went through three generations before it became the component the rest of the system is built around.

Three generations

1. **Local RAM and disk on the workers**, modeled on MapReduce's shuffle. Two problems: the M × N growth of shuffle connections could not be mitigated while producers and consumers were co-located, and tying shuffle data to compute nodes fragmented and stranded resources and isolated tenants poorly.
2. **On Colossus** (2012): disaggregated, but slow, with the problems known from other file-system-based shuffles.
3. **A distributed in-memory transient storage system** (2014), managing RAM and disk for shuffle data separately from compute.

What the in-memory tier delivered

According to the paper, the third generation:

- cut shuffle latency by an order of magnitude;
- made shuffles an order of magnitude larger possible;
- cut the service's resource cost by more than 20%.

Disaggregated memory now makes up about **80% of BigQuery's total memory footprint**, and the same shuffle infrastructure also powers Google Cloud Dataflow.

Why it is more than a faster shuffle

Because a stage's output lives in the tier rather than on the workers that produced it, the shuffle doubles as a **checkpoint of the query's execution state**. That single property enables:

- **Decoupled stages**: producers and consumers of a shuffle need not run at the same time or on the same number of workers.
- **Preemption**: the scheduler can take workers away from a running query to serve other workloads, and give them back later, without losing completed work.
- **Cheap recovery**: a failed worker loses only its in-progress task (Module 6).
- **Dynamic re-planning**: the coordinator sees each stage's real output before deciding how to run the next one (see Dynamic query execution).

The paper calls this "value-added repackaging": raw RAM is hard to disaggregate for general use, but packaged as a shuffle service it becomes cost-effective.

Serverless execution

Dremel was multi-tenant and pay-per-use from the start, years before "serverless" was a product category. The paper names three founding ideas, and later additions built on each.

The three founding ideas

1. **Disaggregation** of compute, storage and (later) memory, so each scales and is shared independently.
2. **Fault tolerance and restartability**: workers are assumed unreliable, so every subtask must be deterministic and repeatable, and the dispatcher must be able to run several copies of the same task. A side effect is that the scheduler can change a query's resources at will, by cancelling and rescheduling subtasks.
3. **Virtual scheduling units (slots)**: abstract units of compute and memory, independent of machine or container shapes. Slots remain the customer-visible unit of resource management.

What came later

- **Centralized scheduling** (2012) replaced the original per-intermediate-server dispatchers with one scheduler that sees the whole cluster's state. It enabled finer-grained allocation and **reservations**, a guaranteed share of capacity for a customer.
- **Guaranteed capacity, without waste**: unused reserved capacity is lent to others, but returned *immediately* when the owner needs it. Workers run a custom thread scheduler that instantly reallocates CPUs to reserved work and pauses non-reserved work.

- **Flexible execution DAGs**: workers form an unstructured pool; the coordinator decides each query's DAG shape and how many workers each stage gets.

How this maps to earlier modules

Deterministic, repeatable tasks are the precondition Module 6 named for speculative execution and safe retries. Thousands of small tasks per query are morsel-driven parallelism (Module 2) applied to a cluster. And slots shared fairly across queries are why a BigQuery query's parallelism changes over its lifetime rather than being fixed at submission.

Dynamic query execution and adaptive scaling

The paper is explicit that accurate cardinality estimates are hard to get, since errors propagate exponentially through joins, and that a large share of Dremel queries run over data seen for the first time, with no statistics at all. Dremel's answer is to let the plan change during execution, using statistics collected while the query runs.

The broadcast-or-hash example

The paper's own example of dynamic query execution:

1. Start with a **hash join**, shuffling *both* inputs on the join key.
2. If one side finishes quickly and turns out smaller than the broadcast threshold, **cancel the other side's shuffle** while it is still running.
3. Switch to a **broadcast join**: the small side goes to every probe worker, and the large side never shuffles.

Two things make this possible: the shuffle tier holds the finished side's output, so it can be broadcast from there, and the coordinator orchestrates centrally, so it can cancel and re-plan in-flight work. Compared with Spark AQE (Module 6), which decides at stage boundaries, this changes course in the middle of a running stage.

Adaptive query scaling

The DAG shape itself is chosen per query and per input size:

- **Global aggregation** (`COUNT` , `SUM` without GROUP BY): never more than two levels. Leaf workers aggregate their input to one row per file, and one final stage combines them. The old fixed serving tree forced extra hops through intermediate levels.

- **Top-k** (`ORDER BY ... LIMIT`): every leaf produces many candidate rows, so a single final node would be a bottleneck. Dremel builds an aggregation tree whose *depth depends on the input size*.
- **Stage parallelism**: since the stages are decoupled by the shuffle tier, each one's worker count can be decided when it starts.

Learning across runs

Current BigQuery adds two mechanisms that use history rather than just the current run:

- **History-based optimizations** reuse statistics from earlier executions of similar queries (Module 6).
- **Short query optimizations** (part of the advanced runtime) identify, dynamically, queries that can run as a **single stage** without a shuffle, using filter selectivity, query structure and past executions. This lowers latency and slot use for small queries, and lets encoded data flow further without being decoded.

The scan path: Capacitor's embedded evaluation

Capacitor's most important design choice is that filtering lives *inside the storage library*: the Capacitor access library contains a small query processor that evaluates SQL predicates. Every system reading Capacitor files gets the same fast filtering: Dremel, F1, Procella, Flume, MapReduce, and BigQuery's Storage API, which accepts predicates as SQL strings in its read options.

The paper's motivation is a measurement: BigQuery filters are very selective. About 15% of queries return no data at all, about 25% return less than 0.01% of the data, and about half return less than 1%. A scan that can skip fast wins.

Five techniques

1. **Partition and predicate pruning with rewrites.** Statistics per column eliminate partitions that cannot match and remove tautologies. Predicates are rewritten into prunable form first: `EXTRACT(YEAR FROM date) = 2020` becomes `date BETWEEN '2020-01-01' AND '2020-12-31'`. Geographic predicates like `ST_DISTANCE(geo, constant) < 100` are pruned by comparing S2 cell coverings against per-file coverings.
2. **Vectorized evaluation**, using the compressed-execution techniques of Abadi et al. from Module 5.

3. **Skip indexes.** At write time, each column's values are grouped into *segments* compressed separately, and the column header indexes the segment offsets. Under a selective filter, segments with no hits are skipped without being decompressed. This is the Capacitor counterpart of Parquet's page index.
4. **Predicate reordering by heuristics**, since true selectivities and costs are unknown: dictionary use, distinct-value count, NULL density and expression complexity. For `p(x) AND q(y)` where `y` is dictionary-encoded with few values and `x` is not, evaluate `q(y)` first even if it is the more complex expression, because it runs only on the handful of dictionary entries.
5. **Row reordering** for better RLE (Module 5): finding the optimal order is NP-complete, so Capacitor uses sampling and heuristics, weighting columns by type and by how often queries select or filter on them. On 40 internal datasets this saved 17% overall, up to 40% on some and 75% on one.

Nested data: a measured trade-off

The paper compares Dremel's repetition and definition levels with ORC-style length and presence encoding (Module 5) on 65 internal datasets with deep, wide records. Length and presence encoding produced files about **13% smaller** on average, because levels repeat ancestor information in every leaf column. But it requires reading ancestor columns to process a leaf, while levels make each column self-contained. The paper calls quantifying this trade-off open research.

Capacitor also added recursive schemas of arbitrary depth, and partial support for schemaless data, later extended for the JSON type (Module 5).

Interactive latency techniques

The paper is candid that its own design principles work *against* low latency: disaggregation moves data away from compute, in situ analysis prevents optimizing layout, and shared serverless machines are less predictable than dedicated ones. Interactive speed came from a long list of targeted fixes.

A telling example: the first GFS-based Dremel was **an order of magnitude slower** than the local-disk version. Scanning a table meant opening hundreds of thousands of files, which alone took seconds, and the metadata format had been designed for disk seeks, not network round trips. Fine-tuning the storage format, metadata representation, query affinity and prefetching eventually made the disaggregated system faster than the local-disk one in both latency and throughput.

Technique	Problem it addresses
Stand-by server pool	No machine allocation, binary copying or startup per query
Thousands of small tasks, plus duplicates for stragglers	The slowest of hundreds of workers can be 10× slower than average; fast workers take more tasks and stragglers get backup copies
Column-oriented schema representation	Schemas with thousands of fields took longer to parse than the data took to scan
Lightweight compression	Balance bytes read against CPU spent decompressing
Approximate results	One-pass sketches for top-k and distinct counts; optionally stop at, say, 98% of the data, which improved latency 2–3× because the tail is all stragglers
Latency tiers and fair dispatching	Small queries must not wait behind large ones; parts of a query can be preempted
Reuse of file metadata	Fetch file-system metadata in one batch and pass it down to the workers, instead of thousands of workers each asking; prefer fewer, larger files
Guaranteed capacity	Reserved capacity returns to its owner immediately when needed
Adaptive query scaling	DAG depth sized per query (previous section)

The common thread with Module 6: at scale, latency is set by the tail (the slowest task, the slowest metadata call, the most loaded server), so most fixes target variance, not average speed.

The runtime today

The 2020 paper says little about how a worker executes its local plan. Google's 2025 material on the **advanced runtime** fills some of that gap, and it reads like a checklist of Modules 2 to 5 applied to BigQuery.

Enhanced vectorization

- **Vectorized execution** as the base model: columns processed in cache-sized blocks with SIMD (Module 2).
- **Filters evaluated on encoded data** using Capacitor's specialized encodings (Module 5).
- **Encodings propagated through the plan**, so data stays encoded across operators rather than being decoded right after the scan. This is the same idea as Velox keeping dictionaries and peeling them (Module 3).
- **Expression folding**: deterministic functions over constants are evaluated once and complex predicates simplified to constants (Module 1's constant folding, applied in the runtime).
- **Parallel joins and aggregations inside a leaf**: for some execution modes, a leaf node builds and probes the hash table with multiple threads (the shared-hash-table model of Module 2 and Velox's join bridge).
- **One engine for storage and execution**: Capacitor uses the same vectorized engine for its pushed-down filters and computations.

Google's blog post reports one query going from 61 seconds to 2.9 seconds, about 21×, with enhanced vectorization.

Short query optimizations

Queries that qualify run as a single stage, skipping the shuffle tier entirely (see Dynamic query execution). That trades the shuffle's flexibility for latency exactly when the flexibility isn't needed.

Rollout

Google announced the advanced runtime becoming the default for all projects between September 15, 2025 and early 2026, with no query changes required.

Lessons, and how BigQuery compares

What the authors say they got right and wrong

In their own assessment, the original paper **got right**: disaggregated compute and storage, on-demand serverless execution, columnar storage for nested data, in situ analysis, and interactive speed over huge datasets. All are now standard industry practice.

It **got wrong or missed** three things:

1. **A reliable, fast shuffle layer instead of a static aggregation tree.** The tree served aggregations well but not joins and complex plans; the shuffle tier is what made BigQuery a general SQL engine.
2. **Managed storage alongside in situ analysis.** In situ files burden users with governance, block DML and schema changes, and, critically for the runtime, leave no chance to optimize layout or gather statistics. A large share of queries run over data never seen before.
3. **Rigorous SQL language design.** Ad hoc dialects across Google systems led to GoogleSQL, a shared, standards-oriented front end, parser and function library, open-sourced as ZetaSQL.

BigQuery next to a Velox-based engine

	BigQuery	Presto C++ or Spark with Gluten (Velox workers)
Front end	GoogleSQL, shared across Google's SQL systems	Presto or Spark planner; Axiom as a composable option
Worker execution	Proprietary vectorized C++ runtime, with encodings propagated through plans	Velox: vectorized, encoding-aware, adaptive per batch
Shuffle	Disaggregated in-memory tier, also the checkpoint	Host-provided: Presto's HTTP exchange, or Spark's shuffle service
Storage	Capacitor on Colossus, managed and re-optimized in the background; open formats via BigLake	Parquet, ORC, DWRF or Nimble on a data lake, managed by Iceberg, Delta or Hive
Filtering in storage	Embedded evaluator inside the Capacitor library	Selective readers inside Velox's file readers
Adaptivity	Mid-stage re-planning, history-based optimizations, single-stage short queries	Dynamic filters; Spark AQE; Presto history-based optimizer

	BigQuery	Presto C++ or Spark with Gluten (Velox workers)
Scheduling	One central scheduler, slots, reservations, preemption	Per-engine coordinator and cluster manager

The deepest difference is not the worker engine; Velox and BigQuery's runtime share most single-node ideas. It is that BigQuery owns every layer: it can reorder rows in storage, keep encodings alive into the shuffle, preempt workers, and rewrite a running plan, because no layer belongs to anyone else. Composable stacks like Velox plus Axiom trade some of that integration for reuse across many engines.

Self-test and reading list

1. Which four resources did Dremel disaggregate, and in what order?
2. What two problems with worker-local shuffle led to the separate in-memory tier?
3. Why is the shuffle tier also a checkpoint, and what four capabilities follow from that?
4. Why must every Dremel subtask be deterministic and repeatable?
5. Walk through the broadcast-or-hash runtime switch. How does it differ from Spark AQE's join conversion?
6. Why does a global aggregation need only two levels, while a top-k query gets a tree whose depth depends on input size?
7. Why put the filter evaluator inside the Capacitor library rather than in the query engine?
8. For `p(x) AND q(y)`, when should `q(y)` run first even if it is more expensive per value?
9. Why was the first GFS-based Dremel an order of magnitude slower, and what fixed it?
10. Why does stopping at 98% of the data improve latency 2–3×?
11. Repetition/definition levels vs. length/presence: what does each optimize for?
12. What did the authors say the original design got wrong?

Reading list:

1. Melnik et al., "Dremel: A Decade of Interactive SQL Analysis at Web Scale", VLDB 2020. Read the whole thing; it is 12 pages.
2. Melnik et al., "Dremel: Interactive Analysis of Web-Scale Datasets", VLDB 2010.

3. Google Cloud blog: "Inside Capacitor, BigQuery's next-generation columnar storage format" (2016), "In-memory query execution in Google BigQuery" (2016), and "BigQuery under the hood: Enhanced vectorization in the advanced runtime".
4. BigQuery documentation: "Use the BigQuery advanced runtime" and "history-based optimizations".
5. Edara and Pasumansky, "Big Metadata: When Metadata is Big Data", VLDB 2021.
6. Afrati, Delorey, Pasumansky, Ullman, "Storing and Querying Tree-Structured Records in Dremel", VLDB 2014.
7. Related Google systems: Samwel et al., "F1 Query: Declarative Querying at Scale", VLDB 2018; Chattopadhyay et al., "Procella", VLDB 2019.

Next: Module 8, the comparative landscape (Photon, Snowflake, Presto/Trino, Spark, DuckDB, Gluten).

Module 8 — The Engine Landscape

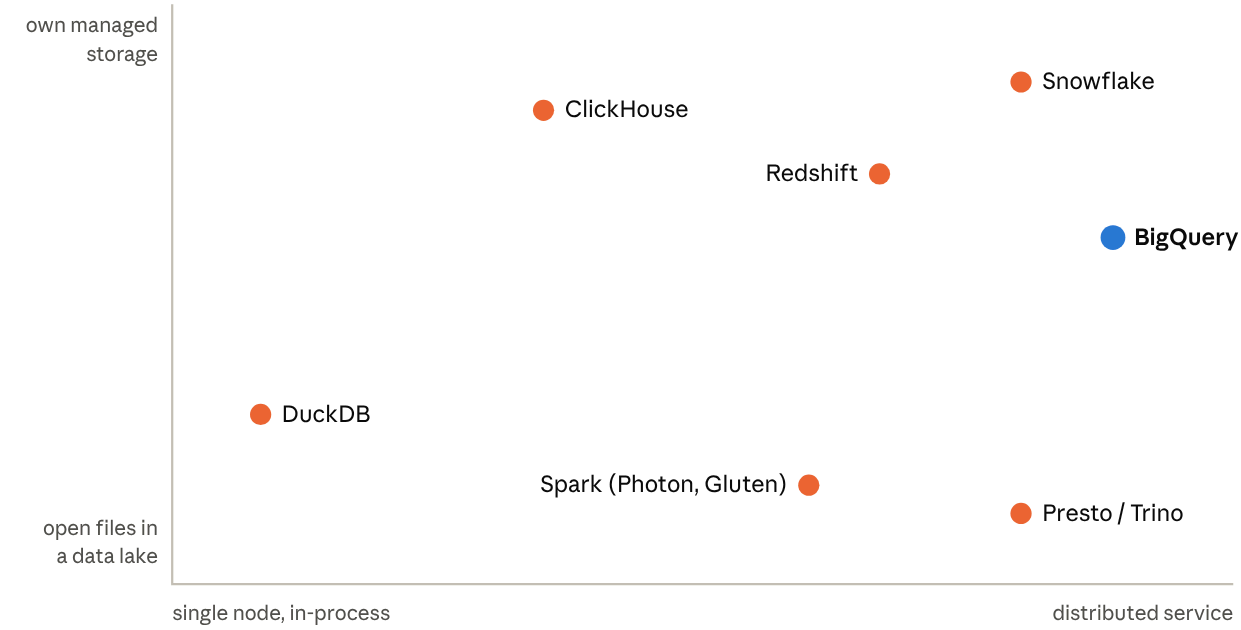
Oct 14, 2026

Why this module

Modules 1 to 7 built up the design space; this module walks through the engines you are most likely to meet and shows where each sits in it. For each one: where it came from, how it executes, where its data lives, and the one or two ideas that make it distinctive.

The most useful single picture is two questions: does the engine run inside one process or as a distributed service, and does it own its storage or read open files that other engines also read?

Where the engines sit: deployment vs. where data lives



engine positioning · qualitative, not measured

Owning storage (top) lets an engine control layout, statistics and encodings end to end, as BigQuery does with Capacitor. Reading open files (bottom) gives up some of that control in exchange for sharing data across engines. BigQuery sits between the two: managed Capacitor storage, plus Parquet and Iceberg through BigLake.

Presto and Trino

Presto was built at Facebook from 2012 as an interactive SQL engine over the company's Hive data warehouse, open-sourced in 2013, and later split into two projects: **Presto** (Linux Foundation, still led by Meta, IBM and others) and **Trino** (the fork by Presto's original creators, renamed from PrestoSQL in 2020, commercially backed by Starburst).

Distinctive ideas

- **Federation through connectors.** The engine has no storage of its own. Connectors expose Hive and Iceberg tables, relational databases, Kafka, Elasticsearch and more as tables, and a single query can join across them. Connectors can also accept pushed-down filters, projections, limits and aggregations.
- **Pipelined, all-stages-at-once execution** with an in-memory HTTP exchange (Module 6): the lowest-latency design for interactive queries.
- **A JVM engine that behaves like a native one**: columnar pages, bytecode generation for expressions, careful control of memory, and an explicit memory pool per query.

Where the two projects have diverged

- **Presto** is replacing its Java workers with **Presto C++ (Prestissimo)** on Velox (Modules 3 and 6), keeping the Java coordinator. Meta added a history-based optimizer, and is building **Axiom** as a reusable front end.
- **Trino** stayed on the JVM and invested in **fault-tolerant execution** (task retries with exchange spooling to object storage), lakehouse connectors (Iceberg, Delta), and polymorphic table functions.

Limits

The classic design restarts a whole query on failure and holds all of a query's stages in memory at once, which is why long ETL traditionally ran on Spark instead. Both projects have narrowed that gap, in different ways.

Spark SQL and Photon

Spark began at UC Berkeley's AMPLab as a faster MapReduce built on resilient distributed datasets (RDDs). Spark SQL (SIGMOD 2015) turned it into a SQL and DataFrame engine, and Databricks' **Photon** (SIGMOD 2022) later replaced much of its execution layer with native code.

Spark SQL

- **Catalyst** optimizer: logical and physical plans as Scala trees, rewritten by pattern-matching rules, with cost-based join reordering added later. Its extensibility is why so many projects plug into Spark.
- **Tungsten** execution: off-heap binary rows (`UnsafeRow`), cache-aware algorithms, and **whole-stage code generation**, which fuses a stage's operators into one generated Java function, JIT-compiled by the JVM (Module 2's compiled model).
- **Materialized shuffle and stage-by-stage scheduling** (Module 6): slower first results than Presto, but task-level retry, lineage recomputation and the natural barrier that makes **Adaptive Query Execution** possible.
- A general platform as much as a SQL engine: SQL and DataFrames share the engine with streaming, ML and arbitrary user code.

Photon

Photon is a vectorized C++ engine embedded inside Databricks' Spark runtime. Its paper is the clearest industry account of choosing vectorization over code generation:

- **Vectorized interpretation, not codegen.** The team prototyped both and chose vectorization for easier development and debugging, per-operator observability, and adaptivity, such as specializing per batch for no nulls or ASCII-only strings (Module 2).
- **Partial adoption**: Spark's planner stays in charge, Photon replaces the operators it supports, and the plan falls back to Spark's JVM operators for the rest, with transition nodes converting between columnar and row formats.
- **Shared resource management**: Photon's memory is accounted inside Spark's memory manager, so the two engines can share an executor and spill each other's state under pressure.
- **Lakehouse focus**: tuned for Delta Lake and Parquet, including the messy realities of data lakes such as raw strings, missing statistics and many small files.

Photon and Gluten (next) are two answers to the same question, how to make Spark fast without rewriting its planner: a proprietary native engine versus an open plugin over Velox.

Apache Gluten

Gluten, started at Intel and now an Apache project, is a Spark plugin that offloads execution to a native C++ backend, Velox (the main one) or ClickHouse, while Spark keeps its planner, scheduler and ecosystem. It is the open counterpart to Photon.

How a query flows

1. Spark plans the query as usual with Catalyst.
2. Gluten **validates** each physical operator and function against what the backend supports.
3. Supported subtrees are converted into **Substrait** plans and passed through JNI to the native backend, which builds and runs a Velox plan (Module 3).
4. Unsupported operators **fall back** to vanilla Spark, with row/column transition nodes at the boundaries.
5. Gluten provides a **columnar shuffle**, so data stays columnar between stages instead of being converted to Spark rows.

What makes it hard

- **Semantic fidelity**: results must match Spark exactly, including overflow, casts, null handling and timestamp behavior. This is why Velox ships a separate Spark function library (Module 3), and why fuzzers compare against Spark.
- **Fallback cost**: each transition between native and JVM operators converts data between columnar and row form. A plan that falls back in the middle of a hot loop can be slower than plain Spark.
- **Memory integration**: native allocations must be accounted inside Spark's task memory manager, so spilling and limits work across both engines.

Production users presented at VeloxCon include Pinterest and Microsoft. For a Spark user, Gluten is the quickest way to put Velox under existing jobs without changing code.

Snowflake

Snowflake (SIGMOD 2016) was designed from scratch for the cloud and is BigQuery's closest architectural relative: a managed service with storage separated from compute, its own storage format, and no user-managed infrastructure. Where it differs most is in how compute is provisioned.

Three layers

- **Storage**: tables are split into immutable **micro-partitions**, columnar PAX-style files on object storage (S3, GCS, Azure Blob), each a few tens to hundreds of MB uncompressed. Per-column min/max and other metadata let queries prune micro-partitions, and Snowflake relies on that pruning instead of indexes. Optional clustering keys, maintained automatically, keep pruning effective (Module 5).
- **Virtual warehouses**: elastic clusters of compute nodes, sized like T-shirts and paid for while running. Each node caches table files on local SSD. Files are assigned to nodes by consistent hashing so the cache stays warm across queries, and idle nodes **steal** unscanned files from slow ones.
- **Cloud services**: the shared brain, running the optimizer, metadata, transactions, access control and query compilation for all warehouses.

Distinctive ideas

- **Isolation by warehouse**: different teams or workloads get separate warehouses over the *same* data, so an ETL job cannot slow a dashboard. Multi-cluster warehouses add clusters automatically for concurrency. Contrast BigQuery's single shared slot pool with fair scheduling and reservations (Module 7).
- **Immutability buys features**: because micro-partitions are never modified, snapshot isolation, **time travel** (querying a table as of an earlier time) and **zero-copy cloning** fall out of keeping old file versions and metadata pointers.
- **Semi-structured data**: the VARIANT type stores JSON-like data, automatically extracting frequently occurring paths into columns with their own statistics. The same idea as Capacitor's JSON shredding (Module 5).
- **Execution**: columnar, vectorized and push-based (Module 2), with a hand-off of runtime decisions such as join strategy to execution when estimates are uncertain. Failed queries are retried transparently.

Snowflake has since added Iceberg tables, moving partway down the positioning map toward open storage, as BigQuery has with BigLake.

DuckDB

DuckDB, from CWI Amsterdam (the institute behind MonetDB and VectorWise), is an analytical database that runs **inside the application process**, like SQLite does for transactions. There is no server: you link a library, or `import duckdb` in Python, and query

local files or data frames.

Why it matters beyond laptops

DuckDB packs most of this course's single-node techniques into one compact, readable codebase, which makes it the best reference implementation to study:

- **Vectorized, push-based pipelines** (switched from pull in 2021) with **morsel-driven parallelism** (Module 2).
- **Larger-than-memory operators**: hash joins, aggregations and sorts that spill gracefully, so a laptop can process data far larger than its RAM.
- **Radix sort on normalized keys** (Module 4).
- **A modern optimizer**: DPhyp join enumeration and *unnesting of arbitrary subqueries* (Neumann and Kemper, Module 1).
- **Its own compressed storage format**, using lightweight encodings including FSST for strings and ALP for floats (Module 5), with MVCC transactions.
- **Reading data where it lies**: Parquet, CSV, JSON, Iceberg and Delta, locally or over HTTP and S3, with projection and filter pushdown; zero-copy exchange with Arrow and pandas.

Limits

One node and essentially one writing process. Services such as MotherDuck add a cloud side, splitting a query between the local DuckDB and a remote one, but DuckDB itself is not a distributed engine. Its influence on the field comes from showing how much one machine can do: many "big data" queries fit comfortably on a single modern server.

ClickHouse and Redshift, briefly

ClickHouse

Built at Yandex for web analytics and open-sourced in 2016, ClickHouse is optimized for **fast ingestion and aggregation over very large single tables**, such as event logs and observability data.

- **MergeTree storage**: data arrives in immutable *parts*, each sorted by a table's sorting key, and background merges combine them, much like an LSM tree. A **sparse primary index** keeps one entry per *granule* of rows (8,192 by default), so a range filter on the

sorting key skips granules cheaply. Optional skipping indexes (min/max, sets, Bloom filters) cover other columns.

- **Aggressive specialization**: vectorized execution with code templated per data type, and many hash-table variants chosen by key type and size (Module 4's "pick the cheapest key representation", taken to the extreme). Optional LLVM JIT compiles hot expressions.
- **Distributed tables** shard data across servers; ClickHouse Cloud moves storage to object storage with shared metadata.
- Historically strongest on single-table scans and aggregations, weaker on large multi-way joins, a gap recent versions have been narrowing.

Amazon Redshift

Launched in 2012 from ParAccel's MPP code, Redshift was originally a classic shared-nothing warehouse (Module 6), and has since been rebuilt in stages:

- **Compiled execution**: each query segment is compiled to C++, with compiled code cached and reused across queries (Module 2's compiled model).
- **Managed storage** (RA3 nodes): data lives on S3 with local SSD caches, so storage is now disaggregated, and **Redshift Serverless** removes cluster management.
- **Automation**: concurrency scaling adds clusters under load, and machine-learning-driven tuning chooses sort and distribution keys automatically.

Its trajectory, from shared-nothing toward disaggregated and serverless, is the same one Dremel followed a decade earlier (Module 7).

Recurring themes

Seen side by side, the engines are converging far more than their marketing suggests. Six trends run through all of them.

1. **Vectorized, columnar execution has won.** Velox, Photon, Snowflake, DuckDB, ClickHouse and BigQuery's runtime are all vectorized; code generation survives mainly in Spark's JVM path, Redshift, and as JIT for hot expressions (Module 2).
2. **Everything is disaggregating.** Redshift RA3, ClickHouse Cloud and Snowflake followed the path Dremel took first: storage on a shared service, stateless compute, local caches to recover locality (Modules 6 and 7).

3. **Open table formats are the meeting point.** Iceberg in particular is now readable and often writable by BigQuery (via BigLake), Snowflake, Trino, Spark and DuckDB. Engines increasingly compete on execution over shared data rather than on owning the data (Module 5).
4. **Composable engines.** Velox, Arrow, Substrait, Axiom, and on the Rust side Apache DataFusion (with Comet as its Spark plugin, the counterpart to Gluten), let new systems assemble an engine instead of writing one. The industry is splitting front ends, execution libraries and storage formats into reusable layers.
5. **Adaptivity and history replace estimation.** Runtime re-planning, dynamic filters and history-based optimization appear in BigQuery, Spark, Presto and Snowflake alike (Module 6).
6. **Single-node is back, and so is new hardware.** DuckDB and Polars show how far one machine goes; GPU engines (NVIDIA's RAPIDS accelerator for Spark, Voltron Data's Theseus, and experimental GPU backends for Velox) push in the opposite direction.

The remaining real differences sit where Module 7 put them: who owns the storage, how shuffle and scheduling work, and how tightly the layers are integrated. BigQuery's bet is on full integration; the Velox ecosystem's bet is on shared, composable parts.

Self-test and reading list

1. Place each engine on the two axes of the positioning map, and explain one consequence of where it sits.
2. Why did Presto split into Presto and Trino, and how have their technical directions diverged?
3. What does Catalyst contribute to Spark, and what does whole-stage codegen do?
4. Give Photon's main reasons for choosing vectorization over code generation.
5. Photon vs. Gluten: what problem do both solve, and how do their approaches differ?
6. What are the two main sources of overhead when Gluten falls back to vanilla Spark?
7. How does Snowflake isolate workloads, and how does that differ from BigQuery's approach?
8. Why do immutable micro-partitions make time travel and zero-copy cloning cheap?
9. Which techniques from Modules 1 to 5 does DuckDB implement, and why is it a good codebase to study?
10. How does ClickHouse's sparse primary index skip data, and what kind of query is it best at?

11. Name three trends shared across these engines.

Reading list:

1. Behm et al., "Photon: A Fast Query Engine for Lakehouse Systems", SIGMOD 2022.
2. Dageville et al., "The Snowflake Elastic Data Warehouse", SIGMOD 2016.
3. Sethi et al., "Presto: SQL on Everything", ICDE 2019.
4. Armbrust et al., "Spark SQL: Relational Data Processing in Spark", SIGMOD 2015.
5. Raasveldt and Mühleisen, "DuckDB: an Embeddable Analytical Database", SIGMOD 2019 (demo); plus the DuckDB blog for its internals.
6. Schulze et al., "ClickHouse: Lightning Fast Analytics for Everyone", VLDB 2024.
7. Armenatzoglou et al., "Amazon Redshift Re-invented", SIGMOD 2022.
8. Apache Gluten and Apache DataFusion documentation.

Next: Module 9, query optimization in depth.

Module 9 — Query Optimization

Oct 14, 2026

Why this module

Module 1 gave the optimizer one section. This module goes deeper into the three questions every optimizer must answer: which equivalent plans to consider (enumeration), how to estimate what each one costs (cardinality and cost models), and how to rewrite queries so good plans exist to be found (rewrites and decorrelation). It ends with what changes in distributed engines, and with the shift from estimating toward learning from execution.

The shape of a typical optimizer

1. **Normalize and rewrite** the bound tree with rules that are almost always beneficial: constant folding, predicate pushdown, subquery decorrelation, outer-join simplification. No costs involved.
2. **Explore** logically equivalent alternatives: join orders, aggregation placement.
3. **Implement** each logical operator with physical algorithms, inserting enforcers (sorts, exchanges) where required properties aren't met.
4. **Cost** each candidate using estimated cardinalities, and keep the cheapest.

Steps 2 to 4 are where the two classic frameworks differ: bottom-up dynamic programming builds plans from small subsets of tables upward, while Cascades searches top-down over a shared memo, interleaving exploration, implementation and costing. Both depend entirely on cardinality estimates, which are the weakest link.

Join enumeration: bottom-up dynamic programming

The number of join orders explodes: 10 tables have 3.6 million left-deep orders alone, and far more bushy trees. Dynamic programming makes the search tractable by noticing that the best plan for a set of tables can be built from the best plans for its subsets.

System R (Selinger et al., 1979)

1. Find the best access path for each single table.

2. For every set of 2 tables, combine the best plans of its parts; then sets of 3; and so on up to all n tables.
3. Keep only the best plan for each set. Any plan containing a worse sub-plan for the same set can be improved by swapping in the better one (the principle of optimality).

System R considered only **left-deep** trees and avoided cross products. Its subtle and lasting idea is **interesting orders**: keep the best plan per set *for each useful output order*, not just the cheapest overall, because a slightly more expensive plan that produces rows sorted on a join key may save a sort later. Distributed optimizers generalize this to **interesting properties**: also keep the best plan per useful *partitioning*, since an output already hash-partitioned on the next join key saves a shuffle.

Modern DP: enumerate only what is connected

- **DPsize and DPsub** enumerate pairs of subsets and then discard pairs that aren't connected by a join predicate, wasting most of their time on dead ends.
- **DPccp** (Moerkotte and Neumann, VLDB 2006) enumerates exactly the pairs of a *connected subgraph* and a *connected complement* of the query graph, so no work is wasted. It is optimal for inner-join graphs.
- **DPhyp** (SIGMOD 2008) extends this to **hypergraphs**: predicates over three or more tables, and outer, semi and anti joins whose reordering constraints are encoded as hyperedges. DuckDB and HyPer use it.

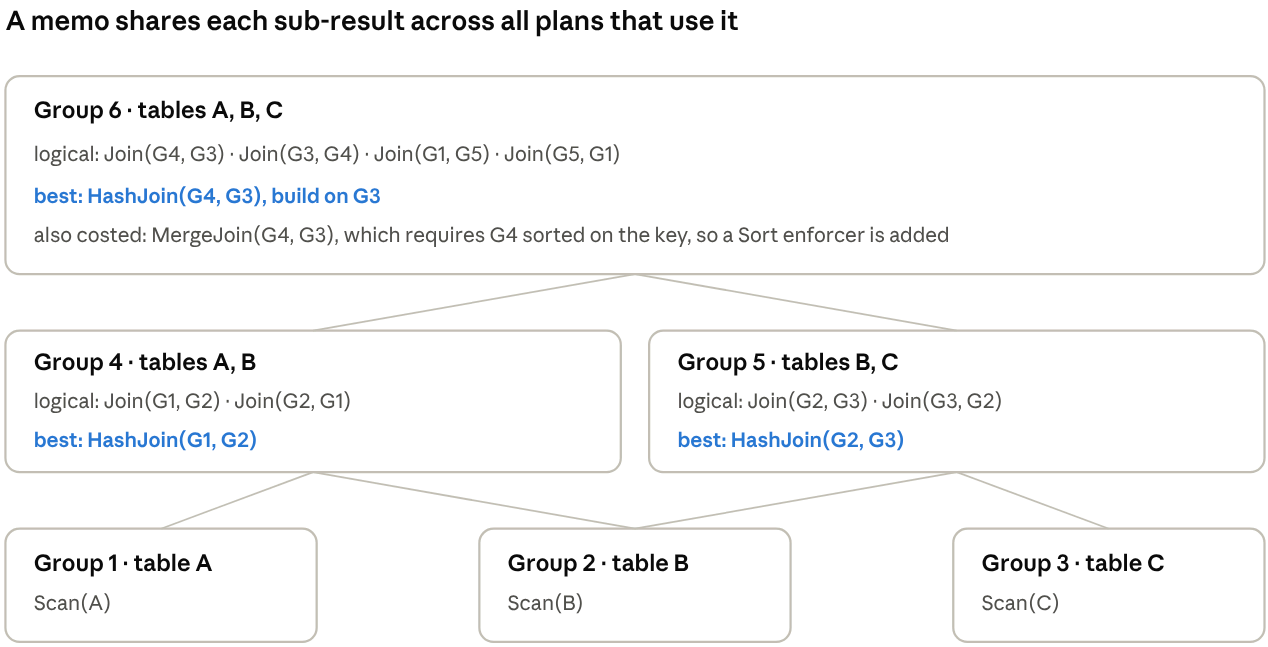
When the query is too big

Cost depends on the query graph's shape: chains and stars enumerate quickly, while cliques (every table joined to every other) explode. Beyond roughly 10 to 15 tables in a dense graph, engines fall back to heuristics:

- **Greedy operator ordering**: repeatedly join the pair with the smallest estimated result.
- **IKKBZ**: optimal left-deep orders in polynomial time for a class of cost functions.
- **Genetic search**: PostgreSQL's GEQO, used from 12 tables by default.
- **Adaptive optimization** (Neumann and Radke, SIGMOD 2018): exact DP for small queries, a linearized DP for medium ones, greedy for huge ones (hundreds or thousands of tables, which real generated SQL does produce).

Cascades: memo, rules and properties

Cascades (Graefe, 1995), the successor of the Volcano optimizer generator, is the framework behind SQL Server, Greenplum's Orca, Apache Calcite and CockroachDB. Instead of a fixed enumeration algorithm, it is a search engine driven by **rules**, with every intermediate result shared in a **memo**.



Cascades memo for A ⋈ B ⋈ C (chain query)

The memo

- A **group** is a set of logically equivalent expressions that all produce the same result, for example every way of computing {A, B}.
- A **group expression** is one operator whose inputs are *groups*, not concrete subplans: `Join(G1, G2)`. That is what keeps the memo compact: Group 4's alternatives are explored once, then shared by every plan of Group 6 that uses {A, B}.
- Each group records its **best physical plan per required property set**, the memo version of interesting orders.

Rules

- **Transformation rules** produce logically equivalent expressions: join commutativity (`Join(G1,G2)` → `Join(G2,G1)`), associativity, pushing an aggregate below a join.

- **Implementation rules** turn logical operators into physical ones: Join → HashJoin, MergeJoin or NestedLoopJoin.
- New optimizations are added as new rules, without changing the search engine. This extensibility is why Cascades dominates industrial optimizers.

Top-down search with required properties

Optimization starts at the root group with the properties the query needs (for example "sorted on x" for an ORDER BY) and recursively asks child groups for their best plans under the properties each physical operator requires. When a child cannot deliver a property, an **enforcer** is inserted: a Sort for order, an Exchange for partitioning. Optimizing top-down lets the search prune: once a complete plan with cost C is known, any partial plan already costing more than C is abandoned (branch and bound).

DP vs. Cascades, briefly

Bottom-up DP is simpler and provably efficient for join ordering. Cascades is more extensible and handles physical properties, enforcers and non-join rewrites in one framework, at the cost of more engineering to keep the search from exploding. Many systems combine them: a DP-style join enumerator inside a broader rule-based framework.

Cardinality estimation and why it fails

Every cost depends on how many rows each operator produces. Estimates come from a few statistics combined through simple formulas, and each formula rests on an assumption that real data routinely breaks.

Statistics engines keep

- Row count per table; per column the number of distinct values (NDV, usually from a HyperLogLog sketch), null fraction, min and max.
- **Histograms** (usually equi-depth: buckets with equal row counts) for range predicates.
- **Most common values** with their frequencies, for skewed columns.
- Sometimes multi-column statistics, on request.

The formulas, and their hidden assumptions

Predicate	Estimate	Assumption
`col = c`	1 / NDV, or c's frequency if it is a most common value	Uniformity: values otherwise equally frequent
`col BETWEEN a AND b`	Fraction of histogram covered	Uniformity within buckets
`p AND q`	sel(p) × sel(q)	**Independence** of columns
`p OR q`	sel(p) + sel(q) − sel(p) × sel(q)	Independence
`R.k = S.k` (join)	|R| × |S| / max(NDV(R.k), NDV(S.k))	**Containment**: the smaller key set is contained in the larger
`LIKE '%x%'`, UDFs	A fixed default guess	None; it's a guess

How they fail

- **Correlation**: `city = 'Zurich' AND country = 'CH'` is estimated as sel(city) × sel(country), far too low, because the city already implies the country.
- **Skew**: one value holding 30% of rows breaks 1/NDV unless it is in the most-common list.
- **Joins compound errors**: each join multiplies the previous error. Leis et al. ("How Good Are Query Optimizers, Really?", VLDB 2015) measured this on real IMDB data: estimates were mostly **underestimates**, off by orders of magnitude after a few joins.
- **Underestimates are the dangerous direction**: they make nested-loop joins, wrong build sides and broadcast joins look cheap, and those plans degrade catastrophically when the real input is large.

The same study found that with *true* cardinalities, even simple cost models chose good plans. Cardinality, not cost modelling, is the main source of bad plans.

What engines do about it

- **Better statistics**: multi-column statistics (PostgreSQL's `CREATE STATISTICS`), and sampling at optimization time.

- **Feedback**: compare estimates with actuals after execution and correct future estimates, from IBM DB2's LEO (2001) to today's history-based optimizers (section on learned optimization).
- **Robust choices**: prefer plans that degrade gracefully when the estimate is wrong, such as a hash join over a nested-loop join when unsure.
- **Adaptivity**: defer the decision until the real size is known (Modules 6 and 7).

Cost models

A cost model turns cardinalities into a number per plan. It needs to rank plans correctly, not predict runtime, which is why fairly simple models work well when the cardinalities are right.

What goes into a cost

- **CPU**: per-row costs per operator (hashing, comparing, evaluating expressions), scaled by input and output cardinalities.
- **I/O**: bytes or pages read, with sequential reads cheaper than random ones.
- **Network**: bytes shuffled or broadcast. In distributed engines this term usually dominates.
- **Memory**: a step change when a hash table or sort no longer fits and must spill.

Constants weighting these terms are calibrated per system and hardware. A useful baseline is **C_out**, the sum of the cardinalities of all intermediate results. It ignores operator differences entirely, yet ranks join orders reasonably well, because large intermediate results are what make plans slow.

Which objective?

A single-node engine minimizes elapsed time. A distributed engine must choose between:

- **Total work** (sum over all tasks): what a shared cluster or a per-resource bill cares about.
- **Latency** (the critical path, roughly the maximum over parallel tasks): what an interactive user cares about.

The two disagree: a broadcast join may finish sooner while using more total resources than a shuffle join. Serverless systems that bill by bytes scanned or slot time make total work directly visible to users, and some let them trade it against latency through

reservations and priorities.

The takeaway: spend effort on cardinalities and on avoiding catastrophic plans; refine cost-model constants last.

Subqueries and decorrelation

A **correlated subquery** refers to a column of the outer query, so read literally it must run once per outer row: a nested loop that is quadratic in the worst case. Decorrelation rewrites it into joins and aggregations that the rest of the optimizer can handle. Getting the rewrite *correct* is harder than it looks.

The easy cases

- `EXISTS (subquery)` becomes a **semi join**; `NOT EXISTS` an **anti join**.
- `x IN (subquery)` becomes a semi join; `x NOT IN (subquery)` needs a **null-aware** anti join (Module 4).

Scalar aggregates: the classic rewrite

```
-- customers whose credit exceeds their total orders
SELECT c.id FROM customers c
WHERE c.credit > (SELECT SUM(o.amount) FROM orders o WHERE o.cust_id = c.id);
```

Kim's 1982 rewrite: aggregate the inner table once per correlation value, then join.

```
SELECT c.id
FROM customers c
JOIN (SELECT cust_id, SUM(amount) AS s FROM orders GROUP BY cust_id) t
  ON t.cust_id = c.id
WHERE c.credit > t.s;
```

The COUNT bug

Now use COUNT instead of SUM:

```
SELECT c.id FROM customers c
WHERE (SELECT COUNT(*) FROM orders o WHERE o.cust_id = c.id) = 0;
```

Customers with no orders should qualify: their subquery returns 0. But after the same rewrite, those customers have *no group at all* in `t`, so the inner join drops them and the query silently returns fewer rows. Ganski and Wong pointed this out in 1987. The fix is to use a **left outer join** and supply the empty-input value of the aggregate: `COALESCE(t.cnt, 0)` for COUNT, but NULL for SUM. Every aggregate's behavior on empty input has to be emulated.

Decorrelating anything

- **Apply / dependent join**: model the subquery as an operator evaluated per outer row, then push it down through the inner plan with algebraic rules until it becomes a plain join (Galindo-Legaria and Joshi, SIGMOD 2001; used in SQL Server).
- **Unnesting arbitrary queries** (Neumann and Kemper, BTW 2015): compute the *domain*, the distinct values of the outer columns the subquery references; evaluate the subquery once for the whole domain, joined with it and grouped by it; then join the result back. Cost scales with the number of distinct correlation values, not outer rows, and it works for any nesting depth. DuckDB implements it.

Engines without general decorrelation either fall back to slow per-row evaluation or reject some correlated forms outright.

Rewrites that matter most in practice

Machine-generated SQL from BI tools, views stacked on views, and ORMs produce queries no person would write. These rewrites remove most of the waste before join enumeration even starts.

Rewrite	What it does	Watch out for
Predicate pushdown	Move filters below joins, aggregations (when on grouping keys) and into scans	Not to the NULL-padded side of an outer join; not past non-deterministic expressions
Predicate inference	From `a = b AND b = 5`, derive `a = 5` (equivalence classes)	Type coercion: equal under one type, not another
Outer-join simplification	A filter that rejects NULLs on the padded side turns the outer join into an inner join	Must prove the predicate is truly null-rejecting

Rewrite	What it does	Watch out for
Join elimination	Drop a join whose table contributes no columns, when a foreign key guarantees exactly one match	Requires trusted (enforced or declared) constraints
Column and field pruning	Read only used columns and nested fields	Columns needed only by filters still count
Aggregation pushdown	Pre-aggregate below a join (eager aggregation, Module 1)	Valid only when grouping covers the join key and the aggregate decomposes
Limit and Top-N pushdown	Push LIMIT through projections and unions, and onto the preserved side of outer joins	Never through filters or inner joins
CTE handling	Inline a CTE used once; materialize one that is expensive and reused	Materializing blocks pushing predicates into the CTE
Partition pruning	Skip partitions statically from predicates, or dynamically from joins	Predicates must be expressible on the partition column
Materialized-view rewrite	Answer from a precomputed view that subsumes the query	View freshness, and proving subsumption

Join elimination deserves emphasis: BI tools often join a fact table to every dimension in a model "just in case". With declared foreign keys, an optimizer can delete those unused joins entirely, which can matter more than any join-ordering cleverness.

Optimizing distributed plans

A distributed optimizer does everything a single-node one does, plus decides where data moves. Because moving data usually dominates cost, the decisive choices are about partitioning, not join algorithms.

Distribution as a physical property

Cascades-style optimizers (Greenplum's Orca is the textbook example) treat data distribution like sort order: a property operators require and deliver.

Property	Meaning	Typical requirer
Singleton	All rows on one node	Final ORDER BY, global aggregate's last step
Hashed on (cols)	Rows with equal values on the same node	Hash join on those keys, GROUP BY on them
Replicated	Every node has all rows	The small side of a broadcast join
Any / random	No guarantee	Scans, filters, projections

When a child doesn't deliver what its parent requires, the optimizer inserts the cheapest **exchange** that does: a hash repartition, a broadcast or a gather. Exchanges are the enforcers of distributed plans, exactly as sorts are for order.

What changes in the search

- **Join order interacts with partitioning.** Joining three tables on the same key in any order needs only the initial shuffles, while a different key per join reshuffles every time. Optimizers keep "interesting partitionings" alive alongside costs (the generalized System R idea).
- **Bushy trees become attractive**: independent subtrees run in parallel stages.
- **Join strategy per join**: broadcast vs. shuffle by comparing |small| × workers with |left| + |right| (Module 1), plus memory limits per worker.
- **Aggregation strategy**: single-phase after a shuffle, or two-phase with partial aggregation first.
- **Degree of parallelism** per stage: too few tasks underuse the cluster; too many add per-task overhead and tiny partitions.

Errors cost more

A cardinality error that picks the wrong build side on one machine costs seconds; one that broadcasts a table believed to be 10 MB but actually 100 GB to a thousand workers can take down the query. Distributed optimizers therefore lean on safe defaults (shuffle when unsure), runtime adaptivity (Spark AQE, BigQuery's dynamic query execution) and history. Skew is usually not modelled at all and is handled at runtime (Module 6).

Learned, history-based and robust optimization

If estimates are the weak link, an obvious idea is to learn from what queries actually did. Research has pursued this for a decade; production systems have adopted its most pragmatic forms.

Learned components (mostly research)

- **Learned cardinality estimation**: neural models over query features (MSCN, CIDR 2019) or over the data distribution itself (Naru, NeuroCard). Often far more accurate on correlated data, but they must be retrained as data changes, add inference latency, and are hard to debug.
- **Learned optimizers**: Neo (VLDB 2019) learned to build plans end to end.
- **Steering an existing optimizer**: Bao (SIGMOD 2021) leaves the optimizer in place and learns, per query, which *hint set* to apply (for example "disable nested-loop joins"), using feedback from actual runtimes. Microsoft applied the same idea to its SCOPE big-data system. This is the learned approach closest to production, because it can only choose among plans the existing optimizer would produce.

History-based optimization (in production)

A large share of analytical workloads are *recurring*: dashboards, scheduled reports and pipelines run the same query shapes over fresh data. For them, the best estimate is what happened last time:

1. Canonicalize each plan fragment (strip literals and naming details) into a stable signature.
2. After execution, store the actual row counts, sizes and resources per signature.
3. On the next run, use the stored values in place of estimates, for join order, join strategy, partition counts and memory sizing.

Presto's history-based optimizer (from Meta) and BigQuery's history-based optimizations both work this way, as did IBM DB2's earlier LEO learning optimizer.

Robustness and stability

Users notice *plan regressions* far more than missed optimizations: a query that ran in 10 seconds yesterday and 10 minutes today. Hence:

- **Robust plan choice**: prefer plans whose cost degrades gracefully if the estimate is wrong, even at a slightly higher expected cost.

- **Plan stability tools**: pinning known-good plans, detecting regressions by comparing runs, and rolling back.

Where it is heading

The optimizer is becoming a loop rather than a one-time decision: **plan, execute, observe, correct**, within a query (adaptive execution) and across queries (history). BigQuery's combination of dynamic query execution and history-based optimization is a production example of that loop.

Self-test, reading list and wrap-up

1. Why does System R keep more than one plan per set of tables? What is the distributed version of that idea?
2. What does DPccp avoid that DPsize wastes time on, and what does DPhyp add?
3. In a Cascades memo, why does a group expression reference groups rather than subplans?
4. What is an enforcer? Give one for order and one for distribution.
5. Estimate `city = 'Zurich' AND country = 'CH'` under independence, and explain why it is wrong.
6. Why are cardinality *under*estimates more dangerous than overestimates?
7. What is C_out, and why does such a crude model work?
8. Rewrite a correlated COUNT subquery into a join without introducing the COUNT bug.
9. When may the optimizer eliminate a join entirely?
10. Why does join order interact with data partitioning in a distributed plan?
11. How does Bao differ from a learned cardinality estimator, and why is it easier to deploy?
12. How does history-based optimization identify "the same" query across runs?

Reading list:

1. Selinger et al., "Access Path Selection in a Relational Database Management System", SIGMOD 1979.
2. Graefe, "The Cascades Framework for Query Optimization", IEEE Data Engineering Bulletin, 1995.
3. Leis et al., "How Good Are Query Optimizers, Really?", VLDB 2015.

4. Moerkotte and Neumann, "Dynamic Programming Strikes Back", SIGMOD 2008 (DPhyp).
5. Neumann and Kemper, "Unnesting Arbitrary Queries", BTW 2015.
6. Soliman et al., "Orca: A Modular Query Optimizer Architecture for Big Data", SIGMOD 2014.
7. Marcus et al., "Bao: Making Learned Query Optimization Practical", SIGMOD 2021.
8. Neumann and Radke, "Adaptive Optimization of Very Large Join Queries", SIGMOD 2018.

Course wrap-up

Across Modules 1 to 9, the whole system fits in one sentence: a query is **parsed and bound** (1), **optimized** into a plan of stages cut at exchanges (1, 9), each stage runs as **vectorized pipelines** on workers (2, 3) built from a few **stateful operators** (4) over **columnar, encoded storage** that is pruned at every level (5), stages exchange data through a **shuffle** and are scheduled, retried and **re-planned at runtime** (6), and BigQuery is one coherent, fully integrated instance of all of it (7) among several other designs (8).

If you want to go further, the two best codebases to read are DuckDB (the whole single-node stack in one readable repository) and Velox (a production-grade execution engine), and the two best papers to reread are the Dremel 2020 retrospective and "How Good Are Query Optimizers, Really?".

Module 10 — Query Processing for ML Data

Oct 14, 2026

Why this module

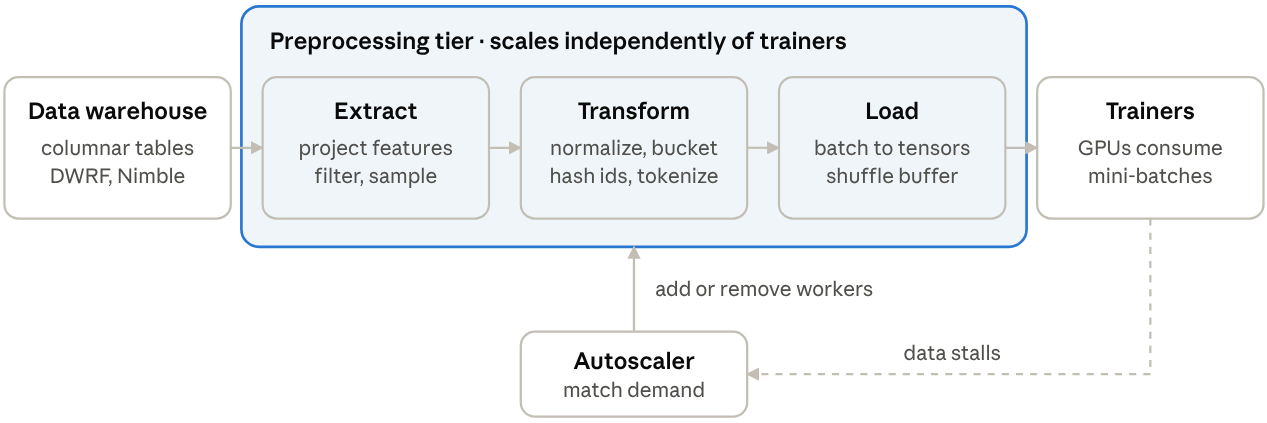
Feeding a training job is query processing under different constraints: scan huge columnar tables, select a few hundred of thousands of features, filter and sample rows, apply transformations, and deliver the result as tensors fast enough that accelerators never wait. At industrial scale this is not a side concern. Meta's ISCA 2022 paper on its data storage and ingestion (DSI) pipeline describes exabytes of training data served at tens of terabytes per second, and for some production recommendation models, storage and preprocessing consumed more power than training itself.

This module looks at that problem through the lens of the course: which query-processing techniques transfer directly, which constraints are genuinely different, and how the systems that solve it are built.

Anatomy of a training data pipeline

A training data pipeline has two halves: **offline data generation**, which builds training tables in a warehouse with ordinary batch query processing (joins of logged features with labels), and **online ingestion**, which reads those tables during training and turns them into batches.

A training data pipeline is a scan, a transform and a load



training data pipeline · extract, transform, load, with autoscaling

- **Extract** is a scan in the Module 5 sense: read only the partitions in the training window, project the features this model uses (often a small fraction of a very wide table), filter rows, and sample, e.g. down-sample negative examples.
- **Transform** applies per-feature operations: normalizing dense features, bucketizing, hashing or mapping sparse ids into embedding-table ranges, tokenizing text, decoding and augmenting images. These are expression evaluation (Module 3) under another name, often with user-defined logic.
- **Load** assembles mini-batches as tensors in the layout the model expects, shuffles within a buffer, and delivers them over the network to trainer hosts.

Two requirements shape everything that follows: the pipeline must sustain the trainers' consumption rate at all times, because an accelerator waiting for data is the most expensive idle resource in the data center, and it must do so at the lowest possible cost in CPU, memory, network and power.

How ML data loading differs from SQL analytics

The operators look familiar, but the objective is different: analytics minimizes the time to answer one question, while ingestion must sustain a rate for hours or weeks at the lowest cost.

Aspect	SQL analytics	ML data ingestion
Output	A result set for a person or tool	A continuous stream of tensors for accelerators
Objective	Minimize latency per query	Never stall the trainers; minimize resources at that rate
Duration	Seconds to minutes	Hours to weeks, often several passes (epochs) over the data
Row order	Irrelevant unless ORDER BY	Randomized order matters for convergence; determinism matters for reproducibility
Failure handling	Retry the query or task	Checkpoint and resume mid-epoch at an exact position in the stream
Transformations	SQL expressions	Feature engineering, often user code: hashing, tokenizing, image decoding
Schema	Tens of columns	Thousands of features, many sparse: lists of ids, maps
Sharing	Queries mostly independent	Many concurrent jobs read overlapping features and samples

Two of these change system design the most. **Steady-state throughput** means capacity planning is a matching problem: preprocessing capacity must track trainer demand, neither starving GPUs nor wasting CPUs. **Heavy overlap between jobs**, which Meta's DSI paper reports (popular features and samples reused across many training jobs), creates the shared-scan and caching opportunities that a pure per-query engine never sees.

Architectures

The history of ML input pipelines repeats the history of query engines in Modules 6 and 7: start co-located with compute, then disaggregate when the coupling becomes the bottleneck.

1. In the trainer process

PyTorch's DataLoader worker processes and tf.data pipelines run preprocessing on the trainer host's CPUs. Simple, with no extra service and no network hop. But the ratio of CPU to accelerators is fixed by the hardware, so a model with heavy preprocessing stalls its GPUs while a light one wastes CPUs, and Python-level per-sample code adds interpretation overhead (Module 2's Volcano problem, in Python).

2. Disaggregated preprocessing services

- **Meta's DPP** (Data PreProcessing Service, described in the ISCA 2022 DSI paper): a control plane divides a training job's data into splits, a fleet of preprocessing workers reads from the warehouse and transforms them, and clients on the trainers receive ready tensors. Workers scale independently of trainers to eliminate data stalls: storage and compute disaggregation again, applied to preprocessing.
- **Google's tf.data service** (SoCC 2023) makes the same argument for TensorFlow pipelines: run them on a separate, horizontally scalable pool of workers.
- **Cachew** (USENIX ATC 2022) adds two policies on top: autoscaling the worker count to the trainers' consumption rate, and deciding automatically whether to cache preprocessed data for later epochs or recompute it.

3. General dataflow engines in the loop

Ray Data streams data through a pipeline of operators across a heterogeneous cluster of CPUs and GPUs, with backpressure between stages, so CPU preprocessing and GPU work (decoding, embedding, batch inference) can sit in one pipeline. Spark and Beam/Dataflow typically handle the offline side: generating and materializing training tables.

4. Materialize the preprocessed data

If many epochs or jobs apply the same transforms, write the transformed tensors once (TFRecord or WebDataset shards, for example) and stream those. This saves CPU at read time at the cost of storage and flexibility: any feature change means regenerating everything.

Choosing

The deciding quantity is preprocessing cost per sample relative to the accelerator's consumption rate. When it is small, in-trainer loading suffices; when it is large or varies by model, a disaggregated service with autoscaling wins; when the same output is reused heavily, materialize or cache it.

Query-engine techniques that transfer

Most of what makes an analytical engine efficient applies directly to ingestion; the art is in where each technique lands in the pipeline.

Read less (Module 5)

- **Formats built for wide feature tables**: DWRF flat maps store each feature key as its own stream, and Nimble keeps metadata cheap at tens of thousands of streams, so a model using 300 of 20,000 features reads 300 streams.
- **Push filters and sampling into the reader**: drop rows, including down-sampled negatives, before decoding the remaining columns. Sampling whole row groups is cheapest; row-level sampling preserves the distribution better.
- **Late materialization**: decode heavy sparse features only for rows that survive filtering.

Compute efficiently (Modules 2 and 3)

- **Vectorized transforms** instead of per-sample Python: hashing, bucketizing and normalizing run as typed loops over batches. This is why an execution library like Velox fits inside a preprocessing service.
- **Encoding-aware transforms**: hashing a dictionary-encoded categorical feature once per dictionary entry is Velox's dictionary peeling applied to feature engineering.
- **Bounded memory with backpressure**: preprocessing workers hold a bounded number of batches in flight, like Drivers blocked on a full output buffer.

Hand off without copying

The last step converts columnar batches into tensors. Layouts line up remarkably well: a dense column is already a contiguous array, and a variable-length sparse feature, stored as offsets plus values (Module 3's ArrayVector), maps directly onto the **jagged tensor** representations recommendation libraries use, such as TorchRec's KeyedJaggedTensor. Done right, the conversion is a few buffer hand-offs rather than a re-encoding.

Share work across jobs

- **Caching** preprocessed batches for later epochs, decided automatically from cost (Cachew).
- **Shared scans**: when many concurrent jobs read the same partitions and features, read once and fan out.
- **Deduplication**: Meta's RecD (MLSys 2023) exploits the fact that samples from the same user session repeat many feature values, deduplicating them through storage, preprocessing and training.

Adapt at runtime (Module 6)

Autoscaling preprocessing workers from observed data stalls is adaptive execution with a different objective: match a consumption rate instead of minimizing latency.

Shuffling, sampling and reproducibility

Stochastic gradient descent assumes mini-batches are drawn roughly at random, but training tables are stored in time or user order. Producing randomness cheaply, deterministically and resumably is a problem query engines don't have.

Approximating a global shuffle

A true global shuffle of the dataset every epoch is a full distributed shuffle (Module 6), far too expensive at these sizes. Pipelines combine cheaper approximations instead:

1. **Shuffle the order of shards** (files, row groups) each epoch: nearly free, but coarse.
2. **Interleave**: read many shards concurrently and mix their output.
3. **Shuffle buffer**: keep N samples in memory and emit a random one each time a new one arrives (tf.data's `shuffle`). Larger buffers give better randomness at the cost of memory.

Together these are usually good enough. Note that order is also a modeling choice: recommendation models are sometimes trained deliberately in time order to mimic how they will be deployed.

Sampling

Down-sampling the abundant class (non-clicks, for example) cuts data volume dramatically; the loss is then re-weighted to correct for it. Pushing the sampling into the reader means discarded rows are never decoded.

Reproducibility and resumption

A training job must be able to restart from a checkpoint and continue *exactly* where it left off, without dropping or repeating samples:

- **Deterministic assignment**: the mapping from epoch and seed to the sequence of splits must not depend on how many workers happen to be running, or autoscaling would change the data order.
- **Position in checkpoints**: store the split and offset each stream had reached, along with the shuffle seeds and buffer state, or accept a small, controlled amount of replay.
- **Exactly-once delivery** despite worker failures: the same attempt-tagging and commit discipline as a shuffle service: every chunk tagged with its task attempt, and only one committed attempt made visible (Module 6).

Generating training data: point-in-time joins and feature stores

Before ingestion comes generation: building the training table by joining labels with features. The core difficulty is time. A training example must see each feature exactly as it was when the labeled event happened, never later.

The leakage trap

Joining click events to a `user_features` table on `user_id` with today's values leaks the future: a feature like "clicks in the last 7 days" may already include the very click being predicted. The model looks great offline and disappoints in production, because at serving time that information doesn't exist yet.

Point-in-time (as-of) joins

For each label row (entity, event time), take the **latest feature value with a timestamp at or before the event time**, optionally no older than a time-to-live:

```
-- DuckDB / Snowflake style ASOF JOIN
SELECT l.*, f.clicks_7d
FROM labels l
ASOF JOIN user_features f
  ON l.user_id = f.user_id AND l.event_ts >= f.feature_ts;
```

- **Single node**: sort both inputs by (key, time) and merge-scan, a variant of sort-merge join (Module 4). DuckDB, Snowflake and ClickHouse support `ASOF JOIN` natively; elsewhere it is written with window functions.
- **Distributed**: hash-shuffle both sides by entity key, then sort-merge within each partition. Hot entities, such as bots or heavy users, make this a skew problem (Module 6).
- **Windowed features** ("clicks in the last 7 days" at each event time) are sliding-window aggregations, computed incrementally rather than per row (Module 4).

Feature stores and logging

- **Feature stores** (Feast, Tecton, Vertex AI Feature Store and others) keep one definition per feature, with an *offline store* in the warehouse for point-in-time training joins and an *online store* (a key-value database) for low-latency serving, so training and serving compute features the same way.
- **Logging at serving time** is the alternative many large companies use: record the exact feature values the model saw at inference, then join them with labels once they arrive. Point-in-time correctness comes for free, at the cost of only training on features that were already being served.

LLM-era data processing

Large language and multimodal models moved the heavy lifting from online preprocessing to offline corpus construction, which is batch query processing at web scale.

Building a pretraining corpus

1. **Extract** text from raw sources: HTML from web crawls such as Common Crawl, code repositories, books, papers.
2. **Filter**: language identification, heuristic quality rules, model-based quality scores, toxicity and PII removal, and decontamination against evaluation benchmarks.

3. **Deduplicate**: exact duplicates via content hashes, which is a `GROUP BY hash`; near-duplicates via **MinHash and locality-sensitive hashing (LSH)**.
4. **Tokenize**, then **mix** sources in chosen proportions and **pack** documents into fixed-length training sequences.

Near-duplicate detection is the most query-like step: LSH "banding" hashes each document into buckets, documents sharing a bucket become candidate pairs (a self-join on bucket ids), and the duplicate clusters are the connected components of the resulting graph. At billions of documents this is a distributed join and graph problem, run on Spark, Ray or Beam, with the same skew issues as any join: a few huge buckets of boilerplate pages.

Training-time ingestion

With tokens precomputed, online work becomes streaming pre-tokenized shards (WebDataset, MosaicML's StreamingDataset and similar) to thousands of accelerators. The hard parts move to the requirements of the Shuffling section: deterministic mixing across sources, and exact resumption after a failure at very large scale.

Multimodal

Images, audio and video bring back expensive online transforms: decoding, resizing and augmentation can cost more than the model's forward pass on CPU. Hence GPU-side decoding libraries such as NVIDIA DALI, and pipelines like Ray Data that place CPU and GPU stages in one stream, which leads into the next module.

Self-test and reading list

1. Which parts of a training data pipeline are scans, which are expression evaluation, and which are something query engines don't do?
2. Why does a fixed CPU-to-accelerator ratio on trainer hosts motivate disaggregated preprocessing?
3. What two decisions does Cachew automate, and what are they analogous to in Modules 6 and 7?
4. How do DWRF flat maps and Nimble reduce the cost of reading 300 of 20,000 features?
5. Why does a variable-length sparse feature convert cheaply into a jagged tensor?

6. Combine three cheap techniques to approximate a global shuffle. What does each one contribute?
7. Why must split assignment be independent of the number of preprocessing workers?
8. Show how a naive feature join leaks future information, and write the as-of join that prevents it.
9. Why is near-duplicate detection over a web corpus a join problem, and where does skew appear?

Reading list:

1. Zhao et al., "Understanding Data Storage and Ingestion for Large-Scale Deep Recommendation Model Training", ISCA 2022.
2. Murray et al., "tf.data: A Machine Learning Data Processing Framework", VLDB 2021.
3. Audibert et al., "tf.data service: A Case for Disaggregating ML Input Data Processing", SoCC 2023.
4. Graur et al., "Cachew: Machine Learning Input Data Processing as a Service", USENIX ATC 2022.
5. Zhao et al., "RecD: Deduplication for End-to-End Deep Learning Recommendation Model Training Infrastructure", MLSys 2023.
6. Ray Data documentation (streaming execution).
7. Broder, "On the Resemblance and Containment of Documents", 1997 (MinHash); and the deduplication sections of open LLM dataset papers such as RefinedWeb or FineWeb.

Next: Module 11, GPU and hardware-accelerated query processing.

Module 11 — GPU and Hardware-Accelerated Query Processing

Oct 14, 2026

Why this module

GPU databases have been "about to take over" for fifteen years, and mostly didn't. What changed recently is not the idea but the conditions: GPU memory grew to well over 100 GB per device with several TB/s of bandwidth, CPU-GPU links got an order of magnitude faster, libraries like NVIDIA's cuDF made GPU relational operators a commodity, and in ML-heavy data centers the GPUs are already there (Module 10).

This module explains when hardware acceleration pays off and when it doesn't, using the same lens as the rest of the course: execution is mostly about moving bytes efficiently, and an accelerator only helps if bytes can reach it faster than the CPU could process them itself.

GPU architecture for query people

A GPU trades the CPU's few fast, clever cores for thousands of simple ones and memory with enormous bandwidth. For analytics, the bandwidth matters more than the compute: most query operators do little arithmetic per byte, so they run at the speed memory can deliver data.

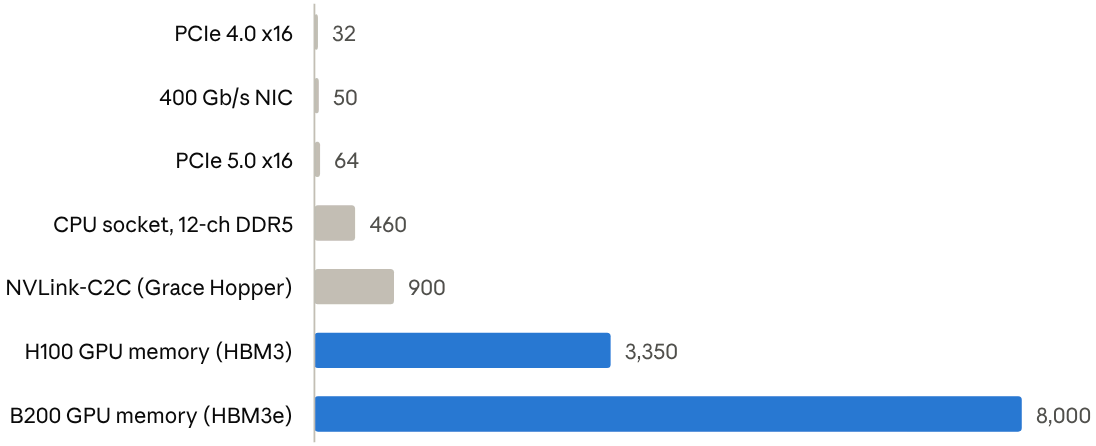
The parts that matter

- **Streaming multiprocessors (SMs)**, each running many threads in groups of 32 called **warps**. A warp executes one instruction across its 32 threads in lockstep (SIMT), much like a 32-wide SIMD vector, so threads in a warp that take different branches serialize (*warp divergence*), the GPU version of a branch misprediction (Module 2).
- **Latency hiding by oversubscription**: instead of caches and out-of-order execution, a GPU keeps tens of thousands of threads in flight and switches warps whenever one waits on memory. Memory-level parallelism (Module 2) is the default mode, not an optimization.

- **Memory hierarchy**: registers, a small fast **shared memory** per SM (software-managed scratchpad, ~100 to 200 KB), an L2 cache of tens of MB, and **HBM** device memory of 80 to 192 GB at several TB/s.
- **Coalescing**: memory is fastest when the 32 threads of a warp read 32 adjacent values. A columnar layout is exactly that: GPUs and column stores are a natural fit.

GPU memory is 50 to 125× faster than the PCIe link that feeds it

peak bandwidth, GB/s



approximate peak bandwidths from vendor specifications; PCIe and NIC per direction

The chart is the whole module in one picture: inside the GPU, data moves at several TB/s; a typical PCIe link into the GPU carries a small fraction of that. Where the data lives decides everything that follows.

The data-movement problem

The classic critique of GPU databases is in the title of Gregg and Hazelwood's 2011 paper: "Where is the data? Why you cannot debate CPU vs. GPU performance without the answer". A kernel that runs 20× faster is no help if moving its input to the GPU takes longer than the CPU would need for the whole job.

Three scenarios

1. **Data already resident in GPU memory.** Operators run at HBM speed, and speedups over a CPU are roughly the ratio of memory bandwidths, often an order of magnitude. This is the case for data cached on GPUs across queries, or for data produced on the GPU (as in ML pipelines).

2. **Data in host memory, shipped over PCIe.** The GPU now processes at most ~64 GB/s (PCIe 5.0), while the CPU can scan its *own* memory at hundreds of GB/s. For simple scans and filters, **the CPU wins**. The GPU only wins when an operator does much more work per byte than it moves: big joins, heavy aggregations, sorts, expensive expressions.
3. **Data on SSDs or remote storage.** Both processors are limited by I/O. The GPU helps only if the CPU couldn't keep up with decompression and decoding at that I/O rate. NVIDIA's GPUDirect Storage lets NVMe drives write straight into GPU memory, skipping a copy through host memory.

Ways around the link

- **Keep hot data on the GPU**: cache columns in HBM across queries and route queries to where data lives.
- **Ship compressed data**: send encoded or compressed bytes over PCIe and decompress on the GPU, multiplying effective link bandwidth by the compression ratio.
- **Shrink before shipping**: filter and project on the CPU or in storage, and ship only surviving rows and columns (Module 5's pruning, again).
- **Overlap**: pipeline transfers and kernels on separate streams, so the link and the compute are busy at the same time.
- **Faster, coherent links**: Grace Hopper's NVLink-C2C lets the GPU read CPU memory at hundreds of GB/s, coherently. Research such as Lutz et al. ("Pump Up the Volume", SIGMOD 2020, on an earlier NVLink generation) showed this lets GPUs join data far larger than their own memory efficiently.
- **Multi-GPU**: GPUs in one server exchange data over NVLink at hundreds of GB/s, so a shuffle *between* GPUs (Module 6) can be cheaper than getting data onto them in the first place.

Relational operators on GPUs

The algorithms are the ones from Module 4, re-expressed for tens of thousands of threads. Two building blocks recur everywhere: **parallel prefix sum** (scan), which tells each thread where to write its output, and **atomic operations** on device memory, which let threads build shared structures.

Filter: stream compaction

Each thread evaluates the predicate on one value and writes a 0 or 1; a parallel prefix sum over those flags gives every surviving row its output position; then survivors are scattered into a dense output. It is the GPU equivalent of a selection vector (Module 2), except that compacting is cheap enough to do eagerly.

Hash join

- **Build**: threads insert keys into an open-addressing table in HBM using atomic compare-and-swap.
- **Probe**: one thread per probe key. Random accesses into a large table are tolerable because thousands of warps hide each other's memory latency.
- **Partitioned joins pay off more on GPUs than on CPUs**: when a build partition fits in shared memory or L2, probes avoid HBM entirely, so radix-partitioning first (Module 4's partitioned join) is often the winning design.
- **Unknown output size**: threads cannot simply append. Kernels either run twice (count matches, prefix-sum the counts, then write) or reserve output space with an atomic counter.

Aggregation

- **Few groups** (the 200 countries): thousands of threads updating the same 200 counters contend on atomics. Each thread block aggregates privately in shared memory first, then merges once. This is partial/final aggregation (Module 1) inside one GPU.
- **Many groups**: a hash table in HBM with atomic updates, or **sort-based** aggregation: radix-sort by key, then a segmented reduction over runs of equal keys.

Sort

Radix sort is the GPU's strongest primitive: libraries such as CUB sort billions of keys per second. Normalized keys (Module 4) make multi-column sorts radix-sortable.

The awkward parts

- **Strings**: variable lengths leave some threads with far more work than others (load imbalance), and LIKE or regex matching branches heavily (warp divergence). Arrow-style offsets-plus-bytes layouts and dictionary encoding help: compare dictionary codes, not strings.

- **Decoding and decompression**: Parquet decoding and codecs like Snappy, LZ4 and ZSTD now run on GPUs (cuDF's readers, NVIDIA's nvCOMP), which matters for the "ship compressed" strategy above.
- **Data larger than GPU memory**: partition and stream it through, spilling to host memory: the grace hash join and external sort of Module 3, with HBM in place of RAM and host memory in place of disk.

Execution models on GPUs

Module 2's debate replays on GPUs with different constants. Launching a kernel costs microseconds, and writing an intermediate result to HBM and reading it back costs bandwidth, the GPU's scarcest resource relative to its compute. So the key question is how much of a pipeline runs in one kernel.

Operator at a time

One kernel (or a few) per operator, each materializing its output in HBM: filter, then project, then aggregate. This is the model of dataframe libraries like cuDF and of early GPU databases, the GPU equivalent of MonetDB's column-at-a-time model (Module 2). It is simple and composable, but every intermediate makes a round trip through HBM.

Fused or compiled kernels

Generate one kernel per pipeline, so values stay in registers across operators: HyPer's data-centric compilation (Module 2) on a GPU. HeavyDB (formerly MapD and OmniSci) compiles queries to GPU code through LLVM. It is the fastest model, with the familiar costs: compilation latency and a much harder engine to build and debug.

Tile-based processing

A middle path, popularized by the Crystal library (Shanbhag, Madden and Yu, SIGMOD 2020): each thread block loads a tile of rows into registers or shared memory and applies a sequence of *block-wide primitives* (load, predicate, project, aggregate) before writing anything back. That fuses a pipeline without generating code. The same paper's measurements found GPU speedups for analytical queries tracked the ratio of GPU to CPU memory bandwidth, which supports the bandwidth-first view of this module.

Batch size and memory

- **Much larger batches**: millions of rows per batch rather than ~1,000, to give tens of thousands of threads enough work and amortize kernel launches. That raises memory pressure and weakens pipelining, so GPU engines lean on partitioning and spilling more than CPU engines do.
- **Pool allocators**: device allocation is slow, so engines allocate from pools; RAPIDS uses its RMM memory manager.
- **Hybrid plans**: operators a GPU engine doesn't support fall back to the CPU, with a transfer at each boundary. This is the same fallback cost as Gluten's (Module 8), but every crossing is now a PCIe transfer.

Systems

The current generation of GPU query processing is built from the same composable pieces as Module 8: existing front ends and optimizers stay, and the GPU replaces only the execution layer, usually through NVIDIA's libcudf.

- **cuDF (RAPIDS)**: NVIDIA's open-source GPU dataframe library: a C++ core (libcudf) of columnar, Arrow-compatible relational operators (joins, aggregations, sorts, shuffles) and GPU readers for Parquet and ORC, plus a pandas-like Python API and a drop-in mode for existing pandas code. It is the building block most systems below share.
- **RAPIDS Accelerator for Apache Spark**: a Spark plugin that replaces supported physical operators with GPU implementations and falls back to the CPU for the rest. Architecturally it is Gluten's design with a GPU backend.
- **Velox with cuDF**: IBM and NVIDIA are adding a cuDF backend to Velox, so Presto C++ and Spark (via Gluten) can run on GPUs without changes above the execution layer. On each Presto worker, the backend rewrites the plan received from the coordinator into GPU operators: TableScan, HashJoin, HashAggregation, FilterProject and more. Distributed queries use a UCX-based exchange over NVLink, which NVIDIA reported as over 6× faster than the HTTP exchange on an eight-GPU DGX A100 node. NVIDIA still labels it experimental.
- **Sirius** (University of Wisconsin–Madison and NVIDIA, CIDR 2026): a GPU-native SQL engine that plugs into existing databases through Substrait plans, built on libcudf. With DuckDB on one node it reported 8.3× and 7.4× better cost efficiency on TPC-H and ClickBench, and up to 12.5× speedup inside the distributed Apache Doris. It falls back to DuckDB's CPU engine for unsupported queries.

- **Theseus** (Voltron Data): a distributed engine designed GPU-first, which uses Velox as its CPU backend.
- **HeavyDB** (formerly MapD and OmniSci): one of the earliest GPU databases, keeping hot data in GPU memory and compiling queries with LLVM, aimed at interactive visual analytics.

The pattern to notice: a decade ago, GPU databases tried to be complete databases and struggled to match mature CPU systems feature for feature. Today's systems reuse a mature front end (Presto, Spark, DuckDB) and swap only the execution layer, which is exactly the composability argument of Velox and Axiom.

How Velox integrates GPUs

Velox's GPU support lives under `velox/experimental/` and is explicitly experimental. Its design follows from Module 3: hosts hand Velox a plan, Velox cuts it into pipelines and runs each as Drivers, and the GPU work plugs in **underneath** that structure, so Presto, Spark/Gluten and the planner above them don't change.

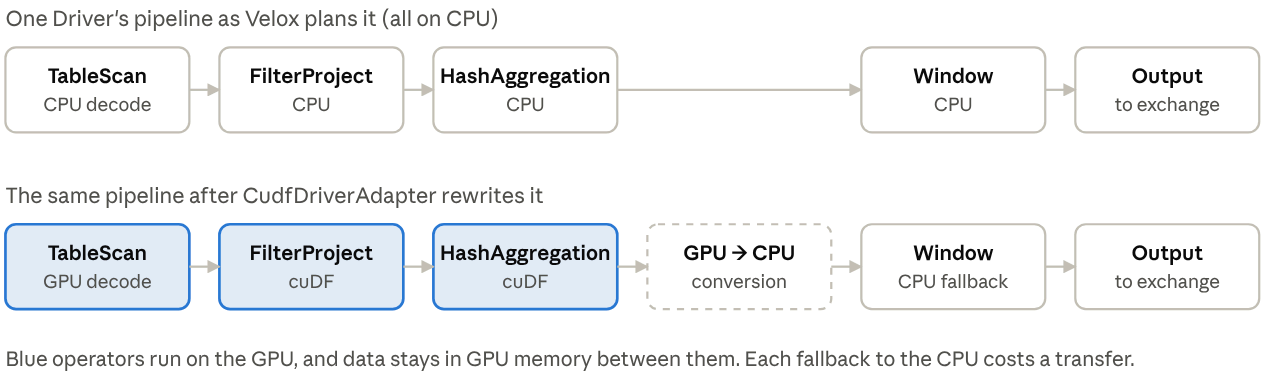
Four components

Component	Role	Approach
cuDF backend	Execute operators on the GPU	Swap Velox operators one-to-one for libcudf-backed versions
Wave	Execute whole pipelines on the GPU	Generate and JIT-compile fused CUDA kernels for a run of operators
Breeze	Portable GPU primitives	Block- and device-level load/store, reduce, scan and radix sort, written once for CUDA, HIP, OpenCL, SYCL, Metal and OpenMP
UCX exchange	GPU-to-GPU shuffle between workers	Move device buffers directly between GPUs, without staging through host memory

The integration point: DriverAdapter

Both execution backends use the same hook. When a Task builds its Drivers (Module 3's LocalPlanner step), a `DriverAdapter` may inspect each Driver's operator chain and rewrite it, replacing CPU operators with GPU ones. Because the rewrite happens at operator boundaries, the plan, the pipelines, the Driver loop and the blocking and scheduling machinery of Module 2 all stay as they are; only the code behind each operator changes.

The adapter swaps operators for GPU ones and converts at boundaries



Blue operators run on the GPU, and data stays in GPU memory between them. Each fallback to the CPU costs a transfer.

cuDF backend · an illustrative pipeline before and after the rewrite

The cuDF backend: operator by operator

`CudfDriverAdapter` replaces each supported operator with a libcudf-backed equivalent and inserts **conversion operators** where a GPU operator meets a CPU one. Unsupported operators simply stay on the CPU. Coverage today includes filter and project, hash and streaming aggregation, hash and nested-loop joins, order by, top-n and limit. If every operator in a pipeline has a GPU version, its data never leaves the GPU.

Details from the NVIDIA and IBM papers and posts:

- **A GPU vector type.** GPU operators pass a cuDF-backed vector that carries its data and a **CUDA stream**, so many Drivers (Velox's usual parallelism) can keep issuing overlapping GPU work without blocking their CPU threads. Velox's Driver model doubles as the GPU's scheduler.
- **Expressions.** Velox typed expressions (Module 3) are translated into cuDF expression trees, preferring cuDF's fused AST evaluation and falling back to individual functions for unsupported operations.

- **Scans.** TableScan was extended to hand file data to cuDF's GPU I/O, decompression and Parquet decoding, so data crosses PCIe still compressed (this module's "ship compressed" strategy).
- **One GPU per worker**: each GPU is driven by its own Velox worker.

The trade-off: one-to-one replacement does less fusion across operators than generated kernels, but it inherits libcudf's breadth and maturity, which is why the Velox docs call it the most complete option today.

Wave: whole pipelines as generated kernels

Wave lowers a contiguous run of supported operators, such as scan, filter, project and aggregate, into an intermediate representation, generates CUDA C++, compiles it at query time with NVRTC, and replaces the run with a single source operator that launches the kernels. Compiled modules are cached and built in the background, and an unsupported operator ends the offloaded run. Wave also decodes encoded columns, dictionaries included, on the GPU.

This is the compiled, data-centric model of Module 2 applied to GPUs, with the highest performance ceiling and still narrow coverage. The two backends mirror the vectorized-versus-compiled debate exactly: cuDF is operator-at-a-time with a mature library, Wave is fused code generation.

Moving data between GPUs

The UCX exchange replaces the inter-worker exchange (PartitionedOutput on one side, Exchange on the other, as in Module 3) with direct device-to-device transfers over NVLink or GPUDirect RDMA. It works with either backend and is chosen per plan node, defaulting to the normal host-memory exchange, so it can be adopted gradually. NVIDIA reported over 6× faster Presto queries with it than with the HTTP exchange on an eight-GPU DGX A100 node.

What it means for hosts

- **Presto C++**: workers receive normal plan fragments from the Java coordinator; the cuDF backend rewrites them on the worker. The front end, optimizer and coordinator are unchanged.
- **Spark via Gluten**: hybrid CPU and GPU execution, offloading only the stages that benefit, such as join-heavy ones.

- **A caveat from the docs**: CPU fallback makes partial support invisible, which is convenient but can hide where a query actually runs. Check plans and metrics, not just speed.

Beyond GPUs: other accelerators

Most other accelerators in data systems share one idea: move simple, byte-heavy work (decompression, filtering, encryption, data movement) off the CPU, ideally to where the data already is.

Accelerator	What it offloads	Examples
CPU vector and matrix units	Vectorized primitives (Module 2); research maps some relational operators onto matrix units	AVX-512 everywhere; research on tensor-core and AMX-based operators
On-chip analytics engines	Decompression, scanning and filtering of compressed columns	Intel's In-Memory Analytics Accelerator (IAA) in recent Xeons, used for example by ClickHouse's `deflate_qpl` codec; earlier Oracle SPARC M7's Data Analytics Accelerator
FPGAs	Custom pipelines for filtering, aggregation, decompression near storage or the network	AWS AQUA for Redshift; ETH Zürich's research on FPGA-based database operators
SmartNICs and DPUs	Shuffle, RDMA transfers, compression and encryption on the network path	Offloaded transport for disaggregated shuffle and storage (Module 6)
Computational storage	Filters and projections inside storage devices or servers	Storage-side pushdown; the software analog is Capacitor's embedded evaluator (Module 7)
CXL memory	Memory capacity expansion and pooling across servers	Larger in-memory working sets; memory disaggregation in hardware, the counterpart of BigQuery's shuffle tier

The recurring obstacle is the same as for GPUs: an accelerator only helps if routing data through it costs less than the work it saves. On-path accelerators (on the CPU die, on the NIC, in the storage server) avoid an extra data movement, which is why they keep reappearing even though general-purpose CPUs keep getting faster.

When GPUs win and when they lose

The question is never "is the GPU faster" but "is this workload limited by something the GPU has more of, and can the data get there cheaply?"

GPUs tend to win	GPUs tend to lose
Data already in GPU memory, or cached there across queries	Data streamed over PCIe for a simple scan or filter
Large joins, aggregations and sorts: much work per byte moved	Small, selective or point queries: kernel launch and transfer overheads dominate
Numeric, fixed-width columns; dictionary-encoded strings	Heavy, irregular string processing; regular expressions
Large batches and wide parallelism	Many tiny concurrent queries
Pipelines next to ML training, where GPUs and data already meet (Module 10)	Data on remote object storage, where both CPU and GPU wait on I/O
Coherent CPU-GPU systems (Grace Hopper) and multi-GPU nodes with NVLink	Features the GPU engine lacks, forcing frequent CPU fallback

Economics

A GPU server costs several times as much per hour as a CPU server, and GPUs are in heavy demand for AI. So the fair comparison is **cost per query at equal hardware rental cost**, which is how the Sirius authors report results, not raw speedup. Two shifts make that comparison more favorable than it used to be: GPU memory capacity keeps growing, and in AI-focused data centers GPUs are already provisioned, so analytics can use spare capacity or run next to the data the models consume.

A checklist

1. Where does the data live, and what is the bandwidth from there to the GPU?

2. How much work is done per byte moved: a simple scan, or a large join?
3. Does the working set fit in aggregate GPU memory, or will it spill?
4. Which operators and functions does the GPU engine support, and how often would it fall back?
5. What is the cost per query, not just the speedup?

Self-test and reading list

1. Why does memory bandwidth, not compute, determine most GPU speedups for analytical queries?
2. Describe the three data-location scenarios. In which one does a CPU beat a GPU on a simple scan, and why?
3. Name four ways to reduce the cost of getting data onto a GPU.
4. How does stream compaction implement a filter, and what plays the role of a selection vector?
5. Why does aggregation into a few groups contend on GPUs, and how does shared memory fix it?
6. Why are partitioned hash joins more attractive on GPUs than on CPUs?
7. Compare operator-at-a-time, fused-kernel and tile-based execution on GPUs.
8. Why do GPU engines use much larger batches, and what does that cost?
9. How does the Velox cuDF backend let Presto run on GPUs without changing Presto's front end?
10. Give three workload traits that favor GPUs and three that favor CPUs.
11. Name one non-GPU accelerator and the byte-heavy work it offloads.

Reading list:

1. Gregg and Hazelwood, "Where is the Data? Why You Cannot Debate CPU vs. GPU Performance Without the Answer", ISPASS 2011.
2. Shanbhag, Madden, Yu, "A Study of the Fundamental Performance Characteristics of GPUs and CPUs for Database Analytics", SIGMOD 2020 (Crystal).
3. Lutz et al., "Pump Up the Volume: Processing Large Data on GPUs with Fast Interconnects", SIGMOD 2020.
4. Yogatama et al., "Rethinking Analytical Processing in the GPU Era", CIDR 2026 (Sirius).

5. NVIDIA and IBM, "Accelerating Large-Scale Data Analytics with GPU-Native Velox and NVIDIA cuDF" (NVIDIA technical blog), and "Accelerating Presto with GPUs" (VLDB 2026 ADMS workshop).
6. Sioulas et al., "Hardware-Conscious Hash-Joins on GPUs", ICDE 2019.

This concludes the course: eleven modules from SQL text to GPUs.